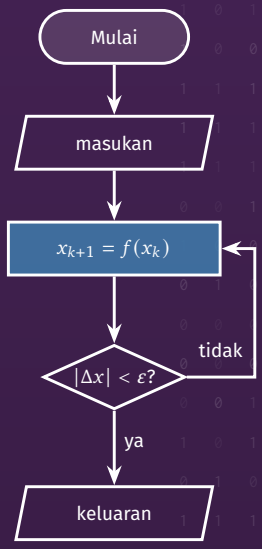
CATATAN KULIAH

Algoritma dan Pemrograman

untuk Teknik Fisika



RWG, IWF, LTD, NWF

Program Studi Teknik Fisika • 2026

DRAF

Algoritma dan Pemrograman

untuk Teknik Fisika

Catatan Kuliah

RWG, IWF, LTD, NWF

Program Studi Teknik Fisika

2026

Kata Pengantar

Buku ini disusun dari catatan kuliah *Algoritma dan Pemrograman* untuk mahasiswa tahun pertama Teknik Fisika. Tujuan utamanya adalah membangun cara berpikir yang memungkinkan persoalan fisika dan rekayasa diterjemahkan menjadi langkah-langkah komputasi yang jelas, benar, dan efisien. Selain itu, tentunya diiringi dengan pengajaran sintaks bahasa pemrograman seperti C dan Python. Sepanjang buku ini kita akan berulang kali menggunakan alur

$$\text{masalah} \longrightarrow \text{model} \longrightarrow \text{algoritma} \longrightarrow \text{program} \longrightarrow \text{hasil}.$$

Program hanyalah salah satu tahap. Sebelum menulis kode, kita perlu memahami persoalan fisisnya, memilih model yang tepat, dan merancang algoritma. Setelah program berjalan, hasilnya masih harus diperiksa dengan perhitungan manual, satuan, orde besaran, dan penalaran fisika.

Bahasa C digunakan pada sebagian besar buku karena sintaksnya relatif kecil, hubungannya dengan representasi data dan memori cukup terlihat, serta banyak digunakan dalam instrumentasi, sistem tertanam, dan komputasi ilmiah. Kuliah terakhir memperkenalkan Python beserta NumPy dan Matplotlib sebagai alat analisis dan visualisasi data. Penekanan buku tetap pada algoritma: gagasan yang sama dapat diekspresikan dalam bahasa apa pun.

Materi buku ini terdiri atas sembilan kuliah dan satu lampiran.

1. *Pengenalan algoritma, komputer, dan representasi data*: cara kerja komputer, kompilasi, sistem bilangan, serta representasi bilangan bulat dan *floating-point*.
2. *Berpikir algoritmik dan perancangan algoritma*: spesifikasi, dekomposisi, diagram alir, *pseudocode*, penelusuran, kebenaran, dan kompleksitas.
3. *Dasar-dasar pemrograman dalam bahasa C*: struktur program, tipe data, operator, logika Boolean, serta masukan dan keluaran.
4. *Percabangan dan pengulangan*: `if`, `switch`, `while`, `do-while`, dan `for`.
5. *Pola algoritma iteratif*: pencacah, akumulator, minimum–maksimum, *sentinel*, validasi, tabel komputasi, dan kriteria penghentian.
6. *Larik, fungsi, dan modularisasi program*, termasuk pengantar *pointer* dan *string*.
7. *Data terstruktur dan pengolahan berkas*: `struct`, berkas teks, dan format CSV.
8. *Integrasi pemrograman C*: satu program utuh untuk analisis data eksperimen, dari rancangan sampai pengujian.

9. *Python untuk komputasi ilmiah*: NumPy, Matplotlib, dan alur kerja gabungan C–Python.

Lampiran A menjelaskan cara memasang GCC, Visual Studio Code, dan Python.

Setiap kuliah ditutup dengan *Cek pemahaman*, sepuluh soal *Latihan*, dan *Rangkuman*. Semua program contoh telah dikompilasi dan dijalankan; keluaran yang ditampilkan dalam kotak gelap adalah keluaran sebenarnya, dan masukan yang diketik pengguna ditandai dengan warna biru muda. Pembaca sangat dianjurkan mengetik, mengompilasi, dan menjalankan sendiri setiap contoh, serta membiasakan diri memprediksi keluaran sebelum program dijalankan:

prediksi → jalankan → bandingkan

Satu catatan tentang penulisan bilangan: agar selaras dengan kode program dan keluarannya, buku ini menggunakan **titik** sebagai pemisah desimal (misalnya 3.14), bukan koma seperti lazimnya dalam teks berbahasa Indonesia.

Penulis menyadari bahwa draf ini masih jauh dari sempurna. Kritik, saran, dan laporan kesalahan dari pembaca akan sangat membantu penyempurnaan edisi berikutnya.

2026

RWG, IWF, LTD, NWF

Daftar Isi

Kata Pengantar **v**

Kuliah 1 Pengenalan Algoritma, Komputer, dan Representasi Data **1**
1.1 Komputer dan algoritma 1
1.2 Kode sumber, kompilasi, dan eksekusi 3
1.3 Bit, byte, dan sistem bilangan 4
1.4 Representasi bilangan bulat dan karakter 5
1.5 Representasi bilangan real 7
1.6 Program C pertama untuk perhitungan fisika 9
1.7 Program sebagai representasi model 12
1.8 Cek pemahaman 13
1.9 Latihan 14
1.10 Rangkuman 15

Kuliah 2 Berpikir Algoritmik dan Perancangan Algoritma **17**
2.1 Menyusun algoritma untuk masalah fisika 17
2.2 Dekomposisi masalah 19
2.3 Kalimat berurutan dan diagram alir 20
2.4 *Pseudocode* 22
2.5 Menelusuri dan memeriksa algoritma 24
2.6 Implementasi algoritma dalam bahasa C 26
2.7 Efisiensi algoritma 27
2.8 Studi kasus: pemantauan temperatur 28
2.9 Cek pemahaman 33
2.10 Latihan 33
2.11 Rangkuman 34

Kuliah 3 Dasar-dasar Pemrograman dalam Bahasa C **35**
3.1 Struktur dasar program C 35
3.2 Variabel, tipe data, dan konstanta 36
3.3 Operator dan ekspresi aritmetika 39
3.4 Operator relasional dan logika Boolean 40
3.5 Masukan dan keluaran 42
3.6 Studi kasus: model fisika sederhana 44
3.7 Kesalahan umum dan kebiasaan yang baik 46
3.8 Cek pemahaman 48
3.9 Latihan 48

3.10 Rangkuman 49

Kuliah 4 Percabangan dan Pengulangan **51**
4.1 Aliran kontrol dalam program 51
4.2 Percabangan `if` dan `if-else` 51
4.3 Percabangan banyak arah dan bersarang 54
4.4 `switch` dan operator kondisi 56
4.5 Pengulangan `while` dan `do-while` 57
4.6 Pengulangan `for` 60
4.7 Mengendalikan pengulangan dan kesalahan umum 62
4.8 Percabangan di dalam pengulangan 64
4.9 Cek pemahaman 66
4.10 Latihan 66
4.11 Rangkuman 67

Kuliah 5 Pola Algoritma Iteratif **69**
5.1 Keadaan dan komponen iterasi 69
5.2 Pencacah dan akumulator 70
5.3 Minimum, maksimum, dan penggabungan pola 71
5.4 *Sentinel* dan validasi masukan 73
5.5 Tabel komputasi 76
5.6 Pembaruan iteratif dan kriteria penghentian 78
5.7 Studi kasus: analisis getaran mesin 82
5.8 Kesalahan umum dan efisiensi 85
5.9 Cek pemahaman 86
5.10 Latihan 86
5.11 Rangkuman 87

Kuliah 6 Larik, Fungsi, dan Modularisasi Program **89**
6.1 Larik satu dimensi 89
6.2 Larik dua dimensi 92
6.3 Fungsi 93
6.4 Modularisasi program 95
6.5 Fungsi yang menerima larik 96
6.6 Pengantar *pointer* 99
6.7 *String* sebagai larik karakter 101
6.8 Kesalahan umum dan kebiasaan yang baik 103
6.9 Cek pemahaman 104
6.10 Latihan 104
6.11 Rangkuman 105

Kuliah 7 Data Terstruktur dan Pengolahan Berkas **107**
7.1 Data terstruktur dengan `struct` 107
7.2 Larik `struct` dan fungsi 108
7.3 Berkas teks: membuka, menulis, dan menutup 110
7.4 Membaca berkas teks 112
7.5 Format CSV 113

7.6 Memisahkan masukan-keluaran dan analisis . . . 115
7.7 Studi kasus: pengolahan data listrik . . . 117
7.8 Kesalahan umum dan kebiasaan yang baik . . . 121
7.9 Cek pemahaman . . . 121
7.10 Latihan . . . 122
7.11 Rangkuman . . . 122

Kuliah 8 Integrasi Pemrograman C untuk Kasus Teknik Fisika 125
8.1 Persoalan: eksperimen gerak satu dimensi . . . 125
8.2 Rancangan program . . . 126
8.3 Implementasi bertahap: membaca dan memvalidasi data . . . 128
8.4 Implementasi bertahap: analisis . . . 129
8.5 Program utuh . . . 131
8.6 Pengujian program . . . 134
8.7 Memeriksa hasil secara fisis . . . 136
8.8 Pola umum dan kesalahan umum . . . 137
8.9 Cek pemahaman . . . 138
8.10 Latihan . . . 138
8.11 Rangkuman . . . 139

Kuliah 9 Python untuk Komputasi Ilmiah 141
9.1 Mengapa Python? . . . 141
9.2 Sintaks dasar: perbandingan dengan C . . . 142
9.3 Fungsi, *list*, dan berkas . . . 144
9.4 NumPy: larik numerik . . . 146
9.5 Membaca data dan menghitung selisih . . . 147
9.6 Matplotlib: grafik ilmiah . . . 149
9.7 Studi kasus: data pendinginan dan gelombang . . . 150
9.8 C dan Python dalam satu alur kerja . . . 153
9.9 Cek pemahaman . . . 155
9.10 Latihan . . . 155
9.11 Rangkuman . . . 156

A Menyiapkan Lingkungan Pemrograman 159
A.1 GCC di Windows melalui MSYS2 . . . 159
A.2 GCC melalui WSL . . . 160
A.3 Visual Studio Code . . . 161
A.4 Python dan pustaka ilmiah . . . 161

KULIAH

1

Pengenalan Algoritma, Komputer, dan Representasi Data

1.1 Komputer dan algoritma

Komputer merupakan salah satu alat kerja utama dalam sains dan rekayasa. Dalam Teknik Fisika, komputer digunakan untuk membaca data sensor, mengolah hasil eksperimen, menyelesaikan persamaan, menjalankan simulasi, mengendalikan perangkat, membuat visualisasi, dan menganalisis data dalam jumlah besar.

Sebagai contoh sederhana, misalkan sebuah eksperimen menghasilkan N data temperatur $T_1, T_2, \ldots, T_N$ dan kita ingin mengetahui temperatur rata-ratanya,

$$\overline{T} = \frac{1}{N} \sum_{i=1}^{N} T_i = \frac{1}{N} \left(T_1 + T_2 + \cdots + T_N \right) .$$

Bagi kita, persamaan tersebut sudah jelas. Komputer, sebaliknya, tidak memahami simbol $\sum$. Komputer membutuhkan instruksi yang jauh lebih eksplisit, misalnya:

1. siapkan sebuah variabel untuk menyimpan jumlah temperatur dan beri nilai awal nol;
2. baca satu data temperatur;
3. tambahkan temperatur tersebut ke jumlah;
4. ulangi langkah 2 dan 3 sampai seluruh data selesai dibaca;
5. bagi jumlah tersebut dengan banyaknya data;
6. tampilkan hasilnya.

Urutan langkah yang terdefinisi dengan jelas seperti ini disebut **algoritma**. **Pemrograman** adalah proses menerjemahkan algoritma ke dalam suatu bahasa pemrograman; kode tersebut kemudian diterjemahkan lebih lanjut menjadi instruksi mesin yang dijalankan oleh komputer.

Sepanjang buku ini kita akan sering menggunakan alur berikut:

masalah ⟶ model ⟶ algoritma ⟶ program ⟶ hasil

Bagi mahasiswa Teknik Fisika, bagian awal alur ini sangat penting. Kita jarang memulai dari program; kita memulai dari persoalan fisika atau rekayasa, merumuskan modelnya, lalu menyusun algoritma.

Sebagai contoh, energi kinetik sebuah benda diberikan oleh $E_k = \frac{1}{2}mv^2$. Jika massa m dan kecepatan v diketahui, prosedurnya adalah membaca m dan v, menghitung v^2, mengalikannya dengan m dan $\frac{1}{2}$, lalu menampilkan hasilnya. Dalam bahasa C bagian perhitungannya dapat ditulis

```
energy = 0.5 * mass * velocity * velocity;
```

sedangkan dalam Python

```
energy = 0.5 * mass * velocity**2
```

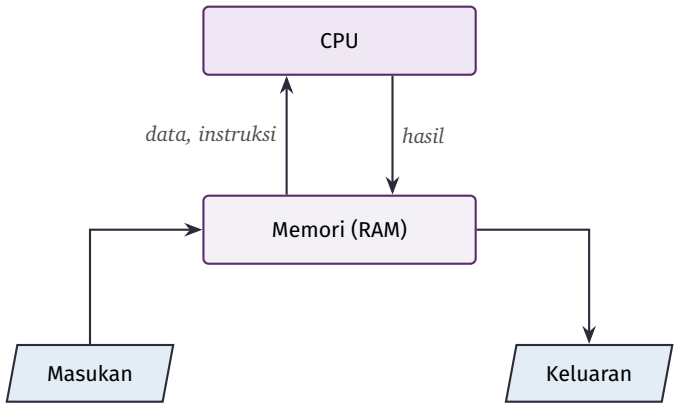
Sintaksnya berbeda, tetapi gagasan komputasinya sama. Inilah perbedaan penting antara keduanya: **algoritma** adalah prosedur penyelesaian masalah, sedangkan **program** adalah implementasi algoritma dalam suatu bahasa pemrograman. Belajar pemrograman, dengan demikian, bukan sekadar menghafal sintaks C atau Python, tetapi belajar merumuskan persoalan menjadi langkah komputasi yang jelas, benar, dan efisien.

1.1.1 Cara kerja komputer secara sederhana

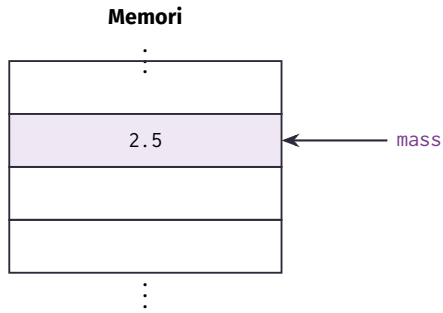
Komputer modern sangat kompleks, tetapi untuk keperluan awal kita cukup memandangnya sebagai sistem yang terdiri atas

- prosesor atau *central processing unit* (CPU) yang menjalankan instruksi;
- memori utama atau *random access memory* (RAM) yang menyimpan data dan instruksi yang sedang digunakan;
- media penyimpanan, seperti SSD, yang menyimpan data secara lebih permanen;
- perangkat masukan (*input*) dan keluaran (*output*).

Aliran informasi di antara bagian-bagian tersebut dapat digambarkan sebagai berikut.



Misalkan dalam bahasa C kita menulis `double mass = 2.5;`. Secara konseptual, komputer menyediakan satu lokasi di memori untuk menyimpan nilai `2.5`, dan nama `mass` menjadi label bagi lokasi tersebut.



Pada kenyataannya, komputer tidak menyimpan tulisan `2.5` seperti yang kita lihat di layar, tetapi pola bit yang mewakili nilai tersebut. Bagaimana pola bit itu disusun akan kita bahas pada Subbab 1.3 dan seterusnya.

CPU hanya mampu melakukan operasi dasar: membaca nilai dari memori, menyimpan nilai ke memori, menjumlahkan, mengurangkan, membandingkan dua nilai, dan menentukan instruksi berikutnya yang akan dijalankan. Satu baris seperti `energy = 0.5 * mass * velocity * velocity;` pada akhirnya harus diterjemahkan menjadi rangkaian operasi dasar semacam itu. CPU tidak memahami sintaks bahasa C secara langsung; diperlukan proses penerjemahan.

1.2 Kode sumber, kompilasi, dan eksekusi

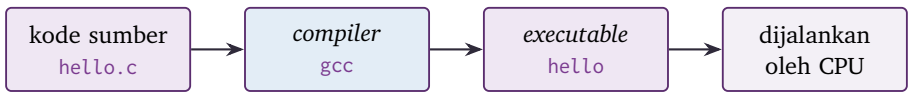
Program yang kita tulis dalam bahasa C disebut **kode sumber** (*source code*). Misalkan kita membuat berkas `hello.c` yang berisi

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, Teknik Fisika!\n");

    return 0;
}
```

Berkas ini dapat dibaca manusia, tetapi tidak dapat langsung dijalankan oleh CPU yang hanya memahami kode mesin. Penerjemahnya adalah program lain yang disebut ***compiler***. Salah satu *compiler* C yang paling banyak digunakan adalah GCC.



Pada Linux atau WSL, program tersebut dikompilasi dan dijalankan dengan

```
gcc -Wall -Wextra hello.c -o hello
./hello
```

dan menghasilkan

```
Hello, Teknik Fisika!
```

Pada Windows dengan GCC dari MSYS2, berkas hasil kompilasi biasanya diberi akhiran `.exe` dan dijalankan dengan `.\hello.exe`. Opsi `-Wall` dan `-Wextra` meminta *compiler* menampilkan lebih banyak peringatan; kita akan selalu menggunakannya.

Langkah-langkah memasang GCC, Visual Studio Code, dan WSL dijelaskan secara rinci pada Lampiran A. Penulis menyarankan mahasiswa membiasakan diri bekerja di lingkungan Linux, baik secara langsung maupun melalui *Windows Subsystem for Linux* (WSL).

Untuk saat ini kita belum perlu memahami setiap baris program `hello.c`; sintaks C akan dibahas secara sistematis pada Kuliah 3. Yang perlu dipahami sekarang adalah alurnya:

$$\boxed{\text{kode sumber} \rightarrow \textit{compiler} \rightarrow \textit{executable} \rightarrow \text{eksekusi}}$$

1.3 Bit, byte, dan sistem bilangan

Bagaimana komputer menyimpan bilangan seperti 42, −17, atau 3.1415926535? Untuk menjawabnya, kita perlu mengenal **bit**, yaitu satuan informasi yang hanya mempunyai dua keadaan: 0 atau 1. Secara elektronik, kedua keadaan tersebut diwujudkan oleh dua rentang kondisi fisik yang berbeda, misalnya dua tingkat tegangan. Detail elektroniknya tidak diperlukan di sini; yang penting, pada tingkat rendah semua data direpresentasikan sebagai kombinasi nol dan satu.

Delapan bit membentuk satu **byte**, misalnya `01001101`. Karena setiap bit mempunyai dua kemungkinan nilai, n bit dapat membentuk

$$\boxed{2^n}$$

pola yang berbeda. Satu bit memberi 2 pola, dua bit memberi $2^2 = 4$ pola (`00`, `01`, `10`, `11`), dan delapan bit memberi $2^8 = 256$ pola. Untuk 16 bit terdapat $2^{16} = 65\,536$ pola, sedangkan untuk 32 bit terdapat $2^{32} = 4\,294\,967\,296$ pola. Pola-pola ini kemudian dipetakan ke nilai yang ingin kita simpan. Prinsip sederhana inilah yang menjelaskan mengapa setiap tipe data di komputer mempunyai rentang nilai yang terbatas.

1.3.1 Sistem desimal dan biner

Dalam kehidupan sehari-hari kita menggunakan sistem desimal (basis sepuluh). Bilangan 572 sebenarnya berarti

$$572 = 5(10^2) + 7(10^1) + 2(10^0) = 5(100) + 7(10) + 2(1).$$

Sistem biner bekerja dengan prinsip yang sama, tetapi dengan basis dua dan digit 0 serta 1 saja. Subskrip menunjukkan basis yang digunakan, misalnya

$$1101_2 = 1(2^3) + 1(2^2) + 0(2^1) + 1(2^0) = 8 + 4 + 0 + 1 = 13_{10},$$
$$10110_2 = 1(2^4) + 0(2^3) + 1(2^2) + 1(2^1) + 0(2^0) = 16 + 0 + 4 + 2 + 0 = 22_{10}.$$

Untuk mengubah bilangan desimal bulat menjadi biner, kita dapat membaginya berulang kali dengan dua dan mencatat sisanya. Untuk 13_{10},

$$\begin{aligned} 13 &= 6(2) + 1, \\ 6 &= 3(2) + 0, \\ 3 &= 1(2) + 1, \\ 1 &= 0(2) + 1. \end{aligned}$$

Sisa pembagian dibaca dari bawah ke atas, yaitu 1101, sehingga $13_{10} = 1101_2$. Cara lain adalah menguraikan bilangan menjadi jumlah pangkat dua. Misalnya, $42 = 32 + 8 + 2 = 2^5 + 2^3 + 2^1$, sehingga bit pada posisi 5, 3, dan 1 bernilai satu dan $42_{10} = 101010_2$.

1.3.2 Sistem heksadesimal

Bilangan biner cepat menjadi panjang dan sulit dibaca, misalnya `1111011010110010`. Karena itu, dalam komputasi sering digunakan sistem **heksadesimal** (basis enam belas) dengan digit `0`–`9` dan `A`–`F`, dengan $A = 10$, $B = 11$, …, $F = 15$.

Heksadesimal sangat praktis karena $16 = 2^4$: satu digit heksadesimal mewakili tepat empat bit. Misalnya $0000_2 = 0_{16}$, $1010_2 = A_{16}$, dan $1111_2 = F_{16}$. Bilangan 10101111_2 dapat dikelompokkan menjadi 1010 1111, sehingga

$$\boxed{10101111_2 = AF_{16}}$$

Notasi heksadesimal akan muncul kembali ketika kita melihat pola bit bilangan real dan alamat memori. Dalam C, bilangan heksadesimal ditulis dengan awalan `0x`, misalnya `0xAF`.

1.4 Representasi bilangan bulat dan karakter

1.4.1 Bilangan bulat tak bertanda

Kasus paling sederhana adalah bilangan bulat tidak negatif (*unsigned integer*). Dengan delapan bit, pola terkecil `00000000` mewakili 0 dan pola terbesar `11111111` mewakili

$$2^7 + 2^6 + \cdots + 2^1 + 2^0 = 255.$$

Secara umum, bilangan bulat tak bertanda dengan n bit mempunyai rentang

$$\boxed{0 \le x \le 2^n - 1}$$

Untuk 16 bit rentangnya $0 \le x \le 65\,535$, dan untuk 32 bit $0 \le x \le 4\,294\,967\,295$.

1.4.2 Bilangan bulat bertanda

Untuk bilangan negatif seperti -1, -27, atau -100, komputer modern umumnya menggunakan representasi ***two's complement***. Kita belum memerlukan detailnya; cukup diketahui bahwa bilangan bulat bertanda (*signed integer*) dengan n bit mempunyai rentang

$$\boxed{-2^{n-1} \le x \le 2^{n-1} - 1}$$

Untuk delapan bit rentangnya $-128 \le x \le 127$, dan untuk 16 bit $-32\,768 \le x \le 32\,767$. Perhatikan bahwa rentang tersebut tidak simetris: terdapat satu bilangan negatif lebih banyak.

1.4.3 *Overflow*

Karena jumlah bitnya terbatas, bilangan bulat di komputer tidak dapat bertambah tanpa batas. Nilai maksimum bilangan bulat tak bertanda delapan bit adalah 255. Secara matematis $255 + 1 = 256$, tetapi $256 = 2^8 = 100000000_2$ membutuhkan sembilan bit. Ketika hasil suatu operasi keluar dari rentang tipe datanya, terjadilah ***overflow***.

Program berikut memperlihatkan apa yang terjadi pada bilangan bulat tak bertanda delapan bit (`unsigned char`) ketika nilainya melewati batas.

```c
#include <stdio.h>

int main(void)
{
    unsigned char x = 255;

    printf("x     = %d\n", x);
    x = x + 1;
    printf("x + 1 = %d\n", x);

    return 0;
}
```

```
x     = 255
x + 1 = 0
```

Bit kesembilan hilang sehingga nilai "berputar" kembali ke nol. *Overflow* merupakan contoh awal bahwa matematika dan komputasi tidak selalu identik. Dalam matematika, bilangan bulat dapat sebesar apa pun; dalam komputer, bilangan tersebut disimpan dengan jumlah bit yang terbatas. Ketika melakukan komputasi, kita harus selalu memperhatikan **nilai matematis** dan **representasi komputer**; keduanya berkaitan erat, tetapi tidak sama.

1.4.4 Karakter sebagai bilangan

Komputer juga menyimpan teks, misalnya huruf `A` atau kalimat `Teknik Fisika`. Caranya adalah memberikan sebuah kode numerik kepada setiap karakter. Dalam ASCII, karakter `A` diberi kode 65, `B` diberi kode 66, dan `a` diberi kode 97. Kode tersebut kemudian disimpan sebagai pola bit:

$$\text{karakter} \longrightarrow \text{kode numerik} \longrightarrow \text{bit}.$$

ASCII hanya mencakup kumpulan karakter yang terbatas; sistem modern menggunakan Unicode untuk kumpulan karakter yang jauh lebih luas. Poin pentingnya adalah bahwa hampir semua informasi digital pada akhirnya direpresentasikan sebagai pola bit.

1.5 Representasi bilangan real

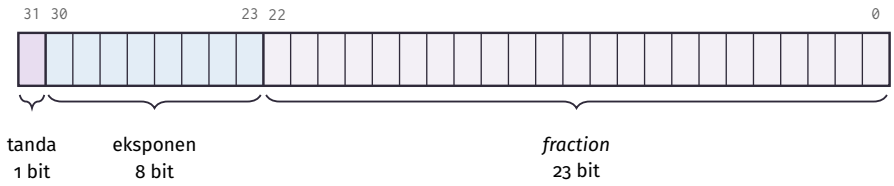
Bilangan real seperti $\pi = 3.141592653589793\ldots$, $\frac{1}{3} = 0.333333\ldots$, atau tetapan Avogadro $6.02214076 \times 10^{23}$ lebih sulit disimpan. Komputer hanya mempunyai jumlah bit yang terbatas, sedangkan sebagian bilangan membutuhkan digit yang tak berhingga banyaknya. Untuk itu digunakan representasi **titik kambang** (*floating-point*).

Gagasannya mirip dengan notasi ilmiah. Bilangan 602 200 000 000 000 000 000 000 ditulis sebagai 6.022×10^{23}, atau secara umum $x = \pm m \times 10^{e}$. Representasi *floating-point* menggunakan gagasan yang sama dengan basis dua:

$$x = (-1)^{s}\, m\, 2^{e},$$

dengan s menyatakan tanda, m bagian signifikan, dan e eksponen.

Standar yang paling banyak digunakan adalah **IEEE 754**. Pada format *single precision* 32 bit, susunan bitnya adalah sebagai berikut.



Untuk bilangan normal, nilainya adalah

$$x = (-1)^{s}\, (1.f)_2\, 2^{E-127},$$

dengan E nilai delapan bit eksponen dan 127 disebut **bias**. Bagian $1.f$ menunjukkan bahwa bit `1` di depan titik biner tidak perlu disimpan; yang disimpan pada bagian *fraction* hanyalah digit setelah titik biner.

1.5.1 Contoh: representasi 5.75

Bagian bulat $5_{10} = 101_2$, sedangkan bagian pecahan $0.75 = 0.5 + 0.25 = 0.11_2$. Jadi

$$5.75_{10} = 101.11_2 = 1.0111_2 \times 2^{2}.$$

Dari bentuk ternormalisasi ini kita memperoleh ketiga bagian IEEE 754:

- **tanda**: bilangan positif, sehingga $s = 0$;
- **eksponen**: $e = 2$ disimpan dengan bias, $E = 2 + 127 = 129 = 10000001_2$;
- ***fraction***: digit setelah titik biner, `0111`, ditambah nol hingga 23 bit, yaitu `01110000000000000000000`.

Jika digabungkan, diperoleh

```
0 10000001 01110000000000000000000
```

atau dalam heksadesimal `0x40B80000`. Kita dapat memeriksanya langsung dengan C. Program berikut menyalin pola bit sebuah `float` ke bilangan bulat 32 bit agar dapat dicetak dalam bentuk heksadesimal.

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

int main(void)
{
    float x = 5.75f;
    uint32_t bits;

    memcpy(&bits, &x, sizeof bits);
    printf("x = %.2f, pola bit = 0x%08X\n", x, (unsigned)bits);

    return 0;
}
```

```
x = 5.75, pola bit = 0x40B80000
```

Bilangan 5.75 dapat disimpan secara eksak karena pecahannya mempunyai representasi biner yang berhingga. Tidak semua bilangan desimal bersifat demikian.

1.5.2 Mengapa 0.1 tidak eksak?

Dalam sistem desimal, $\frac{1}{3} = 0.333333\ldots$ tidak dapat ditulis dengan digit yang berhingga; jika hanya tersedia empat digit, kita terpaksa menulis 0.3333. Hal serupa terjadi dalam sistem biner: bilangan desimal 0.1 tidak mempunyai representasi biner yang berhingga. Komputer harus menyimpan nilai biner yang sangat dekat dengan 0.1, tetapi bukan 0.1 yang eksak. Akibatnya, 0.1 + 0.2 dapat menghasilkan nilai internal yang sedikit berbeda dari 0.3.

```c
#include <stdio.h>

int main(void)
{
    double a = 0.1 + 0.2;
    double b = 0.3;

    printf("0.1 + 0.2 = %.17f\n", a);
    printf("0.3       = %.17f\n", b);

    return 0;
}
```

```
0.1 + 0.2 = 0.30000000000000004
0.3       = 0.29999999999999999
```

Komputer tidak “salah berhitung”. Masalahnya terletak pada representasi: nilai 0.1 dan 0.2 yang dijumlahkan sudah merupakan pendekatan sejak awal.

1.5.3 Kesalahan pembulatan, presisi, dan rentang

Perbedaan kecil akibat representasi terbatas disebut **kesalahan pembulatan** (*rounding error*). Untuk satu operasi, kesalahan ini biasanya sangat kecil. Akan tetapi, komputasi ilmiah dapat melibatkan jutaan operasi, dan dalam algoritma tertentu kesalahan kecil dapat terakumulasi atau diperbesar. Karena itu, hasil komputasi numerik perlu dianalisis, bukan diterima begitu saja. Jika teori memberikan x_{eksak} dan program menghasilkan x_{hitung}, kita sering memeriksa galat absolutnya,

$$|\Delta x| = |x_{\text{hitung}} - x_{\text{eksak}}|,$$

dan membandingkan dua bilangan *floating-point* menggunakan toleransi ε, yaitu $|x_1 - x_2| < \varepsilon$, bukan kesamaan eksak.

Dua istilah yang perlu dibedakan adalah **presisi** (*precision*) dan **rentang** (*range*). Presisi berkaitan dengan banyaknya digit signifikan yang dapat disimpan, sedangkan rentang berkaitan dengan seberapa besar atau kecil nilai yang dapat direpresentasikan. Dalam C kita akan mengenal tipe `float` dan `double`; keduanya bertipe *floating-point*, tetapi `double` menggunakan lebih banyak bit (umumnya 64 bit) dan memberikan presisi lebih tinggi. Untuk komputasi ilmiah dalam buku ini, kita akan menggunakan `double` sebagai pilihan bawaan untuk bilangan real.

1.6 Program C pertama untuk perhitungan fisika

Sekarang kita gabungkan gagasan-gagasan di atas. Sebuah benda bermassa $m = 2.0$ kg bergerak dengan kecepatan $v = 3.0$ m/s. Sebelum menulis program, kita hitung dahulu secara manual:

$$E_k = \tfrac{1}{2}(2.0)(3.0)^2 = \tfrac{1}{2}(2)(9) = 9 \text{ J}.$$

Program C untuk perhitungan tersebut adalah sebagai berikut. Simpan sebagai `kinetic_energy.c`.

```c
#include <stdio.h>

int main(void)
{
    double mass = 2.0;      /* kg  */
    double velocity = 3.0;  /* m/s */

    double energy = 0.5 * mass * velocity * velocity;

    printf("Kinetic energy = %f J\n", energy);

    return 0;
}
```

Kompilasi dan jalankan:

```
gcc -Wall -Wextra kinetic_energy.c -o kinetic_energy
./kinetic_energy
```

```
Kinetic energy = 9.000000 J
```

Baris `double mass = 2.0;` menyimpan massa, `double velocity = 3.0;` menyimpan kecepatan, dan baris berikutnya melakukan perhitungan $E_k = \frac{1}{2}mv^2$. Teks di antara `/*` dan `*/` adalah **komentar**: tidak dijalankan komputer, tetapi membantu manusia memahami program, misalnya dengan mencatat satuan. Untuk sementara kita belum perlu menghafal semua aturan sintaksnya.

1.6.1 Tiga jenis masalah: galat sintaksis, galat logika, dan peringatan

Apakah program yang berhasil dikompilasi pasti benar? Tidak. Perhatikan versi berikut.

```c
#include <stdio.h>

int main(void)
{
    double mass = 2.0;
    double velocity = 3.0;

    double energy = 0.5 * mass * velocity;

    printf("Kinetic energy = %f J\n", energy);

    return 0;
}
```

```
Kinetic energy = 3.000000 J
```

Program dapat dikompilasi tanpa masalah dan berjalan normal, tetapi persamaan yang diimplementasikan adalah $E = \frac{1}{2}mv$, bukan $E_k = \frac{1}{2}mv^2$. Hasilnya salah. Ini adalah contoh **galat logika** (*logic error*). *Compiler* tidak dapat mendeteksinya karena *compiler* tidak mengetahui fisika.

Jenis kesalahan lain adalah **galat sintaksis** (*syntax error*), yaitu kode yang tidak mengikuti aturan bahasa. Jika titik koma setelah `printf(...)` dihapus, *compiler* menolak menerjemahkan program:

```c
#include <stdio.h>

int main(void)
{
    printf("Hello")
    return 0;
}
```

```
program.c: In function ‘main’:
program.c:5:20: error: expected ‘;’ before ‘return’
    5 |     printf("Hello")
      |                    ^
      |                    ;
    6 |     return 0;
      |     ~~~~~~
```

Selain galat, *compiler* juga dapat memberikan **peringatan** (*warning*). Galat menyebabkan kompilasi gagal, sedangkan peringatan biasanya masih memungkinkan program dikompilasi, tetapi menandai bagian yang patut dicurigai. Itulah sebabnya kita selalu menggunakan opsi `-Wall -Wextra` dan membaca pesan *compiler* dengan teliti. Pesan galat dan peringatan bukan musuh pemrogram; keduanya menunjukkan bagian program yang perlu diperiksa.

1.6.2 Memeriksa dan memperbaiki program

Proses mencari dan memperbaiki kesalahan disebut ***debugging***. Pada tahap awal, kebiasaan sederhana berikut sudah sangat membantu:

- baca pesan *compiler* dengan teliti;
- periksa kembali persamaan yang digunakan;
- uji program dengan masukan sederhana yang hasilnya dapat dihitung manual;
- tampilkan nilai variabel antara jika perlu;
- ubah satu hal pada satu waktu;
- jangan langsung menganggap komputer yang salah.

Kebiasaan terpenting dapat diringkas sebagai

$$\boxed{\text{prediksi} \rightarrow \text{jalankan} \rightarrow \text{bandingkan}}$$

Untuk contoh energi kinetik, kita sudah tahu bahwa $m = 2$ dan $v = 3$ seharusnya menghasilkan 9 J. Jika program menghasilkan nilai lain, ada sesuatu yang perlu diperiksa.

1.6.3 Contoh: gerak lurus berubah beraturan

Untuk gerak dengan percepatan konstan,

$$x(t) = x_0 + v_0 t + \tfrac{1}{2} a t^2.$$

Dengan $x_0 = 2.0$ m, $v_0 = 5.0$ m/s, $a = 3.0$ m/s^2, dan $t = 4.0$ s, kita memperoleh prediksi

$$x = 2.0 + (5.0)(4.0) + \tfrac{1}{2}(3.0)(4.0)^2 = 2 + 20 + 24 = 46 \text{ m}.$$

Program C-nya:

```c
#include <stdio.h>

int main(void)
{
```

```c
    double x0 = 2.0;            /* m      */
    double v0 = 5.0;            /* m/s    */
    double acceleration = 3.0;  /* m/s^2  */
    double time = 4.0;          /* s      */

    double position = x0 + v0 * time
                     + 0.5 * acceleration * time * time;

    printf("Initial position = %f m\n", x0);
    printf("Initial velocity = %f m/s\n", v0);
    printf("Acceleration     = %f m/s^2\n", acceleration);
    printf("Time             = %f s\n", time);
    printf("Final position   = %f m\n", position);

    return 0;
}
```

```
Initial position = 2.000000 m
Initial velocity = 5.000000 m/s
Acceleration     = 3.000000 m/s^2
Time             = 4.000000 s
Final position   = 46.000000 m
```

Keluaran program sesuai dengan prediksi $x = 46$ m. Program ini, seperti hampir semua program komputasi, mengikuti pola

masukan → pemrosesan → keluaran

1.7 Program sebagai representasi model

Ada satu gagasan penting yang perlu dipegang sejak awal: komputer tidak mengetahui fisika. Ketika kita menulis `voltage = current * resistance;`, komputer tidak tahu bahwa kita sedang menggunakan hukum Ohm; komputer hanya menjalankan perkalian. Makna fisis berasal dari kita sebagai pembuat model dan program:

komputer menjalankan instruksi; manusia menentukan maknanya

Karena itu, dalam komputasi sains dan rekayasa kita perlu memeriksa setidaknya tiga hal:

1. apakah model fisis yang digunakan sudah tepat;
2. apakah algoritma merepresentasikan model tersebut dengan benar;
3. apakah program mengimplementasikan algoritma dengan benar.

Jika salah satu tahap bermasalah, hasil komputasi juga dapat salah walaupun program berjalan sempurna secara teknis.

1.7.1 Satuan dan kewajaran hasil

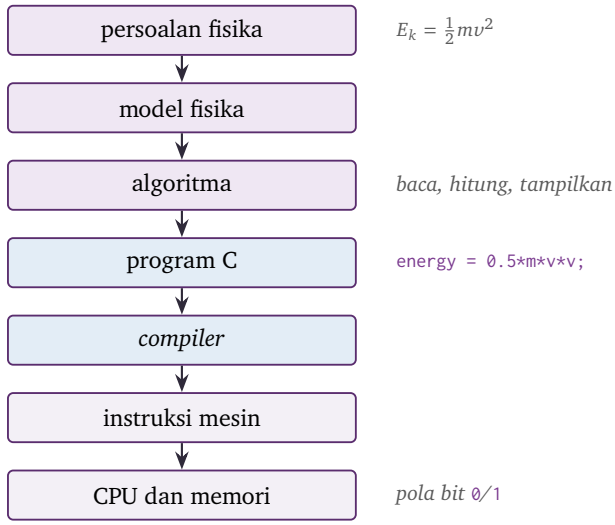
Pemrograman tidak menghapus kebutuhan memahami satuan. Misalkan kecepatan $v = 72$ km/jam dimasukkan ke program yang mengharapkan satuan m/s. Program berjalan

tanpa galat, tetapi karena 72 km/jam = 20 m/s, energi kinetik yang dihitung menjadi $(72/20)^2 \approx 13$ kali terlalu besar. Nama variabel yang jelas dan komentar satuan, misalnya `double velocity = 20.0; /* m/s */`, membantu mencegah kesalahan semacam ini.

Demikian pula, jika sebuah program menghasilkan kecepatan benda $v = 3.7 \times 10^{12}$ m/s, hasil tersebut jauh melampaui kecepatan cahaya $c \approx 3 \times 10^8$ m/s. Program mungkin berjalan tanpa galat, tetapi hasil itu menjadi petunjuk kuat bahwa model, satuan, masukan, atau program perlu diperiksa. Dalam komputasi ilmiah, pertanyaan "Apakah hasil ini masuk akal secara fisika?" sama pentingnya dengan pertanyaan "Apakah program berhasil dijalankan?"

1.7.2 Lapisan abstraksi

Kini kita dapat melihat beberapa lapisan abstraksi dalam komputasi. Pada lapisan paling atas terdapat persoalan dan model fisika, misalnya $E_k = \frac{1}{2}mv^2$. Model itu dirumuskan menjadi algoritma (baca massa, baca kecepatan, hitung, tampilkan), lalu diimplementasikan sebagai program C. *Compiler* menerjemahkan program menjadi instruksi mesin, yang akhirnya dijalankan oleh CPU dan memori.



Seorang pemrogram tidak perlu selalu berpikir pada tingkat transistor atau instruksi mesin. Namun, mengetahui bahwa lapisan-lapisan tersebut ada membantu kita memahami mengapa tipe data mempunyai batas, mengapa bilangan real tidak selalu eksak, dan mengapa program perlu dikompilasi. Pada kuliah berikutnya kita akan memusatkan perhatian pada lapisan algoritma: bagaimana suatu persoalan dirumuskan menjadi langkah-langkah yang jelas sebelum ditulis sebagai program.

1.8 Cek pemahaman

Cobalah menjawab pertanyaan berikut tanpa melihat kembali catatan.

- Apa perbedaan antara algoritma dan program?

- Mengapa CPU tidak dapat langsung menjalankan kode sumber C? Apa fungsi *compiler*?
- Berapa banyak pola berbeda yang dapat dibentuk oleh delapan bit? Mengapa bilangan bulat tak bertanda delapan bit mempunyai rentang 0 sampai 255?
- Mengapa $13_{10} = 1101_2$? Apa keuntungan notasi heksadesimal?
- Apa yang dimaksud dengan *overflow*?
- Bagaimana karakter dapat disimpan oleh komputer?
- Mengapa bilangan seperti 0.1 tidak dapat direpresentasikan secara eksak?
- Apa perbedaan antara presisi dan rentang?
- Apa perbedaan antara galat sintaksis, galat logika, dan peringatan?
- Mengapa hasil program fisika perlu dibandingkan dengan prediksi atau perhitungan manual?

1.9 Latihan

1. **Konversi biner.** Ubah 1011_2, 100101_2, 11111111_2, dan 10000000_2 ke desimal. Ubah pula 7_{10}, 18_{10}, 25_{10}, dan 100_{10} ke biner.
2. **Heksadesimal.** Ubah 1010_2, 11111111_2, dan 1100101011110000_2 ke heksadesimal, lalu ubah $3A_{16}$ dan FF_{16} ke desimal.
3. **Rentang bilangan bulat.** Untuk bilangan bulat tak bertanda 10 bit dan 16 bit, tentukan banyaknya pola bit, nilai minimum, dan nilai maksimumnya. Tentukan pula rentang bilangan bulat bertanda 8 bit dan 16 bit dengan representasi *two's complement*.
4. ***Overflow*.** Jelaskan mengapa 256 tidak dapat disimpan sebagai bilangan bulat tak bertanda delapan bit dan berapa bit minimum yang diperlukan. Ubah program *overflow* pada kuliah ini sehingga menambahkan 10 ke nilai 250; prediksi keluarannya sebelum dijalankan.
5. **IEEE 754.** Tentukan bit tanda, eksponen, dan *fraction* dari -6.5 dalam format *single precision*. Periksa jawaban Anda dengan program pencetak pola bit pada Subbab 1.5.
6. ***Floating-point*.** Jelaskan dengan kalimat sendiri mengapa 0.1 tidak dapat disimpan secara eksak dan apa hubungannya dengan $\frac{1}{3} = 0.333\ldots$ dalam sistem desimal. Mengapa perbandingan `0.1 + 0.2 == 0.3` berbahaya?
7. **Algoritma sederhana.** Tuliskan algoritma dengan kalimat biasa untuk menghitung daya listrik $P = VI$. Tentukan dengan jelas masukan, proses, dan keluarannya. Lakukan hal yang sama untuk menghitung tekanan gas ideal dari $PV = nRT$.
8. **Program daya listrik.** Buat program `power.c` yang menghitung $P = VI$ untuk $V = 12.0$ V dan $I = 2.5$ A. Hitung hasilnya secara manual terlebih dahulu, kompilasi dengan `-Wall -Wextra`, dan pastikan tidak ada peringatan.
9. **Program gerak lurus.** Ubah program gerak lurus pada kuliah ini sehingga juga menampilkan kecepatan akhir $v = v_0 + at$. Prediksi kedua nilai sebelum menjalankan program.

10. **Mencari galat logika.** Seorang mahasiswa menghitung energi potensial $E_p = mgh$ dengan baris `energy = mass * 9.81 + height;`. Program berjalan tanpa peringatan. Jelaskan jenis kesalahannya, bagaimana cara mendeteksinya dengan kasus uji sederhana, dan perbaiki barisnya.

1.10 Rangkuman

- Komputer adalah alat untuk menjalankan prosedur komputasi. Persoalan fisika diterjemahkan menjadi model matematis, model menjadi algoritma, dan algoritma diimplementasikan sebagai program.
- Kode sumber C harus diterjemahkan oleh *compiler* menjadi *executable* sebelum dapat dijalankan oleh CPU.
- Data di komputer direpresentasikan dengan bit. Dengan n bit terdapat 2^n pola berbeda.
- Bilangan bulat tak bertanda n bit mempunyai rentang $0 \le x \le 2^n - 1$, sedangkan bilangan bulat bertanda *two's complement* mempunyai rentang $-2^{n-1} \le x \le 2^{n-1} - 1$. Hasil di luar rentang menyebabkan *overflow*.
- Karakter disimpan sebagai kode numerik, misalnya ASCII atau Unicode.
- Bilangan real disimpan sebagai *floating-point* (IEEE 754) yang terdiri atas tanda, eksponen, dan bagian signifikan. Banyak bilangan desimal, seperti 0.1, hanya dapat disimpan sebagai pendekatan sehingga muncul kesalahan pembulatan.
- Program yang berhasil dikompilasi dan dijalankan belum tentu benar. Hasil harus diperiksa dengan perhitungan manual, satuan, orde besaran, dan penalaran fisika.
- Kuliah berikutnya memusatkan perhatian pada perancangan algoritma: menuliskan prosedur dengan kalimat berurutan, diagram alir, dan *pseudocode*, serta menilai kebenaran dan kebutuhan komputasinya.

KULIAH

2

Berpikir Algoritmik dan Perancangan Algoritma

2.1 Menyusun algoritma untuk masalah fisika

Pada Kuliah 1 kita melihat bahwa komputer hanya menjalankan instruksi yang sangat terstruktur. Komputer tidak memahami tujuan eksperimen, makna fisis suatu besaran, atau alasan kita memilih suatu persamaan. Karena itu, sebelum menulis program kita memerlukan cara berpikir yang menjembatani persoalan fisis dengan instruksi komputasi. Cara berpikir ini disebut **berpikir algoritmik** (*algorithmic thinking*). Pada kuliah ini kita berfokus pada bagian awal alur kerja kita,

$$\boxed{\text{masalah} \longrightarrow \text{model} \longrightarrow \text{algoritma}}$$

Bahasa pemrograman belum menjadi pusat perhatian. Kita akan melihat beberapa implementasi dalam C, tetapi tujuan utamanya adalah memahami cara memecah persoalan menjadi langkah-langkah yang jelas.

Sebagai contoh, misalkan sebuah sensor menghasilkan N data temperatur $T_1, T_2, \ldots, T_N$ dan kita ingin mengetahui temperatur rata-rata, minimum, dan maksimum. Rumus rata-rata

$$\overline{T} = \frac{1}{N} \sum_{i=1}^{N} T_i$$

menyatakan *apa* yang dihitung, tetapi belum menjelaskan kepada komputer *bagaimana* menghitungnya: kapan proses dimulai, bagaimana setiap data dibaca, di mana jumlah sementara disimpan, bagaimana minimum dan maksimum diperbarui, kapan proses berhenti, dan apa yang ditampilkan di akhir. Langkah-langkah tersebut dapat ditulis secara informal sebagai berikut.

```
Baca banyaknya data N
Baca temperatur pertama
Jadikan temperatur pertama sebagai jumlah, minimum, dan maksimum sementara

Untuk setiap temperatur berikutnya:
    tambahkan temperatur ke jumlah
```

```
    jika temperatur lebih kecil daripada minimum sementara, perbarui minimum
    jika temperatur lebih besar daripada maksimum sementara, perbarui maksimum

Hitung rata-rata = jumlah / N
Tampilkan rata-rata, minimum, dan maksimum
```

Langkah-langkah ini sudah dekat dengan program. Inti berpikir algoritmik adalah menjawab pertanyaan: *informasi apa yang saya miliki, hasil apa yang saya inginkan, dan urutan proses apa yang menghubungkan keduanya?* Pertanyaan "sintaks C apa yang harus saya tulis?" datang belakangan.

2.1.1 Sifat algoritma yang baik

Secara praktis, **algoritma** adalah urutan langkah yang terdefinisi dengan jelas untuk menyelesaikan suatu kelas persoalan. Algoritma yang baik

- mempunyai masukan (*input*) dan keluaran (*output*) yang jelas;
- terdiri atas langkah-langkah yang tidak menimbulkan banyak tafsiran dan dapat dilaksanakan;
- berhenti setelah sejumlah langkah yang berhingga;
- berlaku untuk seluruh kelas data yang memenuhi asumsinya, bukan hanya untuk satu kasus.

Instruksi "hitung nilai temperatur yang baik" bukan langkah algoritmik karena kata "baik" tidak mempunyai definisi operasional. Sebaliknya, "jika temperatur berada pada rentang 20–30 °C, nyatakan status sebagai `NORMAL`" sudah dapat diuji. Demikian pula, "ulangi sampai hasilnya cukup dekat" harus diubah menjadi kriteria yang terukur, misalnya $|x_{\text{baru}} - x_{\text{lama}}| \le 10^{-3}$. Berpikir algoritmik menuntut kita mengubah istilah yang samar menjadi aturan yang dapat diperiksa.

2.1.2 Spesifikasi masalah

Sebelum merancang algoritma, tuliskan **spesifikasi masalah**, yang sekurang-kurangnya memuat masukan, keluaran, asumsi, dan model yang digunakan. Untuk energi kinetik, spesifikasinya adalah

Masukan	m, v
Keluaran	E_k
Asumsi	$m \ge 0$
Model	$E_k = \frac{1}{2} m v^2$

Spesifikasi memisahkan pertanyaan fisika (model apa yang digunakan?) dari pertanyaan komputasi (bagaimana m dan v diproses menjadi E_k?). Pemisahan ini makin penting ketika persoalan makin besar.

Dua istilah yang berguna dalam spesifikasi adalah **prakondisi** (*precondition*), yaitu kondisi yang diasumsikan benar sebelum algoritma dimulai, dan **pascakondisi** (*postcondition*),

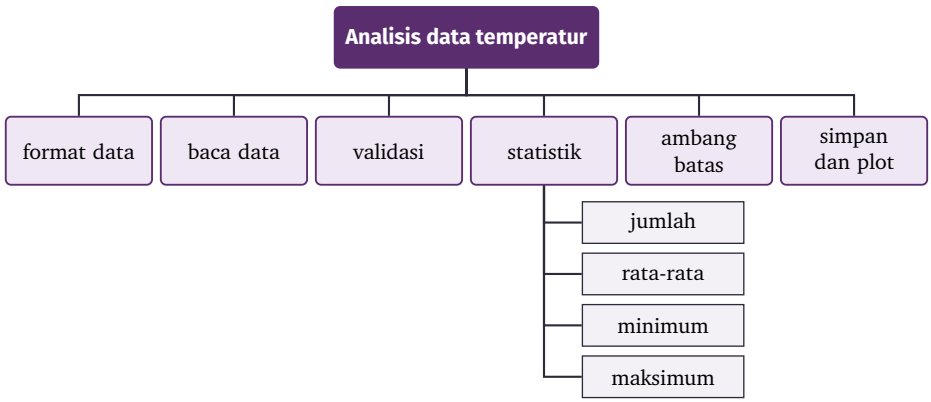
yaitu kondisi yang dijamin benar setelah algoritma selesai. Algoritma rata-rata mempunyai prakondisi $N > 0$; jika $N = 0$, rumusnya mengandung pembagian dengan nol. Pascakondisinya adalah $\overline{x} = (x_1 + x_2 + \cdots + x_N)/N$. Kedua konsep ini menjawab pertanyaan penting: *dalam kondisi apa algoritma ini dijamin benar?*

2.2 Dekomposisi masalah

Persoalan rekayasa yang nyata hampir selalu terlalu besar untuk diselesaikan dalam satu langkah. Strategi utama berpikir algoritmik adalah **dekomposisi**, yaitu memecah persoalan besar menjadi beberapa subpersoalan yang lebih kecil. Misalkan sebuah sistem akuisisi data temperatur menghasilkan berkas pengukuran. Persoalan "analisis data temperatur" dapat dipecah menjadi

1. menentukan format data;
2. membaca data;
3. memvalidasi setiap data;
4. menghitung statistik dasar (jumlah, rata-rata, minimum, maksimum);
5. mengidentifikasi data di atas ambang batas;
6. menyimpan hasil;
7. memvisualisasikan hasil.

Setiap subpersoalan masih dapat dipecah lagi; misalnya "statistik dasar" terdiri atas menghitung jumlah, rata-rata, minimum, dan maksimum. Alih-alih satu program besar yang sulit dipahami, kita memperoleh sekumpulan tugas kecil yang masing-masing mempunyai tujuan jelas.



2.2.1 Menentukan tujuan dan batasan

Dekomposisi juga menuntut kita menentukan tujuan komputasi secara tepat. Tinjau resistor dengan hambatan R yang dialiri arus I. Dari $V = IR$ dan $P = VI$ diperoleh $P = I^2R$. Jika yang dibutuhkan hanya daya, algoritmanya cukup "baca I dan R, hitung $P = I^2R$, tampilkan P". Jika tegangan juga dibutuhkan, lebih wajar menghitung $V = IR$ terlebih

dahulu, lalu $P = VI$. Kedua algoritma sah; pilihannya bergantung pada informasi apa yang ingin diperoleh.

Kita juga perlu menentukan apa yang *tidak* diselesaikan. Untuk contoh resistor, kita mungkin mengasumsikan hambatan tetap, resistor bersifat ohmik, efek temperatur diabaikan, dan semua besaran dalam satuan SI. Asumsi semacam ini penting karena program selalu bekerja di dalam model tertentu; program tidak dapat mengetahui sendiri kapan model fisikanya sudah tidak berlaku.

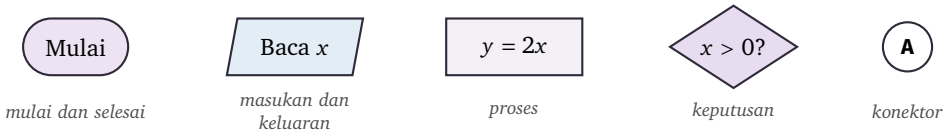
2.3 Kalimat berurutan dan diagram alir

Bentuk paling sederhana sebuah algoritma adalah kalimat berurutan. Untuk posisi benda dengan percepatan konstan, $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$, algoritmanya adalah

```
Baca posisi awal x0, kecepatan awal v0, percepatan a, dan waktu t
Hitung posisi x = x0 + v0*t + 0.5*a*t*t
Tampilkan x
```

Bentuk ini mudah dibaca, tetapi untuk persoalan yang lebih rumit kalimat biasa cepat menjadi panjang dan ambigu. Karena itu kita memerlukan representasi lain, yaitu diagram alir dan *pseudocode*.

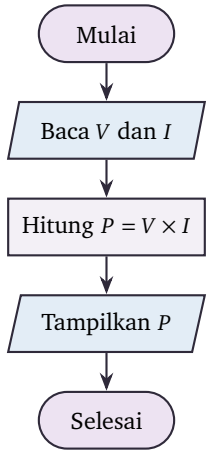
Diagram alir (*flowchart*) adalah representasi visual urutan proses dalam algoritma. Simbol yang digunakan dalam buku ini adalah sebagai berikut.



Panah menunjukkan arah aliran proses. Konektor digunakan untuk menyambung diagram yang terlalu panjang untuk digambar dalam satu kolom.

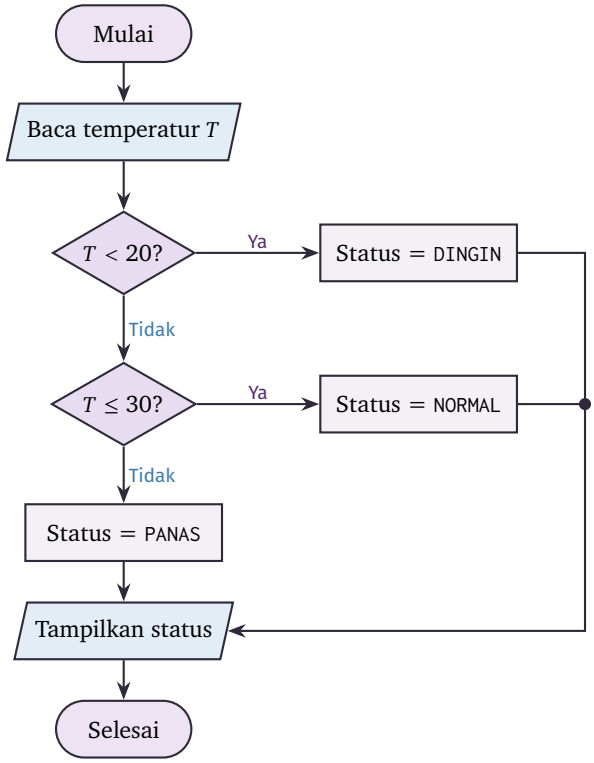
2.3.1 Aliran lurus

Untuk menghitung daya $P = VI$, semua langkah dilaksanakan satu kali secara berurutan.



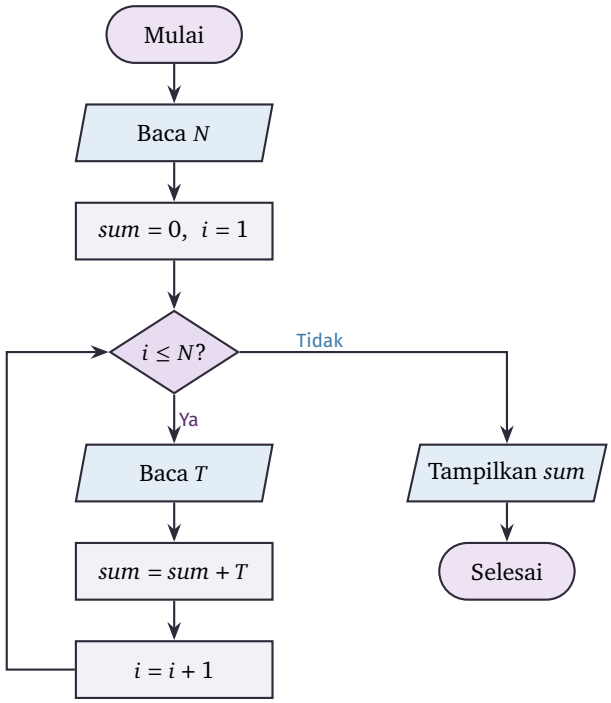
2.3.2 Keputusan

Misalkan sensor temperatur digunakan untuk menentukan status: `DINGIN` jika $T < 20\,^{\circ}\text{C}$, `NORMAL` jika $20\,^{\circ}\text{C} \leq T \leq 30\,^{\circ}\text{C}$, dan `PANAS` jika $T > 30\,^{\circ}\text{C}$. Diagram alirnya memiliki **percabangan**: program tidak selalu menjalankan jalur yang sama untuk semua masukan.



2.3.3 Pengulangan

Untuk menjumlahkan N data temperatur, $S = \sum_{i=1}^{N} T_i$, komputer melakukan penjumlahan berulang. Pencacah i menandai data ke berapa yang sedang diproses, dan panah yang kembali ke atas membentuk **pengulangan**.



Ketiga diagram tersebut memperlihatkan tiga pola dasar aliran kontrol: **urutan**, **keputusan**, dan **pengulangan**. Hampir semua program dapat dipandang sebagai kombinasi ketiganya.

2.4 *Pseudocode*

Diagram alir membantu melihat struktur, tetapi untuk algoritma yang panjang ukurannya menjadi tidak praktis. Representasi yang lebih ringkas adalah ***pseudocode***: algoritma yang ditulis menyerupai bahasa pemrograman, tetapi tidak terikat pada sintaks bahasa tertentu, sehingga mudah diterjemahkan ke C, Python, atau bahasa lain. *Pseudocode* tidak mempunyai standar universal; yang penting adalah konsistensi. Dalam buku ini digunakan konvensi berikut.

Notasi	Kegunaan
INPUT	untuk masukan
OUTPUT	untuk keluaran
IF	untuk keputusan
ELSE	untuk alternatif
FOR	untuk pengulangan dengan pencacah
WHILE	untuk pengulangan berbasis kondisi
RETURN	untuk mengembalikan hasil
$\leftarrow$	untuk pemberian nilai

Tanda panah digunakan agar tidak rancu dengan tanda sama dengan dalam matematika. Dalam matematika, $x = x + 1$ adalah pernyataan yang mustahil benar. Dalam algoritma, $x \leftarrow x + 1$ mempunyai makna yang jelas: nilai baru x diperoleh dari nilai lama x ditambah satu.

Untuk posisi benda dengan percepatan konstan, *pseudocode*-nya masih sangat dekat dengan persamaan matematis karena tidak ada keputusan maupun pengulangan.

ALGORITHM PositionConstantAcceleration
INPUT:
x_0, v_0, a, t
PROCESS:
$x \leftarrow x_0 + v_0 t + 0.5 \cdot a \cdot t \cdot t$
OUTPUT:
x

Untuk rata-rata N data, $\overline{x} = \frac{1}{N} \sum_{i=1}^{N} x_i$, notasi jumlahan diterjemahkan menjadi penambahan satu per satu.

ALGORITHM Mean
INPUT:
N dan $x_1, x_2, \ldots, x_N$
PROCESS:
$sum \leftarrow 0$
FOR $i \leftarrow 1$ **TO** N
$sum \leftarrow sum + x_i$
END FOR
$mean \leftarrow sum/N$
OUTPUT:
$mean$

Inilah contoh penting hubungan matematika dan algoritma: notasi matematika menyatakan *apa* yang dihitung, sedangkan algoritma menjelaskan *bagaimana* perhitungan dilakukan.

Algoritma tersebut dapat diperluas untuk sekaligus mencari minimum dan maksimum.

ALGORITHM Statistics
INPUT:

```
    N dan x_1, x_2, ..., x_N
PRECONDITION:
    N > 0
PROCESS:
    sum ← x_1;   minimum ← x_1;   maximum ← x_1
    FOR i ← 2 TO N
        sum ← sum + x_i
        IF x_i < minimum
            minimum ← x_i
        END IF
        IF x_i > maximum
            maximum ← x_i
        END IF
    END FOR
    mean ← sum/N
OUTPUT:
    mean, minimum, maximum
```

Perhatikan bahwa *minimum* dan *maximum* diinisialisasi dengan data pertama, bukan dengan nol. Mengapa? Jika semua data negatif, misalnya -5, -2, dan -8, lalu *maximum* dimulai dari nol, algoritma akan melaporkan maksimum 0 walaupun nol tidak pernah muncul dalam data. Detail kecil seperti ini menentukan benar atau salahnya algoritma.

2.5 Menelusuri dan memeriksa algoritma

Setelah menulis algoritma, kita belum boleh langsung menganggapnya benar. Teknik paling sederhana untuk memeriksanya adalah **penelusuran** (*tracing* atau *dry run*), yaitu menjalankan algoritma secara manual dengan data kecil. Untuk data temperatur 29.5, 30.2, 28.9, dan 31.1, algoritma Statistics menghasilkan perubahan nilai berikut.

Langkah	x_i	*sum*	*minimum*	*maximum*
awal	29.5	29.5	29.5	29.5
2	30.2	59.7	29.5	30.2
3	28.9	88.6	28.9	30.2
4	31.1	119.7	28.9	31.1

Setelah seluruh data diproses, $\overline{T} = 119.7/4 = 29.925$, $T_{\text{min}} = 28.9$, dan $T_{\text{max}} = 31.1$.

2.5.1 Keadaan dan invarian

Kumpulan nilai variabel pada suatu saat disebut **keadaan** (*state*) algoritma. Setelah data ketiga diproses, keadaannya adalah $i = 3$, $sum = 88.6$, $minimum = 28.9$, dan $maximum = 30.2$. Banyak kesalahan program berasal dari pembaruan keadaan yang tidak tepat, sehingga kebiasaan melacak keadaan sangat berguna.

Cara berpikir lain yang sangat bermanfaat adalah bertanya: *apa yang harus selalu benar setelah setiap iterasi?* Setelah i data diproses, algoritma Statistics harus memenuhi

$$sum = \sum_{k=1}^{i} x_k, \qquad minimum = \min(x_1, \ldots, x_i), \qquad maximum = \max(x_1, \ldots, x_i).$$

Pernyataan yang tetap benar selama pengulangan disebut **invarian**. Kita tidak akan melakukan pembuktian formal, tetapi invarian membantu memahami mengapa algoritma bekerja: jika invarian benar di awal dan setiap iterasi mempertahankannya, maka invarian juga benar di akhir.

2.5.2 Kasus biasa dan kasus batas

Algoritma dikatakan benar jika untuk setiap masukan yang memenuhi prakondisi, keluarannya memenuhi spesifikasi. Dalam praktik kita memeriksanya melalui penelusuran manual, kasus sederhana, kasus batas, perbandingan dengan hasil yang sudah diketahui, serta pemeriksaan satuan dan orde besaran.

Untuk algoritma rata-rata, kasus biasa mungkin menggunakan $N = 5$, tetapi kita juga perlu menguji **kasus batas** seperti $N = 1$ (rata-rata harus sama dengan satu-satunya data) dan $N = 0$ (algoritma harus menolak masukan karena terjadi pembagian dengan nol). Kasus batas sering mengungkap kesalahan yang tidak tampak pada data biasa.

Contohnya adalah pencarian maksimum yang dimulai dari nol:

```
maximum ← 0
FOR setiap x
    IF x > maximum
        maximum ← x
    END IF
END FOR
```

Untuk data $5, 2, 7$ algoritma ini tampak benar, tetapi untuk $-5, -2, -7$ hasilnya 0. Masalahnya bukan pada sintaks, melainkan pada rancangan algoritma. Perbaikannya adalah menginisialisasi *maximum* dengan x_1 lalu memulai pengulangan dari $i = 2$.

2.5.3 Kesalahan umum dalam rancangan

Beberapa pola kesalahan muncul bahkan sebelum satu baris kode ditulis.

- **Langkah tidak jelas**, misalnya “perbaiki nilai jika diperlukan” tanpa kriteria yang dapat diuji.
- **Tidak menetapkan nilai awal**, misalnya *sum* tidak pernah diberi nilai 0 sebelum penjumlahan.
- **Batas pengulangan keliru**: berhenti di $N - 1$ membuat data terakhir terlewat, sedangkan berlanjut sampai $N + 1$ mengakses data yang tidak ada. Kesalahan ini disebut *off-by-one error*.
- **Kondisi penghentian tidak pernah tercapai**, misalnya **WHILE** *error* > *tolerance* tanpa pernah memperbarui *error*.

- **Asumsi tidak dinyatakan**, misalnya algoritma yang hanya benar untuk data terurut atau untuk $N > 0$.

2.5.4 Pemeriksaan berdasarkan fisika

Dalam persoalan teknik, kebenaran algoritma juga diperiksa dengan pengetahuan fisika. Jika $m = 2$ kg dan $v = 3$ m/s, energi kinetiknya 9 J. Hasil -9 J pasti salah karena energi kinetik klasik tidak mungkin negatif, dan hasil 9×10^9 J menandakan kesalahan satuan atau perhitungan. Pemeriksaan algoritma dalam Teknik Fisika sebaiknya selalu menjawab dua pertanyaan: *apakah langkah komputasinya benar?* dan *apakah hasilnya masuk akal secara fisika?*

2.6 Implementasi algoritma dalam bahasa C

Setelah algoritma jelas, kita dapat menerjemahkannya ke bahasa pemrograman. Sintaks C akan dibahas sistematis pada Kuliah 3 dan 4; di sini kita hanya melihat korespondensinya. Algoritma posisi benda diterjemahkan menjadi

```c
#include <stdio.h>

int main(void)
{
    double x0 = 2.0;
    double v0 = 5.0;
    double a = 3.0;
    double t = 4.0;

    double x = x0 + v0 * t + 0.5 * a * t * t;

    printf("x = %.3f m\n", x);

    return 0;
}
```

```
x = 46.000 m
```

Struktur program mengikuti struktur algoritma: nilai awal, perhitungan, lalu keluaran. Kita dapat mengganti nilai x_0, v_0, a, dan t tanpa mengubah gagasan algoritmanya.

Klasifikasi temperatur yang digambarkan dengan diagram alir pada Subbab 2.3 menjadi rangkaian `if`–`else if`–`else`:

```c
#include <stdio.h>

int main(void)
{
    double temperature = 32.5;

    if (temperature < 20.0)
    {
```

```
        printf("DINGIN\n");
    }
    else if (temperature <= 30.0)
    {
        printf("NORMAL\n");
    }
    else
    {
        printf("PANAS\n");
    }

    return 0;
}
```

```
PANAS
```

Pengulangan **FOR** pada *pseudocode* diterjemahkan menjadi struktur `for` dalam C. Kerangka umumnya adalah

```
for (int i = 0; i < n; i++)
{
    /* lakukan proses berulang */
}
```

Secara kasar, konstruksi *pseudocode* mempunyai pasangan dalam C sebagai berikut.

Pseudocode	**Gagasan dalam C**
$x \leftarrow 5$	`x = 5;`
IF kondisi	`if (kondisi)`
ELSE	`else`
FOR ...	`for (...)`
WHILE kondisi	`while (kondisi)`
OUTPUT x	misalnya `printf(...)`

Tabel ini bukan aturan penerjemahan otomatis. Tujuannya menunjukkan bahwa *pseudocode* berada satu tingkat di atas bahasa pemrograman: kita merancang logika terlebih dahulu, baru kemudian memilih sintaks.

2.7 Efisiensi algoritma

Dua algoritma dapat menghasilkan jawaban yang sama, tetapi membutuhkan jumlah pekerjaan yang berbeda. Selain bertanya “apakah algoritma ini benar?”, kita perlu bertanya “berapa banyak pekerjaan yang diperlukan ketika ukuran data membesar?” Pertanyaan ini membawa kita pada **kompleksitas algoritma**, yang di sini dibahas secara intuitif.

Pertama, kita perlu menentukan **ukuran masukan**. Untuk N data temperatur, ukurannya adalah N; untuk citra berukuran $M \times N$, banyak pikselnya MN; untuk matriks bujur sangkar $N \times N$, ukurannya sering dinyatakan dengan N.

- **Waktu konstan,** $O(1)$**.** Menghitung $E_k = \frac{1}{2}mv^2$ membutuhkan jumlah operasi yang sama berapa pun nilai m dan v. Notasi $O(1)$ tidak berarti hanya ada satu operasi, tetapi jumlah operasinya tidak bertambah terhadap ukuran masukan.
- **Waktu linear,** $O(N)$**.** Menghitung $S = \sum_{i=1}^{N} x_i$ mengharuskan setiap data dikunjungi sekali. Jika N menjadi dua kali lebih besar, pekerjaannya kira-kira juga dua kali lebih besar.
- **Waktu kuadratik,** $O(N^2)$**.** Membandingkan setiap sensor dengan setiap sensor lain membutuhkan dua pengulangan bersarang, sehingga jumlah operasinya kira-kira $N \times N = N^2$.

```
FOR i ← 1 TO N
    FOR j ← 1 TO N
        bandingkan sensor i dengan sensor j
    END FOR
END FOR
```

Perbedaan antara pertumbuhan linear dan kuadratik tampak jelas pada tabel berikut.

N	N	N^2
10	10	100
100	100	10 000
1 000	1 000	1 000 000
10 000	10 000	100 000 000

Algoritma yang membutuhkan $3N+5$ operasi dan algoritma yang membutuhkan $100N+20$ operasi sama-sama ditulis $O(N)$. Notasi "O besar" (*Big-O*) menggambarkan *pola pertumbuhan* ketika ukuran masalah membesar; konstanta tetap penting untuk kinerja nyata, tetapi pada analisis awal kita lebih tertarik pada perbedaan antara $O(1)$, $O(N)$, dan $O(N^2)$.

Perlu diingat bahwa kompleksitas bukan waktu dalam detik. Waktu eksekusi nyata bergantung pada prosesor, *compiler*, bahasa, sistem operasi, dan implementasi. Jadi $O(N)$ tidak berarti program memerlukan N detik.

Selain waktu, algoritma juga menggunakan memori. Jika data dibaca satu per satu dan kita hanya menyimpan *sum*, *minimum*, dan *maximum*, memori tambahan yang dibutuhkan tidak bergantung pada N, yaitu $O(1)$. Jika seluruh data disimpan dalam larik, kebutuhan memorinya $O(N)$. Kita membedakan **kompleksitas waktu** dan **kompleksitas ruang**.

2.8 Studi kasus: pemantauan temperatur

Sekarang kita gabungkan seluruh konsep di atas. Sebuah sensor mengukur temperatur suatu sistem termal sebanyak N kali, dan kita ingin memperoleh rata-rata, minimum, maksimum, serta banyaknya pengukuran yang melebihi ambang T_{limit}, misalnya 30 °C.

2.8.1 Spesifikasi dan dekomposisi

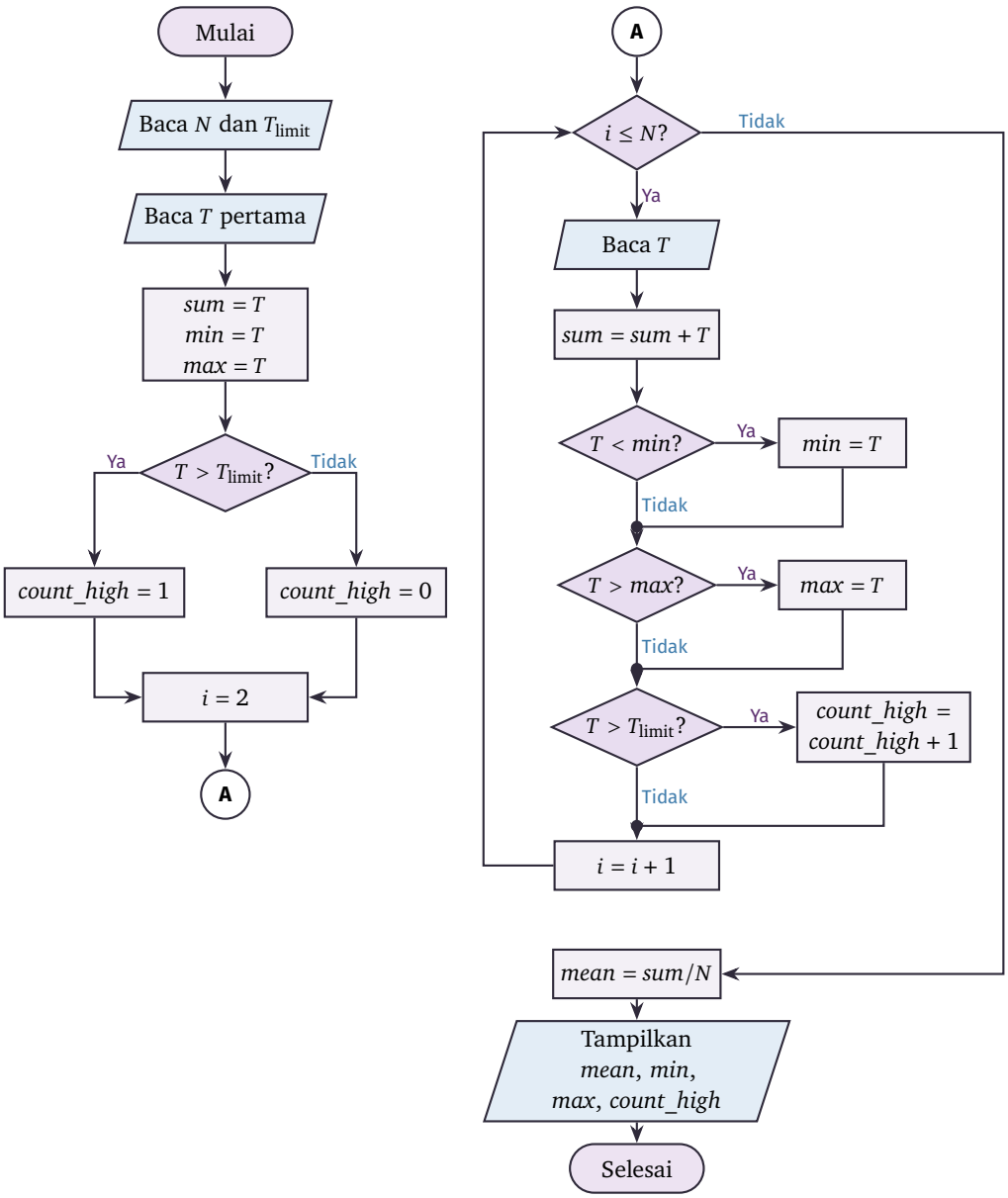
Masukan	N, T_{limit}, dan $T_1, T_2, \ldots, T_N$
Prakondisi	$N > 0$
Keluaran	$\overline{T}$, $T_{\min}$, $T_{\max}$, dan banyaknya $T_i > T_{\text{limit}}$

Persoalan ini dipecah menjadi: membaca N dan ambang; menginisialisasi statistik dengan data pertama; untuk setiap data berikutnya memperbarui jumlah, minimum, maksimum, dan pencacah data tinggi; lalu menghitung rata-rata dan menampilkan hasil.

2.8.2 *Pseudocode* dan diagram alir

```
ALGORITHM TemperatureMonitoring
INPUT:
    N, T_limit, dan T_1, T_2, ..., T_N
PRECONDITION:
    N > 0
PROCESS:
    sum ← T_1;   minimum ← T_1;   maximum ← T_1
    IF T_1 > T_limit
        count_high ← 1
    ELSE
        count_high ← 0
    END IF
    FOR i ← 2 TO N
        sum ← sum + T_i
        IF T_i < minimum
            minimum ← T_i
        END IF
        IF T_i > maximum
            maximum ← T_i
        END IF
        IF T_i > T_limit
            count_high ← count_high + 1
        END IF
    END FOR
    mean ← sum/N
OUTPUT:
    mean, minimum, maximum, count_high
```

Diagram alir algoritma ini cukup panjang sehingga digambar dalam dua kolom yang disambung dengan konektor `A`.



2.8.3 Penelusuran dan kompleksitas

Dengan $N = 5$, $T_{\text{limit}} = 30.0$, dan data {29.5, 30.2, 28.9, 31.1, 30.0}, penelusuran algoritma menghasilkan tabel berikut.

i	T_i	*sum*	*minimum*	*maximum*	*count_high*
1	29.5	29.5	29.5	29.5	0
2	30.2	59.7	29.5	30.2	1
3	28.9	88.6	28.9	30.2	1
4	31.1	119.7	28.9	31.1	2
5	30.0	149.7	28.9	31.1	2

Jadi $\overline{T} = 149.7/5 = 29.94\,^{\circ}\text{C}$, $T_{\text{min}} = 28.9\,^{\circ}\text{C}$, $T_{\text{max}} = 31.1\,^{\circ}\text{C}$, dan terdapat dua data dengan $T_i > 30.0\,^{\circ}\text{C}$. Data 30.0 tidak dihitung karena kondisinya adalah "lebih besar daripada", bukan "lebih besar atau sama dengan".

Setiap temperatur diproses tepat satu kali, sehingga kompleksitas waktunya $O(N)$. Jika data dibaca satu per satu tanpa disimpan seluruhnya, algoritma hanya memerlukan beberapa variabel tambahan, sehingga kebutuhan memorinya $O(1)$.

2.8.4 Implementasi dalam C

Implementasi berikut diperlihatkan sebagai gambaran hubungan *pseudocode* dengan program C; sintaks `scanf`, `if`, dan `for` akan dibahas pada kuliah-kuliah berikutnya.

```c
#include <stdio.h>

int main(void)
{
    int n;
    double limit;

    printf("Number of measurements: ");
    scanf("%d", &n);
    printf("Temperature limit: ");
    scanf("%lf", &limit);

    if (n <= 0)
    {
        printf("Number of measurements must be positive.\n");
        return 1;
    }

    double temperature;
    printf("T1: ");
    scanf("%lf", &temperature);

    double sum = temperature;
    double minimum = temperature;
    double maximum = temperature;
    int count_high = (temperature > limit) ? 1 : 0;

    for (int i = 2; i <= n; i++)
    {
        printf("T%d: ", i);
```

```
        scanf("%lf", &temperature);

        sum = sum + temperature;
        if (temperature < minimum)
        {
            minimum = temperature;
        }
        if (temperature > maximum)
        {
            maximum = temperature;
        }
        if (temperature > limit)
        {
            count_high = count_high + 1;
        }
    }

    double mean = sum / n;

    printf("\nMean        = %.3f C\n", mean);
    printf("Minimum     = %.3f C\n", minimum);
    printf("Maximum     = %.3f C\n", maximum);
    printf("Above limit = %d\n", count_high);

    return 0;
}
```

Dengan data yang sama seperti pada penelusuran, program menghasilkan keluaran berikut (masukan pengguna ditandai dengan warna biru).

```
Number of measurements: 5
Temperature limit: 30
T1: 29.5
T2: 30.2
T3: 28.9
T4: 31.1
T5: 30.0

Mean        = 29.940 C
Minimum     = 28.900 C
Maximum     = 31.100 C
Above limit = 2
```

Keluaran program sama dengan hasil penelusuran manual. Bagian inisialisasi, pengulangan, dan keputusan dalam program berkorespondensi langsung dengan bagian-bagian *pseudocode*; program tersebut pada dasarnya adalah *pseudocode* yang telah diterjemahkan ke sintaks C.

2.8.5 Kebiasaan sebelum menulis program

Sebelum mulai menulis kode, biasakan menjawab pertanyaan berikut:

1. Apa persoalannya? Nyatakan dalam satu atau dua kalimat.
2. Apa masukan dan keluarannya, lengkap dengan satuannya?
3. Model atau persamaan apa yang digunakan, dan apa asumsinya?
4. Dapatkah masalah dipecah menjadi subpersoalan?
5. Apakah diperlukan keputusan? Jika ya, apa kondisinya secara eksplisit?
6. Apakah diperlukan pengulangan? Apa yang berubah pada setiap iterasi dan kapan proses berhenti?
7. Bagaimana memeriksa hasilnya? Siapkan setidaknya satu kasus yang dapat dihitung manual.
8. Seberapa besar pekerjaan komputasinya: $O(1)$, $O(N)$, atau $O(N^2)$?

Kebiasaan ini mengurangi kecenderungan menulis kode dengan cara coba-coba.

2.9 Cek pemahaman

- Apa perbedaan antara masalah, model, algoritma, dan program?
- Mengapa persamaan matematika belum cukup untuk menjadi algoritma?
- Apa yang dimaksud dengan dekomposisi masalah, prakondisi, dan pascakondisi?
- Apa tiga pola dasar aliran kontrol dalam diagram alir?
- Apa perbedaan tanda = dalam matematika dan operasi pemberian nilai $\leftarrow$ dalam algoritma?
- Mengapa inisialisasi *maximum* $\leftarrow$ 0 dapat menghasilkan kesalahan?
- Apa tujuan penelusuran? Apa yang dimaksud dengan keadaan dan invarian?
- Apa yang dimaksud dengan kasus batas? Berikan contohnya.
- Apa arti $O(1)$, $O(N)$, dan $O(N^2)$? Mengapa *Big-O* tidak sama dengan waktu dalam detik?
- Apa perbedaan kompleksitas waktu dan kompleksitas ruang?

2.10 Latihan

1. **Spesifikasi.** Sebuah sensor tekanan menghasilkan nilai P dalam pascal. Tuliskan spesifikasi (masukan, keluaran, prakondisi) dan algoritma kalimat berurutan untuk mengubahnya menjadi kilopascal, $P_{\text{kPa}} = P_{\text{Pa}}/1000$.
2. **Dekomposisi.** Sebuah stasiun cuaca mencatat temperatur, kelembapan, dan tekanan setiap jam selama satu hari. Pecahlah persoalan “membuat laporan harian” menjadi subpersoalan dan gambarkan diagram dekomposisinya.
3. **Diagram alir dengan keputusan.** Buat diagram alir untuk menghitung $P = VI$, lalu perluas agar menampilkan peringatan jika $P > 100$ W.

4. ***Pseudocode* gerak lurus.** Tuliskan *pseudocode* yang menghitung posisi $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$ dan kecepatan $v(t) = v_0 + at$ dari masukan x_0, v_0, a, dan t.
5. **Penelusuran.** Telusuri algoritma Statistics untuk data 4.2, 3.8, 5.1, 2.9, 4.7. Buat tabel yang memuat indeks, data, *sum*, *minimum*, dan *maximum*.
6. **Mencari kesalahan.** Algoritma berikut menginisialisasi $minimum \leftarrow 0$ lalu memperbaruinya jika $x_i < minimum$. Uji dengan data $2, 5, 1, 4$ dan $-2, 5, 1, 4$. Jelaskan mengapa satu kasus uji dapat "menipu" kita, lalu perbaiki algoritmanya.
7. **Kasus batas.** Untuk algoritma TemperatureMonitoring, tentukan keluaran yang diharapkan jika $N = 1$ dan jika semua data tepat sama dengan T_{limit}. Apa yang harus dilakukan algoritma jika $N = 0$?
8. **Kompleksitas.** Tentukan kompleksitas waktu (a) $y \leftarrow ax + b$; (b) penjumlahan N data dengan satu pengulangan; (c) dua pengulangan bersarang yang mencetak semua pasangan (i, j) untuk $i, j = 1, \ldots, N$. Jelaskan alasannya.
9. **Pertumbuhan.** Sebuah komputer melakukan 10^8 operasi per detik. Perkirakan waktu yang dibutuhkan algoritma $O(N)$ dan $O(N^2)$ untuk $N = 10^4$ dan $N = 10^6$. Apa kesimpulan Anda tentang algoritma kuadratik untuk data besar?
10. **Studi kasus.** Untuk data $T = \{27.5,\ 31.2,\ 29.0,\ 32.8,\ 30.1,\ 28.4\}$ dengan $T_{\text{limit}} = 30\,^{\circ}\text{C}$, tentukan secara manual rata-rata, minimum, maksimum, dan banyaknya data di atas ambang. Lakukan penelusuran algoritma TemperatureMonitoring, lalu bandingkan dengan keluaran program C pada kuliah ini.

2.11 Rangkuman

- Berpikir algoritmik adalah proses mengubah persoalan menjadi langkah-langkah komputasi yang jelas dan dapat dilaksanakan.
- Sebelum menulis program, tentukan spesifikasi: masukan, keluaran, asumsi, prakondisi, dan model.
- Persoalan besar dipecah menjadi subpersoalan melalui dekomposisi.
- Algoritma dapat direpresentasikan dengan kalimat berurutan, diagram alir, dan *pseudocode*. Tiga pola aliran kontrol dasarnya adalah urutan, keputusan, dan pengulangan.
- Penelusuran memperlihatkan perubahan keadaan algoritma; invarian menjelaskan mengapa algoritma bekerja.
- Kebenaran algoritma diperiksa dengan kasus sederhana, kasus batas, hasil yang telah diketahui, serta pemeriksaan makna fisis.
- Kompleksitas menggambarkan bagaimana kebutuhan komputasi bertumbuh terhadap ukuran masukan; pola penting pada tahap awal adalah $O(1)$, $O(N)$, dan $O(N^2)$.
- Kuliah berikutnya membahas struktur dasar program C secara sistematis: tipe data, variabel, operator, ekspresi, logika Boolean, serta masukan dan keluaran.

KULIAH

3

Dasar-dasar Pemrograman dalam Bahasa C

3.1 Struktur dasar program C

Pada dua kuliah sebelumnya kita membangun dua fondasi: alur masalah → model → algoritma → program → hasil, serta cara merumuskan algoritma melalui dekomposisi, diagram alir, *pseudocode*, dan penelusuran. Kuliah ini berfokus pada langkah berikutnya,

algoritma ⟶ program C

Di era AI kita tidak perlu menghafal seluruh sintaks C. Namun, kita perlu memahami bagaimana konsep algoritmik—masukan, penyimpanan nilai, perhitungan, keputusan logika, dan keluaran—direpresentasikan dalam C.

C dipilih sebagai bahasa utama pada bagian awal buku ini karena sintaksnya relatif kecil dan terstruktur, hubungan antara program, tipe data, dan memori cukup terlihat, serta banyak digunakan dalam sistem tertanam, mikrokontroler, instrumentasi, sistem kendali, dan komputasi berkinerja tinggi. Banyak bahasa modern juga meminjam sintaks C. Perlu diingat bahwa pemrograman tidak identik dengan satu bahasa: algoritma $E_k \leftarrow 0.5 \cdot m \cdot v \cdot v$ dapat ditulis sebagai `0.5 * mass * velocity * velocity` dalam C maupun `0.5 * mass * velocity**2` dalam Python. Bahasa pemrograman hanyalah media untuk mengekspresikan algoritma.

3.1.1 Bagian-bagian program

Perhatikan kembali program berikut.

```c
#include <stdio.h>

int main(void)
{
    printf("Hello, Teknik Fisika!\n");

    return 0;
}
```

- `#include <stdio.h>` meminta *preprocessor* menyertakan deklarasi pustaka standar masukan-keluaran (*standard input/output*). Berkas berakhiran `.h` disebut *header file*. Kita memerlukannya untuk fungsi seperti `printf` dan `scanf`.
- `int main(void)` adalah fungsi utama, titik awal eksekusi program. Kata `int` menunjukkan bahwa `main` mengembalikan bilangan bulat kepada sistem operasi, sedangkan `(void)` menyatakan bahwa fungsi ini tidak menerima argumen. Tubuh fungsi ditulis di antara kurung kurawal `{` dan `}`, dan pernyataan di dalamnya dijalankan dari atas ke bawah.
- `printf("...");` adalah sebuah **pernyataan** (*statement*). Banyak pernyataan dalam C diakhiri titik koma; tanpa titik koma, *compiler* melaporkan galat sintaksis.
- `return 0;` mengakhiri `main` dan mengembalikan nilai 0, yang menurut konvensi berarti program selesai secara normal. Nilai selain nol biasanya menandakan kegagalan.

C tidak menganggap baris baru sebagai akhir pernyataan; akhir pernyataan ditentukan oleh titik koma. Karena itu, ekspresi panjang boleh ditulis dalam beberapa baris agar mudah dibaca.

Program dikompilasi dan dijalankan seperti pada Kuliah 1:

```
gcc -Wall -Wextra hello.c -o hello
./hello
```

```
Hello, Teknik Fisika!
```

Peringatan *compiler* harus diperlakukan sebagai informasi penting, bukan gangguan. Program yang dikompilasi tanpa peringatan belum tentu benar, tetapi program yang menghasilkan banyak peringatan hampir pasti perlu diperiksa.

3.1.2 Komentar

Komentar memberikan informasi kepada manusia yang membaca program dan tidak dijalankan komputer. Komentar satu baris diawali `//`, sedangkan komentar beberapa baris ditulis di antara `/*` dan `*/`. Komentar sebaiknya menjelaskan hal yang tidak langsung terlihat dari kode. Komentar seperti `x = x + 1; // tambah x dengan 1` tidak menambah informasi, sedangkan `double velocity = 20.0; // m/s` berguna karena menyatakan satuan.

3.2 Variabel, tipe data, dan konstanta

Tempat penyimpanan bernama untuk sebuah nilai disebut **variabel**. Pernyataan

```
double mass;            /* deklarasi                         */
double velocity = 3.0;  /* deklarasi sekaligus inisialisasi */
mass = 2.0;             /* assignment (pemberian nilai)      */
```

memperlihatkan tiga konsep. **Deklarasi** memberi tahu *compiler* nama dan tipe variabel. **Inisialisasi** memberikan nilai awal ketika variabel dibuat. ***Assignment*** memberikan nilai baru kepada variabel yang sudah ada.



Simbol `=` dalam C bukan tanda kesamaan matematika, melainkan perintah "simpan nilai di sebelah kanan ke variabel di sebelah kiri". Karena itu, `x = x + 1;` valid: ambil nilai lama `x`, tambahkan satu, lalu simpan hasilnya kembali ke `x`. Jika semula $x = 4$, setelah pernyataan tersebut $x = 5$.

Variabel lokal yang belum diberi nilai berisi "sampah" memori, sehingga hasil program yang menggunakannya tidak dapat diandalkan. Biasakan selalu memberi nilai sebelum variabel digunakan.

3.2.1 Nama variabel

Nama variabel (*identifier*) dapat menggunakan huruf, angka, dan garis bawah, tetapi tidak boleh diawali angka; `x0`, `sensor_value`, dan `pressure_1` valid, sedangkan `2mass` tidak. C juga peka terhadap besar-kecil huruf (*case-sensitive*): `temperature`, `Temperature`, dan `TEMPERATURE` adalah tiga nama yang berbeda.

Gunakan nama yang bermakna. Program `double c = a * b;` dan `double power = voltage * current;` dapat menghasilkan nilai yang sama, tetapi yang kedua jauh lebih mudah dipahami. Dalam program teknik, satuan juga dapat dicantumkan pada nama, misalnya `temperature_celsius` atau `pressure_pa`, terutama jika beberapa sistem satuan digunakan sekaligus.

3.2.2 Tipe data dasar

Tipe data menentukan jenis nilai yang dapat disimpan, cara nilai direpresentasikan, operasi yang dapat dilakukan, dan banyaknya memori yang digunakan. Tipe dasar yang kita gunakan adalah sebagai berikut.

Tipe	Kegunaan	Contoh
`int`	bilangan bulat	`int sample_count = 100;`
`float`	bilangan real presisi tunggal	`float t = 25.5f;`
`double`	bilangan real presisi ganda	`double p = 101325.0;`
`char`	satu karakter	`char grade = 'A';`
`bool`	nilai logika	`bool alarm = false;`

Literal `25.5f` bertipe `float`, sedangkan `25.5` tanpa akhiran bertipe `double`. Karakter ditulis dengan tanda petik tunggal (`'A'`), sedangkan petik ganda (`"A"`) menyatakan *string* yang akan dibahas pada Kuliah 6. Tipe `bool` dengan nilai `true` dan `false` tersedia melalui `#include <stdbool.h>`.

Untuk komputasi ilmiah dalam buku ini, `double` adalah pilihan bawaan untuk besaran real karena presisinya jauh lebih tinggi daripada `float`. Ukuran setiap tipe dapat diperiksa dengan operator `sizeof`, sedangkan batas dan presisinya tersedia di `limits.h` dan `float.h`.

```c
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char   : %zu byte\n", sizeof(char));
    printf("int    : %zu byte\n", sizeof(int));
    printf("float  : %zu byte\n", sizeof(float));
    printf("double : %zu byte\n", sizeof(double));
    printf("bool   : %zu byte\n", sizeof(bool));

    printf("\nINT_MIN = %d\n", INT_MIN);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("FLT_DIG = %d digit\n", FLT_DIG);
    printf("DBL_DIG = %d digit\n", DBL_DIG);

    return 0;
}
```

Pada komputer penulis (Linux 64 bit), keluarannya adalah

```
char   : 1 byte
int    : 4 byte
float  : 4 byte
double : 8 byte
bool   : 1 byte

INT_MIN = -2147483648
INT_MAX = 2147483647
FLT_DIG = 6 digit
DBL_DIG = 15 digit
```

Standar C tidak menetapkan ukuran yang sama untuk semua arsitektur, sehingga hasil di komputer lain dapat berbeda. Yang lebih penting adalah memahami rantai

$$\boxed{\text{tipe data} \longrightarrow \text{representasi} \longrightarrow \text{rentang dan presisi}}$$

Dengan `int` 32 bit, rentangnya $-2^{31} \le x \le 2^{31} - 1$, sesuai dengan pembahasan bilangan bulat bertanda pada Kuliah 1. `FLT_DIG` dan `DBL_DIG` menyatakan banyaknya digit desimal yang dapat diandalkan: sekitar 6–7 digit untuk `float` dan 15–16 digit untuk `double`.

3.2.3 Konstanta dan literal

Nilai yang tidak dimaksudkan berubah dideklarasikan dengan `const`, misalnya

```
const double gravity = 9.81;               /* m/s^2       */
const double gas_constant = 8.314462618;   /* J/(mol K)   */
const double planck_constant = 6.62607015e-34;  /* J s    */
```

Kata `const` mendokumentasikan maksud program dan membuat *compiler* menolak upaya mengubah nilai tersebut.

Nilai yang ditulis langsung di dalam kode disebut **literal**. Literal bilangan bulat ditulis seperti `0`, `10`, atau `-25`. Literal *floating-point* dapat menggunakan notasi ilmiah: `1.0e-6` berarti 1.0×10^{-6} dan `2.99792458e8` berarti 2.99792458×10^{8}. Notasi ini sangat berguna untuk besaran fisika yang sangat besar atau sangat kecil.

3.3 Operator dan ekspresi aritmetika

C menyediakan operator aritmetika `+`, `-`, `*`, `/`, serta `%` untuk sisa pembagian bilangan bulat. Model fisika diterjemahkan menjadi ekspresi, misalnya $P = VI$ menjadi `power = voltage * current;` dan $T_f = T_i + \Delta T$ menjadi `final_temperature = initial_temperature + delta_temperature;`.

Dua hal perlu diperhatikan. Pertama, C tidak mempunyai perkalian implisit: $\frac{1}{2}mv^2$ harus ditulis `0.5 * mass * velocity * velocity`. Kedua, C tidak mempunyai operator pangkat. Ekspresi `v ^ 2` *bukan* v^2, melainkan operasi XOR bit demi bit. Untuk kuadrat sederhana tulislah `v * v`; untuk pangkat umum gunakan fungsi `pow(x, y)` dari `math.h`, misalnya `pow(x, 3.5)`. Program yang memakai `math.h` pada Linux dikompilasi dengan tambahan `-lm`.

3.3.1 Pembagian bilangan bulat

Salah satu sumber kesalahan paling umum adalah pembagian bilangan bulat. Jika kedua operand bertipe `int`, C melakukan pembagian bilangan bulat dan membuang bagian pecahannya.

```
#include <stdio.h>

int main(void)
{
    int sum = 7;
    int n = 2;

    double mean_wrong = sum / n;
    double mean_right = (double)sum / n;

    printf("5 / 2        = %d\n", 5 / 2);
    printf("5.0 / 2.0    = %f\n", 5.0 / 2.0);
    printf("sum / n      = %f\n", mean_wrong);
    printf("(double)sum/n = %f\n", mean_right);
    printf("17 %% 5       = %d\n", 17 % 5);

    return 0;
}
```

```
5 / 2         = 2
5.0 / 2.0     = 2.500000
sum / n       = 3.000000
(double)sum/n = 3.500000
17 % 5        = 2
```

Pada `mean_wrong`, operasi `sum / n` dilakukan sebagai pembagian bilangan bulat terlebih dahulu (7/2 = 3), baru kemudian hasilnya diubah menjadi `double`. Contoh ini menegaskan bahwa **tipe data memengaruhi hasil perhitungan**. Perbaikannya adalah mengubah salah satu operand menjadi `double`, atau menggunakan `double` sejak awal.

Operator `%` menghasilkan sisa pembagian: 17 mod 5 = 2 karena $17 = 3(5) + 2$. Operator ini berguna untuk memeriksa bilangan genap atau ganjil (`n % 2 == 0`), pola periodik, dan pengindeksan.

3.3.2 Prioritas operator

Perkalian, pembagian, dan modulo dikerjakan sebelum penjumlahan dan pengurangan, dan ekspresi dalam tanda kurung dikerjakan lebih dahulu. Jadi `a + b * c` berarti $a + bc$, sedangkan $(a+b)c$ harus ditulis `(a + b) * c`. Jangan terlalu bergantung pada hafalan tabel prioritas; jika ekspresi mulai sulit dibaca, tambahkan tanda kurung atau pecah menjadi variabel antara:

```c
double velocity_squared = velocity * velocity;
double energy = 0.5 * mass * velocity_squared;
```

Variabel antara memudahkan pemeriksaan dan *debugging*, walaupun terlalu banyak variabel yang tidak perlu juga membuat program panjang.

3.3.3 Konversi tipe

Ketika tipe berbeda dicampur dalam satu ekspresi, misalnya `double sum` dibagi `int n`, C melakukan **konversi implisit** sehingga `n` diperlakukan sebagai `double`. Konversi juga dapat diminta secara eksplisit dengan ***cast***, seperti `(double)sum` di atas.

Konversi ke tipe yang lebih "sempit" dapat menghilangkan informasi: `int y = (int)3.9;` menghasilkan $y = 3$ karena bagian pecahan dibuang. Karena itu, *cast* tidak boleh digunakan hanya untuk membungkam peringatan *compiler* tanpa memahami akibatnya.

3.4 Operator relasional dan logika Boolean

Program sering perlu membandingkan nilai. Operator relasional C adalah `<`, `<=`, `>`, `>=`, `==` (sama dengan), dan `!=` (tidak sama dengan). Hasil perbandingan adalah nilai logika benar atau salah; misalnya `temperature > 30.0` menanyakan apakah temperatur lebih besar dari 30.

Jangan tertukar antara `=` dan `==`. Pernyataan `x = 5;` berarti "simpan 5 ke `x`", sedangkan ekspresi `x == 5` berarti "apakah `x` sama dengan 5?". Kesalahan ini sangat umum dan sering lolos tanpa galat sintaksis.

Operasi logika $p \wedge q$, $p \vee q$, dan $\neg p$ ditulis dalam C sebagai `p && q` (AND), `p || q` (OR), dan `!p` (NOT). Tabel kebenarannya adalah sebagai berikut.

p	q	$p \wedge q$	$p \vee q$	$\neg p$
salah	salah	salah	salah	benar
salah	benar	salah	benar	benar
benar	salah	salah	benar	salah
benar	benar	benar	benar	salah

Kondisi matematis $20 \leq T \leq 30$ *tidak* boleh ditulis `20.0 <= temperature <= 30.0`; ekspresi tersebut tidak bermakna seperti notasi matematika. Kondisi tersebut harus dipecah menjadi dua perbandingan yang digabung dengan AND, $(T \geq 20) \wedge (T \leq 30)$:

```
bool safe  = temperature >= 20.0 && temperature <= 30.0;
bool alarm = temperature < 10.0 || temperature > 50.0;
bool off   = !sensor_active;
```

Nilai `safe` benar jika kedua kondisi benar, `alarm` benar jika salah satu kondisi benar, dan `off` benar jika sensor tidak aktif. Ekspresi Boolean ini menjadi dasar percabangan dan pengulangan pada Kuliah 4.

3.4.1 Membandingkan bilangan *floating-point*

Pada Kuliah 1 kita melihat bahwa $0.1 + 0.2$ tidak tepat sama dengan 0.3 dalam representasi biner. Karena itu, kesamaan eksak `x == y` berbahaya untuk bilangan *floating-point*. Dalam komputasi numerik kita memeriksa $|x - y| < \varepsilon$ dengan fungsi `fabs` dari `math.h`.

```
#include <stdio.h>
#include <math.h>
#include <stdbool.h>

int main(void)
{
    double x = 0.1 + 0.2;
    double y = 0.3;
    const double tolerance = 1e-12;

    bool exactly_equal = (x == y);
    bool approximately_equal = fabs(x - y) < tolerance;

    printf("x = %.17f\n", x);
    printf("y = %.17f\n", y);
    printf("exactly equal       = %d\n", exactly_equal);
    printf("approximately equal = %d\n", approximately_equal);

    return 0;
}
```

```
gcc -Wall -Wextra compare.c -o compare -lm
```

```
./compare
```

```
x = 0.30000000000000004
y = 0.29999999999999999
exactly equal       = 0
approximately equal = 1
```

Nilai `bool` dicetak sebagai `0` (salah) atau `1` (benar). Pemilihan toleransi akan dibahas lebih lanjut ketika kita mempelajari algoritma numerik; yang penting sekarang adalah menyadari bahwa tipe *floating-point* mempunyai konsekuensi numerik.

3.5 Masukan dan keluaran

Program yang hanya menggunakan nilai tetap kegunaannya terbatas. Pustaka `stdio.h` menyediakan `printf` untuk keluaran dan `scanf` untuk masukan.

3.5.1 Menampilkan nilai dengan `printf`

Fungsi `printf` menerima sebuah *format string* yang dapat memuat penentu format (*format specifier*) dan karakter khusus seperti `\n` (baris baru).

```c
#include <stdio.h>

int main(void)
{
    int n = 25;
    double temperature = 29.875;
    double h = 6.62607015e-34;
    char grade = 'A';

    printf("n           = %d\n", n);
    printf("temperature = %f C\n", temperature);
    printf("temperature = %.2f C\n", temperature);
    printf("h           = %e J s\n", h);
    printf("h           = %g J s\n", h);
    printf("grade       = %c\n", grade);

    return 0;
}
```

```
n           = 25
temperature = 29.875000 C
temperature = 29.88 C
h           = 6.626070e-34 J s
h           = 6.62607e-34 J s
grade       = A
```

Format `%.2f` menampilkan dua digit setelah titik desimal (dengan pembulatan), `%e` menampilkan notasi ilmiah, dan `%g` memilih bentuk yang lebih ringkas. Ringkasan format yang sering digunakan:

Tipe	`printf`	`scanf`
`int`	`%d`	`%d`
`float`	`%f`	`%f`
`double`	`%f`, `%e`, `%g`	`%lf`
`char`	`%c`	`%c`
string	`%s`	`%s`

Perhatikan asimetri pada baris `double`: untuk `printf`, `%f` dipakai baik untuk `float` maupun `double`, tetapi untuk `scanf`, `double` harus dibaca dengan `%lf`.

3.5.2 Membaca masukan dengan `scanf`

Fungsi `scanf` membaca masukan sesuai format dan menyimpannya ke variabel. Tanda `&` di depan nama variabel berarti "alamat variabel"; `scanf` memerlukan alamat agar dapat menulis nilai ke variabel tersebut. Konsep alamat akan dibahas pada Kuliah 6 ketika kita mempelajari *pointer*.

`scanf` mengembalikan banyaknya item yang berhasil dibaca. Nilai ini dapat digunakan untuk memeriksa masukan secara sederhana:

```c
#include <stdio.h>

int main(void)
{
    double mass;

    printf("Mass (kg): ");
    if (scanf("%lf", &mass) != 1)
    {
        printf("Invalid input.\n");
        return 1;
    }

    printf("Mass = %.3f kg\n", mass);

    return 0;
}
```

Jika pengguna mengetik teks yang bukan bilangan, program menolak masukan:

```
Mass (kg): dua
Invalid input.
```

Struktur `if` akan dibahas sistematis pada Kuliah 4. Contoh ini sudah menunjukkan prinsip penting: program teknik sebaiknya tidak menganggap masukan pengguna selalu benar.

3.6 Studi kasus: model fisika sederhana

Setiap studi kasus berikut mengikuti pola yang sama: spesifikasi, *pseudocode*, implementasi, lalu pemeriksaan dengan hitungan manual.

3.6.1 Daya pada resistor

Program membaca arus I dan hambatan R, lalu menghitung $V = IR$ dan $P = VI$. Masukannya I dan R dengan asumsi $R \geq 0$; keluarannya V dan P.

> **INPUT** *current*, *resistance*
> *voltage* ⟵ *current* · *resistance*
> *power* ⟵ *voltage* · *current*
> **OUTPUT** *voltage*, *power*

```c
#include <stdio.h>

int main(void)
{
    double current;
    double resistance;

    printf("Current (A): ");
    scanf("%lf", &current);
    printf("Resistance (ohm): ");
    scanf("%lf", &resistance);

    double voltage = current * resistance;
    double power = voltage * current;

    printf("\nVoltage = %.3f V\n", voltage);
    printf("Power   = %.3f W\n", power);

    return 0;
}
```

```
Current (A): 2.5
Resistance (ohm): 10

Voltage = 25.000 V
Power   = 62.500 W
```

Hitungan manual memberikan $V = (2.5)(10) = 25$ V dan $P = (25)(2.5) = 62.5$ W, sesuai dengan keluaran program. Pemeriksaan satuan juga konsisten: $[I][R] = \text{A} \cdot \Omega = \text{V}$ dan $[V][I] = \text{V} \cdot \text{A} = \text{W}$. Pemeriksaan satuan adalah salah satu alat *debugging* paling ampuh dalam komputasi fisika.

3.6.2 Konversi temperatur

Hubungan Celsius dan Kelvin adalah $T_K = T_C + 273.15$. Contoh ini sederhana, tetapi menunjukkan manfaat nama variabel yang menyertakan satuan: `temperature_c` dan `temperature_k` jauh lebih aman daripada `t` dan `x`.

```c
#include <stdio.h>

int main(void)
{
    double temperature_c;

    printf("Temperature (C): ");
    scanf("%lf", &temperature_c);

    double temperature_k = temperature_c + 273.15;

    printf("Temperature = %.2f K\n", temperature_k);

    return 0;
}
```

```
Temperature (C): 25
Temperature = 298.15 K
```

3.6.3 Gas ideal

Dari $PV = nRT$, tekanan gas ideal adalah $P = nRT/V$ dengan $R = 8.314462618\ \mathrm{J\,mol^{-1}\,K^{-1}}$, yang dideklarasikan sebagai konstanta.

```c
#include <stdio.h>

int main(void)
{
    const double gas_constant = 8.314462618;  /* J/(mol K) */

    double amount, temperature, volume;

    printf("Amount of substance (mol): ");
    scanf("%lf", &amount);
    printf("Temperature (K): ");
    scanf("%lf", &temperature);
    printf("Volume (m^3): ");
    scanf("%lf", &volume);

    double pressure = amount * gas_constant * temperature / volume;

    printf("Pressure = %.6e Pa\n", pressure);

    return 0;
```

```
}
```

```
Amount of substance (mol): 1
Temperature (K): 300
Volume (m^3): 0.024
Pressure = 1.039308e+05 Pa
```

Secara manual, $P = (1)(8.314462618)(300)/0.024 \approx 1.04 \times 10^5$ Pa. Nilai tersebut masuk akal karena berada pada orde tekanan atmosfer. Di sini kita melihat pentingnya **orde besaran** sebagai pemeriksaan fisis.

3.6.4 Energi foton

Energi foton $E = hf$ dengan $h = 6.62607015 \times 10^{-34}$ J s melibatkan bilangan yang sangat kecil, sehingga format notasi ilmiah `%e` sangat tepat.

```c
#include <stdio.h>

int main(void)
{
    const double planck_constant = 6.62607015e-34;  /* J s */
    double frequency;

    printf("Frequency (Hz): ");
    scanf("%lf", &frequency);

    double energy = planck_constant * frequency;

    printf("Photon energy = %.6e J\n", energy);
    printf("              = %.4f eV\n", energy / 1.602176634e-19);

    return 0;
}
```

```
Frequency (Hz): 5.0e14
Photon energy = 3.313035e-19 J
              = 2.0678 eV
```

Untuk cahaya tampak dengan $f = 5.0 \times 10^{14}$ Hz, $E \approx 3.31 \times 10^{-19}$ J atau sekitar 2 eV, sesuai dengan energi foton cahaya tampak yang dikenal dari fisika modern.

3.7 Kesalahan umum dan kebiasaan yang baik

Saat belajar C, kita akan bertemu tiga kategori masalah yang telah diperkenalkan pada Kuliah 1: **galat sintaksis** yang ditolak *compiler*, **peringatan** yang menandai kode mencurigakan, dan **galat logika** yang hanya dapat ditemukan melalui pengujian, hitungan manual, pemeriksaan satuan, dan penalaran fisika. Program berikut mengandung beberapa kesalahan khas sekaligus.

```c
#include <stdio.h>

int main(void)
{
    int mass = 2.5;
    double velocity = 3.0;

    double energy = 1 / 2 * mass * velocity * velocity;

    printf("Energy = %f J\n", energy);
    return 0;
}
```

```
Energy = 0.000000 J
```

Program dikompilasi dan berjalan, tetapi hasilnya nol. Ada dua kesalahan: `mass` bertipe `int` sehingga 2.5 terpotong menjadi 2, dan `1 / 2` adalah pembagian bilangan bulat yang bernilai 0. Kesalahan khas lainnya diringkas dalam tabel berikut.

Salah	Benar
`double mass = 2.0` (tanpa titik koma)	`double mass = 2.0;`
`double y = x + 1.0;` dengan `x` belum diberi nilai	beri nilai awal `double x = 0.0;`
`double x = 1 / 2;`	`double x = 1.0 / 2.0;`
`scanf("%f", &x);` untuk `double x`	`scanf("%lf", &x);`
`scanf("%lf", x);`	`scanf("%lf", &x);`
`if (mode = 1)`	`if (mode == 1)`
`energy = 0.5 * m * v ^ 2;`	`energy = 0.5 * m * v * v;`
`20.0 <= T <= 30.0`	`T >= 20.0 && T <= 30.0`

Beberapa kebiasaan baik yang perlu dibangun sejak awal:

- gunakan nama variabel yang informatif dan nyatakan satuan, baik pada nama maupun komentar;
- gunakan `const` untuk konstanta fisika dan `double` untuk besaran real;
- gunakan indentasi yang konsisten agar struktur program terlihat;
- pecah ekspresi panjang menjadi variabel antara bila perlu;
- selalu kompilasi dengan `-Wall -Wextra` dan pahami setiap peringatan;
- prediksi keluaran dengan hitungan manual sebelum menjalankan program, lalu bandingkan.

Kebiasaan terakhir jauh lebih efektif daripada menekan tombol *Run* berulang kali sambil berharap hasilnya benar.

3.8 Cek pemahaman

- Apa peran `#include <stdio.h>` dan fungsi `main`?
- Apa perbedaan deklarasi, inisialisasi, dan *assignment*?
- Mengapa `double` lebih sesuai daripada `float` untuk komputasi ilmiah?
- Apa perbedaan literal `2`, `2.0`, dan `2.0f`?
- Mengapa `5 / 2` berbeda dari `5.0 / 2.0`? Apa fungsi operator `%`?
- Apa perbedaan `=` dan `==`?
- Bagaimana menuliskan kondisi $20 \le T \le 30$ dalam C?
- Mengapa `v ^ 2` bukan cara menulis v^2?
- Apa perbedaan konversi implisit dan *cast*? Mengapa konversi `double` ke `int` dapat kehilangan informasi?
- Mengapa `scanf` membutuhkan `&` dan format `%lf` untuk `double`?

3.9 Latihan

1. **Struktur program.** Tulis program yang menampilkan `Algoritma dan Pemrograman` dan `Teknik Fisika` pada dua baris berbeda. Kompilasi dengan `-Wall -Wextra` dan pastikan tidak ada peringatan.
2. **Pemilihan tipe.** Pilih tipe yang sesuai (`int`, `double`, `char`, atau `bool`) untuk jumlah sensor, temperatur, tekanan, karakter status, dan kondisi alarm. Jelaskan alasannya.
3. **Ukuran dan batas tipe.** Jalankan program `sizeof` pada kuliah ini di komputer Anda. Bandingkan hasilnya dengan teman yang menggunakan sistem operasi lain. Apakah hasilnya harus sama? Bandingkan `INT_MAX` dengan prediksi $2^{31} - 1$.
4. **Pembagian bilangan bulat.** Prediksi keluaran program yang menghitung `double x = a / b;` dan `double y = (double)a / b;` untuk `int a = 7, b = 2;`. Jalankan dan jelaskan perbedaannya.
5. **Konversi temperatur.** Perluas program konversi temperatur agar juga menampilkan temperatur dalam Fahrenheit, $T_F = \frac{9}{5}T_C + 32$. Uji untuk 0, 25, dan 100 °C; hitung hasilnya secara manual terlebih dahulu. Waspadai pembagian bilangan bulat pada $\frac{9}{5}$!
6. **Gerak lurus.** Buat program yang membaca x_0, v_0, a, dan t, lalu menghitung $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$ dan $v(t) = v_0 + at$. Uji dengan $x_0 = 2$ m, $v_0 = 5$ m/s, $a = 3$ m/s^2, $t = 4$ s dan prediksi keluarannya.
7. **Energi foton dan panjang gelombang.** Buat program yang membaca panjang gelombang λ (dalam nm), lalu menghitung frekuensi $f = c/\lambda$ dan energi foton $E = hf$ dalam joule dan elektronvolt. Uji untuk $\lambda = 500$ nm.
8. **Ekspresi Boolean.** Tuliskan ekspresi C untuk: (a) $T > 30$ °C; (b) $20 \le T \le 30$; (c) T di luar rentang aman $10 \le T \le 50$; (d) sensor aktif dan $T > 40$; (e) sensor tidak aktif.
9. **Mencari kesalahan.** Identifikasi semua kesalahan pada program berikut, lalu tulis versi yang benar: `int mass = 2.5; scanf("%f", velocity); double energy = 1 / 2 * mass * velocity ^ 2; printf("Energy = %d\n", energy);`.

10. **Validasi masukan.** Modifikasi program daya resistor sehingga memeriksa nilai kembalian `scanf` untuk kedua masukan. Jika pembacaan gagal, tampilkan `Invalid input.` dan akhiri program dengan `return 1;`.

3.10 Rangkuman

- Program C terdiri atas direktif `#include`, fungsi `main`, dan pernyataan yang diakhiri titik koma; program dikompilasi menjadi *executable* sebelum dijalankan.
- Variabel adalah lokasi penyimpanan bernama yang bertipe. Deklarasi menentukan nama dan tipe, inisialisasi memberi nilai awal, dan *assignment* (`=`) memberi nilai baru.
- Tipe dasar meliputi `int`, `float`, `double`, `char`, dan `bool`. Tipe menentukan representasi, rentang, dan presisi; `double` adalah pilihan bawaan untuk besaran real.
- Konstanta dideklarasikan dengan `const`; literal dapat ditulis dalam notasi ilmiah seperti `6.022e23`.
- Pembagian dua bilangan bulat membuang bagian pecahan. C tidak mempunyai operator pangkat; gunakan `x * x` atau `pow`.
- Operator relasional (`<`, `<=`, `>`, `>=`, `==`, `!=`) dan logika (`&&`, `||`, `!`) membentuk ekspresi Boolean. Kondisi rentang harus dipecah menjadi dua perbandingan.
- `printf` menampilkan keluaran dengan format seperti `%d`, `%f`, dan `%e`; `scanf` membaca masukan dan memerlukan alamat variabel serta format `%lf` untuk `double`.
- Program yang berhasil dikompilasi belum tentu benar. Hasil harus diperiksa dengan hitungan manual, satuan, orde besaran, dan penalaran fisika.
- Kuliah berikutnya menggunakan ekspresi Boolean untuk membangun aliran kontrol program melalui percabangan dan pengulangan.

KULIAH

4

Percabangan dan Pengulangan

4.1 Aliran kontrol dalam program

Program-program pada Kuliah 3 berjalan lurus: masukan, proses, lalu keluaran. Banyak persoalan rekayasa tidak dapat diselesaikan dengan urutan lurus seperti itu. Program sering perlu **mengambil keputusan**: jika temperatur terlalu tinggi, aktifkan peringatan; jika tekanan di luar rentang aman, tampilkan alarm; jika masukan tidak valid, minta pengguna mengulanginya. Program juga sering perlu **mengulangi proses**: membaca 100 data sensor, menghitung posisi benda untuk banyak nilai waktu, atau mengulang perhitungan sampai kondisi tertentu tercapai.

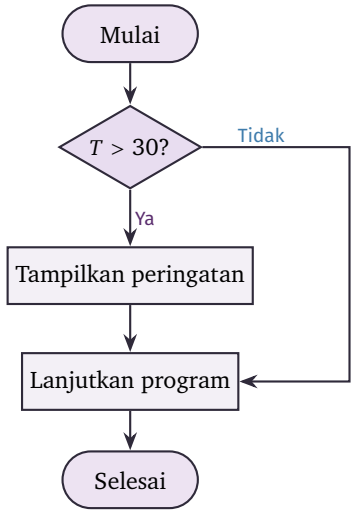
Kedua kemampuan tersebut disebut **percabangan** (*branching*) dan **pengulangan** (*looping* atau *iteration*). Bersama urutan, keduanya membentuk tiga pola dasar **aliran kontrol** (*control flow*) yang telah kita lihat dalam diagram alir pada Kuliah 2. Kuliah ini membahas bagaimana ketiga pola tersebut ditulis dalam C: `if`, `else`, dan `switch` untuk percabangan, serta `while`, `do-while`, dan `for` untuk pengulangan.

4.2 Percabangan `if` dan `if-else`

Bentuk percabangan paling sederhana menjalankan suatu blok hanya jika kondisinya benar:

```
if (temperature > 30.0)
{
    printf("Temperature is high.\n");
}
```

Bagian `temperature > 30.0` adalah ekspresi Boolean. Jika bernilai benar, blok di dalam kurung kurawal dijalankan; jika salah, blok tersebut dilewati dan program melanjutkan ke pernyataan berikutnya. Diagram alirnya adalah sebagai berikut.



Misalkan sebuah sistem termal harus memberikan peringatan jika $T > 80\,^{\circ}\text{C}$.

```c
#include <stdio.h>

int main(void)
{
    double temperature;

    printf("Temperature (C): ");
    scanf("%lf", &temperature);

    if (temperature > 80.0)
    {
        printf("WARNING: temperature too high.\n");
    }

    printf("Measurement complete.\n");

    return 0;
}
```

Dengan masukan 85 dan 60, keluarannya masing-masing adalah

```
Temperature (C): 85
WARNING: temperature too high.
Measurement complete.
```

```
Temperature (C): 60
Measurement complete.
```

Perhatikan bahwa `if` tidak menghentikan program; `if` hanya menentukan apakah suatu blok dijalankan atau dilewati.

4.2.1 Dua kemungkinan: if-else

Sering kali kita harus memilih satu di antara dua kemungkinan. Dalam *pseudocode* bentuknya **IF** kondisi … **ELSE** … **END IF**, dan dalam C ditulis dengan `if` dan `else`. Misalkan sensor dinyatakan normal jika $20\,^{\circ}\text{C} \le T \le 30\,^{\circ}\text{C}$. Kondisi rentang ini harus dipecah menjadi dua perbandingan (Kuliah 3).

```c
#include <stdio.h>

int main(void)
{
    double temperature;

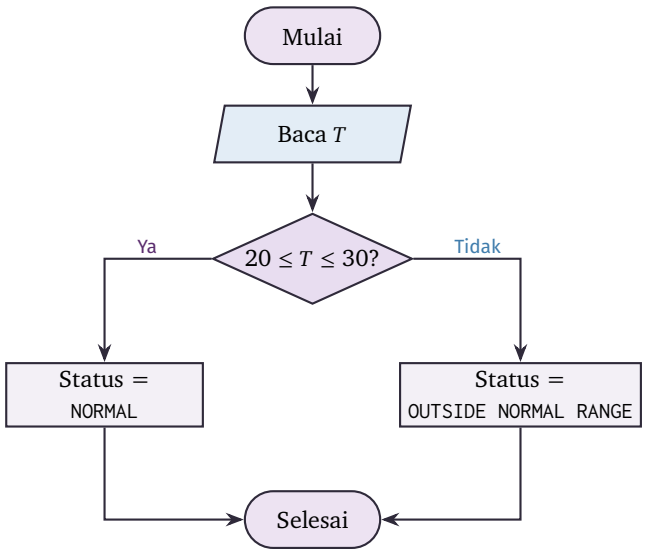
    printf("Temperature (C): ");
    scanf("%lf", &temperature);

    if (temperature >= 20.0 && temperature <= 30.0)
    {
        printf("Status: NORMAL\n");
    }
    else
    {
        printf("Status: OUTSIDE NORMAL RANGE\n");
    }

    return 0;
}
```

```
Temperature (C): 34.5
Status: OUTSIDE NORMAL RANGE
```

Pada struktur `if-else`, tepat satu dari dua blok pasti dijalankan.



4.3 Percabangan banyak arah dan bersarang

Persoalan rekayasa sering memiliki lebih dari dua kategori. Klasifikasi temperatur dari Kuliah 2 (`DINGIN` jika $T < 20$, `NORMAL` jika $20 \le T \le 30$, `PANAS` jika $T > 30$) ditulis dengan rantai `if`–`else if`–`else`:

```
if (temperature < 20.0)
{
    printf("Status: DINGIN\n");
}
else if (temperature <= 30.0)
{
    printf("Status: NORMAL\n");
}
else
{
    printf("Status: PANAS\n");
}
```

Pada cabang kedua kita cukup menulis `temperature <= 30.0`, bukan `temperature >= 20.0 && temperature <= 30.0`. Alasannya, cabang `else if` hanya diperiksa jika kondisi sebelumnya salah; ketika program sampai di sana, $T \ge 20$ sudah pasti benar.

Akibatnya, **urutan kondisi sangat penting**. Pada potongan berikut, cabang kedua tidak akan pernah dijalankan, karena setiap $T < 20$ juga memenuhi $T \le 30$ sehingga sudah ditangkap oleh cabang pertama.

```
if (temperature <= 30.0)
{
    printf("Not hot\n");
}
else if (temperature < 20.0)   /* tidak pernah tercapai */
```

```
{
    printf("Cold\n");
}
```

Kondisi yang terlalu umum dapat menutupi kondisi yang lebih khusus. Sebaliknya, kondisi yang tidak lengkap dapat membuat sebagian kasus tidak ditangani sama sekali: jika hanya ada cabang $T < 20$ dan $T > 30$, nilai $20 \le T \le 30$ tidak menghasilkan keluaran apa pun.

4.3.1 Contoh: klasifikasi aliran fluida

Bilangan Reynolds $Re = \rho v L / \mu$ menentukan rezim aliran fluida. Sebagai ilustrasi, kita gunakan batas sederhana: laminar jika $Re < 2300$, transisi jika $2300 \le Re < 4000$, dan turbulen jika $Re \ge 4000$. (Nilai batas yang sebenarnya bergantung pada konfigurasi aliran.)

```
#include <stdio.h>

int main(void)
{
    double density, velocity, length, viscosity;

    printf("Density (kg/m^3): ");
    scanf("%lf", &density);
    printf("Velocity (m/s): ");
    scanf("%lf", &velocity);
    printf("Characteristic length (m): ");
    scanf("%lf", &length);
    printf("Dynamic viscosity (Pa s): ");
    scanf("%lf", &viscosity);

    double reynolds = density * velocity * length / viscosity;

    printf("Re = %.1f\n", reynolds);

    if (reynolds < 2300.0)
    {
        printf("Flow regime: laminar\n");
    }
    else if (reynolds < 4000.0)
    {
        printf("Flow regime: transition\n");
    }
    else
    {
        printf("Flow regime: turbulent\n");
    }

    return 0;
}
```

Untuk air ($\rho = 1000$ kg/m^3, $\mu = 1.0 \times 10^{-3}$ Pa s) yang mengalir 0.5 m/s dalam pipa berdiameter 2 cm:

```
Density (kg/m^3): 1000
Velocity (m/s): 0.5
Characteristic length (m): 0.02
Dynamic viscosity (Pa s): 1.0e-3
Re = 10000.0
Flow regime: turbulent
```

Program ini memperlihatkan rangkaian model fisika → perhitungan → keputusan → klasifikasi.

4.3.2 Percabangan bersarang

Sebuah `if` dapat berada di dalam `if` lain (*nested if*). Alarm yang hanya aktif jika sensor aktif *dan* $T > 80$ dapat ditulis dengan dua cara yang setara:

```c
if (sensor_active)
{
    if (temperature > 80.0)
    {
        printf("ALARM\n");
    }
}
```

atau

```c
if (sensor_active && temperature > 80.0)
{
    printf("ALARM\n");
}
```

Percabangan yang terlalu dalam (empat atau lima tingkat) membuat kode sulit dibaca. Jika kondisinya dapat digabung secara wajar dengan `&&`, bentuk gabungan biasanya lebih mudah dipahami. Jika kondisi sangat panjang, simpan dalam variabel Boolean bernama, misalnya `bool safe_temperature = temperature >= 20.0 && temperature <= 30.0;`.

4.4 `switch` dan operator kondisi

Untuk memilih satu dari beberapa kasus berdasarkan *satu nilai diskret* (bilangan bulat atau karakter), C menyediakan `switch`. Misalkan sebuah alat ukur mempunyai mode 1 (*standby*), 2 (pengukuran), dan 3 (kalibrasi).

```c
#include <stdio.h>

int main(void)
{
    int mode;
```

```
    printf("Mode (1-3): ");
    scanf("%d", &mode);

    switch (mode)
    {
        case 1:
            printf("STANDBY mode\n");
            break;
        case 2:
            printf("MEASUREMENT mode\n");
            break;
        case 3:
            printf("CALIBRATION mode\n");
            break;
        default:
            printf("Invalid mode\n");
            break;
    }

    return 0;
}
```

```
Mode (1-3): 2
MEASUREMENT mode
```

Kata kunci `break` mengakhiri `switch`. Tanpa `break`, eksekusi berlanjut ke kasus berikutnya; perilaku ini disebut *fall-through*. Jika `break` pada `case 1` dihilangkan, masukan `1` akan mencetak `STANDBY mode` *dan* `MEASUREMENT mode`. *Fall-through* kadang disengaja, tetapi bagi pemula sebaiknya `break` selalu ditulis secara eksplisit. Cabang `default` menangani semua nilai yang tidak tercantum.

Sebagai pedoman, gunakan `if` untuk kondisi berupa rentang, perbandingan, atau kombinasi logika (misalnya `pressure >= p_min && pressure <= p_max`), dan gunakan `switch` jika keputusan bergantung pada satu nilai diskret seperti nomor mode atau pilihan menu.

C juga mempunyai **operator kondisi** (*ternary operator*) `kondisi ? nilai_jika_benar : nilai_jika_salah`. Misalnya, `int max = (a > b) ? a : b;` memilih nilai terbesar dari `a` dan `b`. Operator ini berguna untuk ekspresi pendek; untuk logika yang lebih rumit, `if` lebih mudah dibaca.

4.5 Pengulangan `while` dan `do-while`

Setiap pengulangan memerlukan tiga hal: **keadaan awal**, **kondisi untuk melanjutkan**, dan **perubahan keadaan**. Untuk mencetak bilangan 1 sampai 5, *pseudocode*-nya adalah

$i \leftarrow 1$
WHILE $i \leq 5$
    OUTPUT i
    $i \leftarrow i + 1$

END WHILE

Ketika $i = 6$, kondisi $i \leq 5$ menjadi salah dan pengulangan berhenti. Dalam C, bentuk ini ditulis dengan `while`:

```c
#include <stdio.h>

int main(void)
{
    int i = 1;

    while (i <= 5)
    {
        printf("%d\n", i);
        i = i + 1;
    }

    return 0;
}
```

```
1
2
3
4
5
```

Kondisi `while` diperiksa *sebelum* blok dijalankan, sehingga blok mungkin tidak dijalankan sama sekali. Penelusuran berikut memperlihatkan perubahan keadaan program.

Iterasi	i sebelum	$i \leq 5$	Keluaran	i sesudah
1	1	benar	1	2
2	2	benar	2	3
3	3	benar	3	4
4	4	benar	4	5
5	5	benar	5	6
akhir	6	salah	–	–

Pengulangan memungkinkan kita mengevaluasi model fisika pada banyak titik. Program berikut menghitung posisi $x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$ untuk $t = 0, 1, \ldots, 5$ s dengan $x_0 = 0$, $v_0 = 5$ m/s, dan $a = 2$ m/s^2.

```c
#include <stdio.h>

int main(void)
{
    const double x0 = 0.0;
    const double v0 = 5.0;
    const double acceleration = 2.0;
```

```
    int t = 0;

    while (t <= 5)
    {
        double position = x0 + v0 * t + 0.5 * acceleration * t * t;
        printf("t = %d s, x = %6.2f m\n", t, position);
        t = t + 1;
    }

    return 0;
}
```

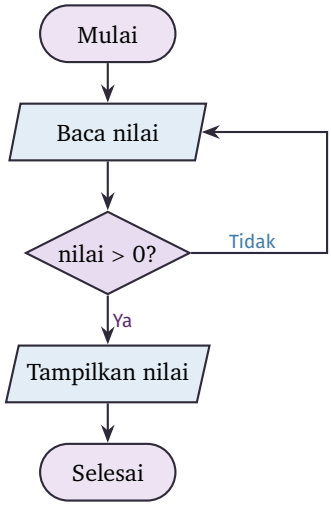
```
t = 0 s, x =   0.00 m
t = 1 s, x =   6.00 m
t = 2 s, x =  14.00 m
t = 3 s, x =  24.00 m
t = 4 s, x =  36.00 m
t = 5 s, x =  50.00 m
```

4.5.1 do-while

Pada do-while, blok dijalankan terlebih dahulu dan kondisi diperiksa *setelahnya*, sehingga blok dijalankan *setidaknya satu kali*:

```
do
{
    /* statements */
}
while (condition);
```

Bentuk ini cocok untuk tindakan yang memang harus dilakukan minimal sekali, misalnya menampilkan menu atau meminta masukan. Program berikut terus meminta bilangan bulat positif sampai pengguna memasukkan nilai yang valid.




```c
#include <stdio.h>

int main(void)
{
    int value;

    do
    {
        printf("Enter a positive integer: ");
        scanf("%d", &value);
    }
    while (value <= 0);

    printf("Accepted value = %d\n", value);

    return 0;
}
```

```
Enter a positive integer: -3
Enter a positive integer: 0
Enter a positive integer: 7
Accepted value = 7
```

Pola validasi masukan semacam ini akan dibahas lebih mendalam pada Kuliah 5.

4.6 Pengulangan `for`

Struktur `for` mengumpulkan ketiga komponen pengulangan—inisialisasi, kondisi, dan pembaruan—dalam satu baris:

```c
for (int i = 1; i <= 5; i++)
{
```

```
    printf("%d\n", i);
}
```

Bagian `int i = 1` adalah inisialisasi, `i <= 5` adalah kondisi, dan `i++` adalah pembaruan. Operator `i++` setara dengan `i = i + 1`. Potongan tersebut setara dengan pengulangan `while` pada subbab sebelumnya dan menghasilkan keluaran yang sama.

`for` sangat nyaman untuk membuat tabel komputasi. Program berikut menghitung posisi benda untuk $t = 0, 1, \ldots, 10$ s dan mencetaknya sebagai tabel yang nantinya dapat disimpan atau digambar.

```c
#include <stdio.h>

int main(void)
{
    const double x0 = 2.0;
    const double v0 = 5.0;
    const double acceleration = 3.0;

    printf("t(s)   x(m)\n");

    for (int t = 0; t <= 10; t++)
    {
        double position = x0 + v0 * t + 0.5 * acceleration * t * t;
        printf("%4d %7.2f\n", t, position);
    }

    return 0;
}
```

```
t(s)   x(m)
   0    2.00
   1    8.50
   2   18.00
   3   30.50
   4   46.00
   5   64.50
   6   86.00
   7  110.50
   8  138.00
   9  168.50
  10  202.00
```

Format `%4d` dan `%7.2f` menetapkan lebar kolom sehingga tabel tersusun rapi.

4.6.1 Memilih `while` atau `for`

Secara teori, `while` dan `for` dapat saling menggantikan. Sebagai pedoman, gunakan `for` jika jumlah iterasi atau pola pencacahnya sudah jelas, misalnya `for (int i = 0; i < 100; i++)`, dan gunakan `while` jika pengulangan lebih alami dinyatakan dengan suatu kondisi,

misalnya `while (error > tolerance)`. Pilihan yang tepat membuat maksud program lebih jelas.

4.6.2 Pengulangan bersarang

Pengulangan dapat ditempatkan di dalam pengulangan lain. Pada potongan berikut, pengulangan luar dilakukan tiga kali, dan untuk setiap iterasi luar, pengulangan dalam dilakukan empat kali.

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 3; i++)
    {
        for (int j = 1; j <= 4; j++)
        {
            printf("(%d,%d) ", i, j);
        }
        printf("\n");
    }

    return 0;
}
```

```
(1,1) (1,2) (1,3) (1,4)
(2,1) (2,2) (2,3) (2,4)
(3,1) (3,2) (3,3) (3,4)
```

Totalnya $3 \times 4 = 12$ kombinasi. Jika kedua batas sama-sama N, jumlah iterasinya kira-kira N^2; inilah sebabnya pengulangan bersarang sering berkaitan dengan kompleksitas $O(N^2)$ yang dibahas pada Kuliah 2.

4.7 Mengendalikan pengulangan dan kesalahan umum

4.7.1 `break` dan `continue`

Kata kunci `break` menghentikan pengulangan sebelum kondisi normalnya selesai, sedangkan `continue` melewati sisa blok pada iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya. Program berikut membaca paling banyak 100 nilai, tetapi berhenti segera setelah menemukan nilai negatif.

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++)
    {
        double x;
```

```
        printf("x%d: ", i);
        scanf("%lf", &x);

        if (x < 0.0)
        {
            printf("Negative value found at position %d.\n", i);
            break;
        }
    }

    return 0;
}
```

```
x1: 3.2
x2: 1.5
x3: -0.4
Negative value found at position 3.
```

Contoh `continue`: pengulangan berikut hanya mencetak bilangan genap karena iterasi untuk bilangan ganjil dilewati.

```
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 10; i++)
    {
        if (i % 2 != 0)
        {
            continue;
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

```
2 4 6 8 10
```

`break` dan `continue` berguna, tetapi penggunaan berlebihan membuat aliran program sulit ditelusuri. Gunakan secukupnya.

4.7.2 Kesalahan umum pada percabangan dan pengulangan

- `=` **sebagai pengganti** `==`**.** `if (mode = 1)` melakukan *assignment* dan kondisinya selalu benar. Yang dimaksud adalah `if (mode == 1)`. Opsi `-Wall` membantu mendeteksinya.
- **Kondisi rentang yang salah.** `if (20.0 <= temperature <= 30.0)` harus ditulis `if (temperature >= 20.0 && temperature <= 30.0)`.

- **Pengulangan tak hingga.** Jika variabel yang memengaruhi kondisi tidak pernah diperbarui, misalnya `while (i <= 5) { printf("%d\n", i); }`, kondisi selalu benar dan program tidak pernah berhenti. Hal yang sama terjadi jika pembaruannya ke arah yang salah, misalnya `i-` pada pengulangan `while (i <= 10)`. Untuk setiap pengulangan berbasis kondisi, tanyakan: *apa yang berubah pada setiap iterasi, dan bagaimana perubahan itu akhirnya membuat kondisi berhenti tercapai?*
- ***Off-by-one error*.** Untuk N data dengan indeks $0, 1, \ldots, N-1$, pengulangan yang benar adalah `for (int i = 0; i < n; i++)`. Versi `i <= n` berjalan $N+1$ kali.
- **Mengubah variabel pencacah di dalam `for`.** Menulis `i = i + 2;` di dalam `for (...; i++)` membuat `i` diperbarui dua kali per iterasi dan pola iterasinya sulit dipahami.
- **Kondisi berhenti pada *floating-point*.** Kondisi `while (x != 1.0)` bisa tidak pernah salah jika `x` diperbarui dengan operasi yang tidak pernah menghasilkan 1.0 secara eksak. Gunakan toleransi, misalnya `while (fabs(x - 1.0) > tolerance)`.

Beberapa kebiasaan baik untuk aliran kontrol: selalu gunakan kurung kurawal meskipun blok hanya berisi satu baris, gunakan indentasi yang konsisten, hindari percabangan yang terlalu dalam, pastikan setiap pengulangan mempunyai mekanisme berhenti, lakukan penelusuran untuk kasus sederhana, dan **uji nilai batas**. Jika klasifikasi berubah pada $T = 20$ dan $T = 30$, ujilah tepat pada kedua nilai tersebut.

4.8 Percabangan di dalam pengulangan

Program teknik hampir selalu menggabungkan kedua struktur. Program berikut membaca lima temperatur dan mengklasifikasikan setiap data; keputusan diambil untuk setiap iterasi.

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 5; i++)
    {
        double temperature;

        printf("T%d (C): ", i);
        scanf("%lf", &temperature);

        if (temperature < 20.0)
        {
            printf("    -> DINGIN\n");
        }
        else if (temperature <= 30.0)
        {
            printf("    -> NORMAL\n");
        }
        else
        {
            printf("    -> PANAS\n");
```

```
        }
    }

    return 0;
}
```

Perhatikan bahwa data uji sengaja memuat kedua nilai batas, 20 dan 30.

```
T1 (C): 18.5
    -> DINGIN
T2 (C): 20
    -> NORMAL
T3 (C): 25.3
    -> NORMAL
T4 (C): 30
    -> NORMAL
T5 (C): 31.2
    -> PANAS
```

Contoh kedua menghitung rata-rata $\bar{x} = \frac{1}{N}\sum x_i$ dari N data. Program menolak $N \leq 0$ sebelum pengulangan dimulai, sehingga pembagian dengan nol tidak mungkin terjadi.

```c
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of data: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of data must be positive.\n");
        return 1;
    }

    double sum = 0.0;

    for (int i = 1; i <= n; i++)
    {
        double x;
        printf("x%d: ", i);
        scanf("%lf", &x);
        sum = sum + x;
    }

    printf("Mean = %.3f\n", sum / n);

    return 0;
```

```
}
```

```
Number of data: 3
x1: 2
x2: 4
x3: 6
Mean = 4.000
```

Penelusuran untuk data 2, 4, 6 memperlihatkan perubahan `sum`: $0 \to 2 \to 6 \to 12$, sehingga $\bar{x} = 12/3 = 4$. Variabel `sum` yang terus diperbarui ini disebut **akumulator**. Pola akumulator, pencacah, minimum–maksimum, *sentinel*, dan validasi masukan adalah pola-pola iteratif yang akan dibahas secara sistematis pada Kuliah 5. Program pemantauan temperatur pada Kuliah 2 adalah contoh lengkap yang menggabungkan semua pola tersebut dengan percabangan di dalam pengulangan.

4.9 Cek pemahaman

- Apa yang dimaksud dengan aliran kontrol, dan apa tiga pola dasarnya?
- Apa perbedaan `if`, `if-else`, dan `if-else if-else`? Mengapa urutan kondisi dalam rantai `else if` penting?
- Kapan `switch` lebih sesuai daripada `if`? Apa fungsi `break` di dalamnya, dan apa yang dimaksud dengan *fall-through*?
- Apa perbedaan `while` dan `do-while`?
- Apa tiga komponen utama pada `for`?
- Kapan `for` lebih nyaman daripada `while`?
- Bagaimana sebuah pengulangan dapat menjadi tak hingga?
- Apa yang dimaksud dengan *off-by-one error*?
- Apa perbedaan `break` dan `continue` di dalam pengulangan?
- Mengapa nilai batas harus diuji secara eksplisit?

4.10 Latihan

1. **Klasifikasi temperatur.** Buat program yang menampilkan `DINGIN`, `NORMAL`, atau `PANAS` berdasarkan batas pada kuliah ini. Uji tepat pada $T = 20$ dan $T = 30$, serta pada $T = 19.99$ dan $T = 30.01$.
2. **Status tekanan.** Rentang tekanan aman suatu sistem adalah 95–105 kPa. Buat program yang menampilkan `LOW`, `NORMAL`, atau `HIGH`.
3. **Bilangan Reynolds.** Ubah program bilangan Reynolds agar menolak masukan viskositas yang tidak positif. Hitung Re untuk udara ($\rho = 1.2$ kg/m^3, $\mu = 1.8 \times 10^{-5}$ Pa s) yang mengalir 2 m/s dalam saluran berukuran 5 cm, dan tentukan rezim alirannya.

4. **Menu alat.** Gunakan `switch` untuk menu `1. Standby`, `2. Measurement`, `3. Calibration`, `4. Shutdown`. Untuk masukan lain tampilkan `Invalid mode.` Bungkus menu dalam `do-while` agar terus ditampilkan sampai pengguna memilih `4`.
5. **Bilangan genap.** Gunakan `while` untuk mencetak bilangan 1 sampai 20, lalu modifikasi agar hanya mencetak bilangan genap tanpa menggunakan `continue`.
6. **Tabel posisi.** Gunakan `for` untuk mencetak tabel t dan $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$ untuk $t = 0, 1, \ldots, 10$ s dengan $x_0 = 0$, $v_0 = 2$ m/s, dan $a = 1$ m/s^2. Tambahkan kolom kecepatan $v(t) = v_0 + at$.
7. **Validasi efisiensi.** Gunakan `do-while` untuk membaca efisiensi η sampai memenuhi $0 \le \eta \le 1$. Tampilkan pesan `Invalid value.` setiap kali masukan ditolak.
8. **Tabel perkalian.** Cetak tabel perkalian 1 sampai 10 dengan dua `for` bersarang dan format kolom yang rapi. Berapa banyak perkalian yang dilakukan? Bagaimana jumlah ini bertumbuh jika tabel diperbesar menjadi $N \times N$?
9. **Mencari kesalahan.** Program `int i = 1; while (i <= 10) { printf("%d\n", i); }` tidak pernah berhenti. Jelaskan penyebabnya dan perbaiki. Kemudian jelaskan mengapa `for (int i = 0; i <= 10; i++)` mencetak 11 nilai, dan perbaiki agar mencetak tepat 10 nilai mulai dari nol.
10. **Klasifikasi berulang dengan ringkasan.** Perluas program klasifikasi lima temperatur sehingga di akhir menampilkan banyaknya data pada setiap kategori. Uji dengan data yang memuat kedua nilai batas.

4.11 Rangkuman

- Aliran kontrol menentukan urutan eksekusi pernyataan. Tiga pola dasarnya adalah urutan, percabangan, dan pengulangan.
- `if` menjalankan blok jika kondisi benar; `if-else` memilih satu dari dua jalur; `if-else if-else` digunakan untuk klasifikasi banyak kategori, dan urutan kondisinya menentukan hasil.
- `switch` cocok untuk pilihan berdasarkan satu nilai diskret; `break` mencegah *fall-through* yang tidak diinginkan.
- `while` memeriksa kondisi sebelum menjalankan blok, `do-while` menjalankan blok setidaknya sekali, dan `for` cocok untuk pengulangan dengan inisialisasi, kondisi, dan pembaruan yang jelas.
- Setiap pengulangan memerlukan keadaan awal, kondisi untuk melanjutkan, dan perubahan keadaan yang akhirnya membuat pengulangan berhenti.
- `break` menghentikan pengulangan lebih awal, sedangkan `continue` melompat ke iterasi berikutnya.
- Kesalahan umum meliputi `=` sebagai pengganti `==`, kondisi rentang yang salah, pengulangan tak hingga, *off-by-one error*, dan kesamaan eksak pada *floating-point*.
- Pengulangan bersarang dengan batas N menghasilkan sekitar N^2 iterasi.

- Kuliah berikutnya membahas pola-pola algoritma iteratif yang dibangun dari percabangan dan pengulangan: pencacah, akumulator, minimum–maksimum, *sentinel*, validasi masukan, tabel komputasi, dan kriteria penghentian.

KULIAH

5

Pola Algoritma Iteratif

Pada Kuliah 4 kita mempelajari sintaks percabangan dan pengulangan. Kuliah ini memusatkan perhatian pada cara menggunakannya: mengenali **pola algoritma iteratif** yang muncul berulang kali dalam komputasi sains dan rekayasa. Banyak program yang tampak berbeda—menghitung energi total, menghitung banyaknya data sensor yang melewati ambang, mencari temperatur maksimum, atau mengulang perhitungan sampai galat cukup kecil—sebenarnya dibangun dari beberapa pola yang sama: pencacah, akumulator, minimum–maksimum, *sentinel*, validasi masukan, tabel komputasi, serta pembaruan iteratif dengan kriteria penghentian. Dengan mengenali pola, kita dapat merancang pengulangan secara sistematis, bukan dengan coba-coba.

5.1 Keadaan dan komponen iterasi

Setiap algoritma iteratif mempunyai **keadaan** (*state*) yang berubah pada setiap iterasi. Keadaan direpresentasikan oleh sekumpulan variabel yang menyimpan informasi penting tentang proses yang sudah berlangsung. Dalam penjumlahan data, misalnya, `sum` menyimpan jumlah sementara dan `i` menyimpan banyaknya data yang sudah diproses. Setelah data ke-i diproses, `sum` harus mempunyai makna yang jelas,

$$sum = \sum_{k=1}^{i} x_k,$$

yaitu **invarian** yang diperkenalkan pada Kuliah 2. Hampir semua algoritma iteratif mempunyai tiga komponen:

```
inisialisasi keadaan
WHILE kondisi masih terpenuhi
    lakukan proses dan perbarui keadaan
END WHILE
```

Jika salah satu dari **keadaan awal**, **aturan pembaruan**, atau **kondisi penghentian** salah, algoritma memberikan hasil keliru atau bahkan tidak pernah berhenti. Karena itu,

merancang iterasi tidak cukup hanya dengan mengetahui sintaks `while` atau `for`; kita harus menetapkan makna setiap variabel keadaan.

5.2 Pencacah dan akumulator

5.2.1 Pencacah

Pencacah (*counter*) adalah variabel yang menghitung banyaknya kejadian tertentu. Nilainya dimulai dari nol, karena sebelum data diproses belum ada kejadian yang tercatat, dan bertambah satu setiap kali kondisinya terpenuhi. Program berikut menghitung banyaknya pengukuran dengan $T > 30\,^{\circ}\mathrm{C}$ serta persentasenya.

```c
#include <stdio.h>

int main(void)
{
    int n;

    printf("Number of measurements: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of measurements must be positive.\n");
        return 1;
    }

    int count_high = 0;

    for (int i = 1; i <= n; i++)
    {
        double temperature;
        printf("T%d (C): ", i);
        scanf("%lf", &temperature);

        if (temperature > 30.0)
        {
            count_high++;
        }
    }

    double percentage = 100.0 * count_high / n;

    printf("Above 30 C: %d of %d (%.1f %%)\n", count_high, n, percentage);

    return 0;
}
```

```
Number of measurements: 4
T1 (C): 29.1
```

```
T2 (C): 31.5
T3 (C): 30.0
T4 (C): 32.2
Above 30 C: 2 of 4 (50.0 %)
```

Program tidak perlu menyimpan seluruh data temperatur; ia hanya mempertahankan informasi yang diperlukan, yaitu banyaknya data yang memenuhi kondisi. Persentase $p = (N_h/N) \times 100\%$ ditulis `100.0 * count_high / n`: angka `100.0` sengaja ditulis sebagai bilangan *floating-point* agar tidak terjadi pembagian bilangan bulat. Pola algoritma selalu harus dipadukan dengan pemahaman tipe data.

5.2.2 Akumulator

Akumulator adalah variabel yang mengumpulkan nilai secara bertahap. Pola ini muncul ketika kita menghitung jumlah, energi total, massa total, atau besaran kumulatif lainnya. Untuk $S = \sum_{i=1}^{N} x_i$, nilai awal akumulator adalah nol karena nol merupakan elemen identitas penjumlahan; untuk perkalian kumulatif, nilai awalnya satu. Dalam C, pembaruan `sum = sum + x;` biasanya ditulis ringkas sebagai `sum += x;`.

Rata-rata menggabungkan akumulator dengan informasi banyaknya data. Jika N diketahui sejak awal (seperti contoh rata-rata pada Kuliah 4), pengulangan `for` sudah cukup. Jika banyaknya data tidak diketahui sebelumnya, kita memerlukan pencacah tambahan, sehingga setiap data valid memperbarui dua keadaan sekaligus:

```
sum += x;
count++;
```

Setelah proses selesai, rata-rata hanya boleh dihitung jika $count > 0$. Kasus tanpa data valid adalah kasus batas yang harus dirancang sejak awal, bukan ditambahkan setelah program gagal.

5.3 Minimum, maksimum, dan penggabungan pola

Pola minimum dan maksimum mencari nilai ekstrem $x_{\min} = \min(x_1, \ldots, x_N)$ dan $x_{\max} = \max(x_1, \ldots, x_N)$. Seperti dibahas pada Kuliah 2, strategi yang aman adalah menggunakan data pertama sebagai nilai awal. Inisialisasi dengan nol salah jika semua data negatif (maksimumnya tetap nol) atau semua data positif (minimumnya tetap nol), padahal nol bahkan tidak ada di dalam data.

Satu pengulangan dapat memperbarui beberapa keadaan sekaligus. Program berikut menghitung jumlah, rata-rata, minimum, maksimum, dan banyaknya data di atas ambang dengan membaca setiap data tepat satu kali.

```
#include <stdio.h>

int main(void)
{
    int n;
    double limit;
```

```
    printf("Number of measurements: ");
    scanf("%d", &n);
    printf("Limit: ");
    scanf("%lf", &limit);

    if (n <= 0)
    {
        printf("N must be positive.\n");
        return 1;
    }

    double x;
    printf("x1: ");
    scanf("%lf", &x);

    double sum = x;
    double minimum = x;
    double maximum = x;
    int count_high = (x > limit) ? 1 : 0;

    for (int i = 2; i <= n; i++)
    {
        printf("x%d: ", i);
        scanf("%lf", &x);

        sum += x;
        if (x < minimum)
        {
            minimum = x;
        }
        if (x > maximum)
        {
            maximum = x;
        }
        if (x > limit)
        {
            count_high++;
        }
    }

    printf("\nMean        = %.3f\n", sum / n);
    printf("Minimum     = %.3f\n", minimum);
    printf("Maximum     = %.3f\n", maximum);
    printf("Above limit = %d\n", count_high);

    return 0;
}
```

Dengan data negatif, inisialisasi dari data pertama terbukti penting:

```
Number of measurements: 4
Limit: -1.0
x1: -4.2
x2: -2.7
x3: -0.5
x4: -3.1

Mean        = -2.625
Minimum     = -4.200
Maximum     = -0.500
Above limit = 1
```

Perhatikan bahwa `maximum` bernilai -0.5, bukan 0. Program ini adalah versi umum dari studi kasus pemantauan temperatur pada Kuliah 2.

5.4 *Sentinel* dan validasi masukan

5.4.1 Pengulangan dengan *sentinel*

Pada banyak persoalan, banyaknya data tidak diketahui sebelumnya. Proses dapat dihentikan ketika pengguna memasukkan nilai khusus yang disebut ***sentinel***. Misalkan temperatur dibaca terus sampai pengguna memasukkan -999; nilai ini adalah tanda berhenti, bukan data.

```
sum ← 0;   count ← 0
INPUT T
WHILE T ≠ −999
    sum ← sum + T;   count ← count + 1
    INPUT T
END WHILE
IF count > 0
    OUTPUT sum/count
END IF
```

```c
#include <stdio.h>

int main(void)
{
    const double sentinel = -999.0;

    double sum = 0.0;
    int count = 0;
    double temperature;

    printf("Temperature (-999 to stop): ");
    scanf("%lf", &temperature);

    while (temperature != sentinel)
    {
        sum += temperature;
```

```
        count++;

        printf("Temperature (-999 to stop): ");
        scanf("%lf", &temperature);
    }

    if (count > 0)
    {
        printf("Count = %d\n", count);
        printf("Mean  = %.3f C\n", sum / count);
    }
    else
    {
        printf("No valid data.\n");
    }

    return 0;
}
```

```
Temperature (-999 to stop): 28.4
Temperature (-999 to stop): 29.9
Temperature (-999 to stop): 31.2
Temperature (-999 to stop): -999
Count = 3
Mean  = 29.833 C
```

Program ini menggunakan pola **pembacaan awal** (*priming read*): data pertama dibaca sebelum masuk ke `while`, dan pembacaan berikutnya dilakukan di akhir setiap iterasi. Dengan cara ini, *sentinel* selalu diperiksa sebelum diproses dan tidak pernah ikut masuk ke `sum` atau `count`. Jika pengguna langsung memasukkan −999, `count` tetap nol dan program tidak melakukan pembagian dengan nol.

Membandingkan `temperature != sentinel` dengan kesamaan eksak di sini aman, karena −999 diketik pengguna dan direpresentasikan secara eksak. *Sentinel* harus dipilih jauh dari data valid. Untuk temperatur Celsius di laboratorium, −999 cukup aman; dalam sistem nyata, format masukan yang lebih eksplisit sering lebih baik daripada bergantung pada satu angka khusus.

5.4.2 Validasi masukan

Validasi memastikan data memenuhi syarat sebelum digunakan. Syarat tersebut dapat berupa batas fisis (massa $m > 0$, tekanan absolut $P \geq 0$, efisiensi $0 \leq \eta \leq 1$) atau batas operasional alat ukur. Pada Kuliah 4 kita melihat pola `do-while` yang terus meminta masukan sampai valid. Polanya dalam *pseudocode*:

```
DO
    INPUT η
    IF η < 0 OR η > 1
        OUTPUT "nilai tidak valid"
```

```
    END IF
  WHILE η < 0 OR η > 1
```

Pola ini memisahkan dua gagasan: memperoleh masukan dan memastikan masukan memenuhi syarat. Selain rentang nilai, kita juga perlu memeriksa apakah `scanf` berhasil membaca angka, yaitu dengan memeriksa `scanf(...) != 1` seperti pada Kuliah 3.

5.4.3 Menggabungkan *sentinel* dan validasi

Kadang-kadang kita harus membedakan tiga jenis masukan: data valid, data tidak valid, dan *sentinel*. Urutan pemeriksaannya harus jelas: periksa *sentinel* terlebih dahulu, baru kemudian validitas data. Misalkan rentang temperatur yang valid secara operasional adalah $-100 \le T \le 200\,^{\circ}\mathrm{C}$.

```c
#include <stdio.h>

int main(void)
{
    const double sentinel = -999.0;

    double sum = 0.0;
    int count = 0;
    int rejected = 0;

    while (1)
    {
        double temperature;

        printf("Temperature (-999 to stop): ");
        scanf("%lf", &temperature);

        if (temperature == sentinel)
        {
            break;
        }

        if (temperature < -100.0 || temperature > 200.0)
        {
            printf("  invalid measurement, ignored\n");
            rejected++;
            continue;
        }

        sum += temperature;
        count++;
    }

    printf("Valid = %d, rejected = %d\n", count, rejected);
    if (count > 0)
    {
```

```
        printf("Mean  = %.3f C\n", sum / count);
    }

    return 0;
}
```

```
Temperature (-999 to stop): 25.1
Temperature (-999 to stop): 450
  invalid measurement, ignored
Temperature (-999 to stop): 26.3
Temperature (-999 to stop): -999
Valid = 2, rejected = 1
Mean  = 25.700 C
```

Pengulangan `while (1)` berjalan terus sampai dihentikan oleh `break`. Program ini menggabungkan `break`, `continue`, *sentinel*, validasi, pencacah, dan akumulator; walaupun strukturnya lebih kompleks, setiap bagian mempunyai peran yang jelas.

5.5 Tabel komputasi

Pengulangan sangat cocok untuk membuat tabel nilai suatu model fisika, baik untuk memeriksa perilaku model, membuat grafik, maupun sebagai data awal analisis lanjutan. Pada Kuliah 4 kita membuat tabel posisi dengan langkah waktu bilangan bulat. Dalam fisika, langkah sering berupa bilangan real, misalnya $\Delta t = 0.1$ s.

Pendekatan yang aman adalah menggunakan **pencacah bilangan bulat**, lalu menghitung nilai real darinya: `t = i * dt`. Alternatif `for (double t = 0.0; t <= 2.0; t += 0.1)` terlihat wajar, tetapi `t` mengalami pembulatan berulang sehingga titik terakhir dapat hilang. Program berikut membandingkan kedua cara.

```
#include <stdio.h>

int main(void)
{
    int count_real = 0;
    for (double t = 0.0; t <= 2.0; t += 0.1)
    {
        count_real++;
    }

    int count_int = 0;
    for (int i = 0; i <= 20; i++)
    {
        double t = i * 0.1;
        (void)t;
        count_int++;
    }

    printf("real step   : %d points\n", count_real);
    printf("integer step: %d points\n", count_int);
```

```
    return 0;
}
```

```
real step   : 20 points
integer step: 21 points
```

Kita mengharapkan 21 titik ($t = 0, 0.1, \ldots, 2.0$). Karena 0.1 tidak eksak dalam biner, kesalahan pembulatan terakumulasi; setelah dua puluh penambahan, nilai `t` sedikit lebih besar dari 2.0, sehingga titik terakhir terlewat. Pencacah bilangan bulat tidak mengalami masalah ini.

5.5.1 Contoh: hukum Hooke

Untuk pegas ideal, $F = -kx$. Kita mengevaluasi gaya untuk $x = -0.10, -0.08, \ldots, 0.10$ m dengan $k = 100$ N/m menggunakan pencacah bilangan bulat dari -5 sampai 5.

```
#include <stdio.h>

int main(void)
{
    const double k = 100.0;    /* N/m */
    const double dx = 0.02;    /* m   */

    printf("  x(m)     F(N)\n");

    for (int i = -5; i <= 5; i++)
    {
        double x = i * dx;
        double force = -k * x;
        printf("% .2f  % 7.2f\n", x, force);
    }

    return 0;
}
```

```
  x(m)     F(N)
-0.10    10.00
-0.08     8.00
-0.06     6.00
-0.04     4.00
-0.02     2.00
 0.00    -0.00
 0.02    -2.00
 0.04    -4.00
 0.06    -6.00
 0.08    -8.00
 0.10   -10.00
```

Keluaran simetris terhadap $x = 0$ dan gaya selalu berlawanan tanda dengan perpindahan, sesuai sifat gaya pemulih. Pola fisis semacam ini adalah cara cepat memeriksa kewajaran tabel.

5.5.2 Pemetaan dua parameter

Pengulangan bersarang digunakan untuk mengevaluasi model pada dua parameter. Program berikut menghitung $P = VI$ untuk $V = 5, 10, 15$ V dan $I = 1, 2, 3, 4$ A, sehingga setiap pasangan parameter menghasilkan satu nilai daya.

```c
#include <stdio.h>

int main(void)
{
    printf("V\\I ");
    for (int current = 1; current <= 4; current++)
    {
        printf("%7d A", current);
    }
    printf("\n");

    for (int voltage = 5; voltage <= 15; voltage += 5)
    {
        printf("%2d V", voltage);
        for (int current = 1; current <= 4; current++)
        {
            double power = voltage * current;
            printf("%7.1f W", power);
        }
        printf("\n");
    }

    return 0;
}
```

```
V\I       1 A      2 A      3 A      4 A
 5 V    5.0 W   10.0 W   15.0 W   20.0 W
10 V   10.0 W   20.0 W   30.0 W   40.0 W
15 V   15.0 W   30.0 W   45.0 W   60.0 W
```

Banyaknya kombinasi sama dengan hasil kali banyaknya nilai kedua parameter, $3 \times 4 = 12$.

5.6 Pembaruan iteratif dan kriteria penghentian

Tidak semua iterasi membaca data. Banyak algoritma numerik menghasilkan nilai baru dari nilai sebelumnya,

$$x_{k+1} = f(x_k),$$

dimulai dari tebakan awal x_0 sehingga diperoleh barisan $x_1, x_2, x_3, \ldots$. Pola ini sangat penting dalam metode numerik, optimasi, simulasi, dan pemodelan dinamik. Proses dihentikan oleh **kriteria penghentian** (*stopping criterion*), misalnya

- perubahan antariterasi cukup kecil, $|x_{k+1} - x_k| < \varepsilon$;
- galat terhadap target cukup kecil;
- suatu kondisi fisis tercapai;
- jumlah iterasi mencapai batas maksimum.

Nilai ε disebut **toleransi** dan dipilih sesuai skala serta tujuan perhitungan. Kriteria berbasis toleransi saja tidak menjamin algoritma berhenti: jika metode tidak konvergen, program dapat berjalan terus. Karena itu, algoritma numerik yang baik selalu juga mempunyai **batas iterasi maksimum** sebagai pengaman.

5.6.1 Pencarian dengan langkah tetap

Untuk mencari x yang memenuhi $x^2 = 2$, cara paling sederhana adalah memulai dari $x = 0$ dan menaikkan x sebesar Δx selama $x^2 < 2$.

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    const double target = 2.0;
    const double dx = 0.001;

    double x = 0.0;
    int steps = 0;

    while (x * x < target)
    {
        x += dx;
        steps++;
    }

    printf("Approximation = %.6f\n", x);
    printf("Error         = %.2e\n", fabs(x - sqrt(target)));
    printf("Steps         = %d\n", steps);

    return 0;
}
```

```
Approximation = 1.415000
Error         = 7.86e-04
Steps         = 1415
```

Galatnya sebanding dengan ukuran langkah, sedangkan banyaknya langkah berbanding terbalik dengan Δx. Langkah yang lebih kecil meningkatkan ketelitian, tetapi membutuhk-

an lebih banyak iterasi; langkah terlalu besar dapat melewati solusi. Tidak ada ukuran langkah yang selalu terbaik.

5.6.2 Metode Babilonia

Metode yang jauh lebih efisien untuk menghitung $\sqrt{S}$ adalah iterasi Babilonia

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{S}{x_k}\right).$$

Untuk $S > 0$, kita mulai dari tebakan positif dan memperbarui nilai sampai perubahan antariterasi lebih kecil dari toleransi, dengan pengaman batas iterasi maksimum. Program berikut juga mencetak setiap iterasi agar konvergensinya terlihat.

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    const double s = 2.0;
    const double tolerance = 1e-12;
    const int max_iteration = 100;

    double x = 1.0;
    int iteration = 0;
    double difference;

    do
    {
        double x_new = 0.5 * (x + s / x);
        difference = fabs(x_new - x);
        x = x_new;
        iteration++;

        printf("%2d  %.15f  %.2e\n", iteration, x, difference);
    }
    while (difference >= tolerance && iteration < max_iteration);

    printf("sqrt(%.1f) ~= %.15f after %d iterations\n", s, x, iteration);

    return 0;
}
```

```
 1  1.500000000000000  5.00e-01
 2  1.416666666666667  8.33e-02
 3  1.414215686274510  2.45e-03
 4  1.414213562374690  2.12e-06
 5  1.414213562373095  1.59e-12
 6  1.414213562373095  0.00e+00
sqrt(2.0) ~= 1.414213562373095 after 6 iterations
```

Hanya dalam enam iterasi diperoleh 15 digit yang benar, dibandingkan lebih dari seribu langkah pada metode langkah tetap. Perhatikan bahwa banyaknya digit benar kira-kira berlipat dua pada setiap iterasi. Tampilan per iterasi seperti ini sangat berguna untuk mempelajari perilaku algoritma numerik dan menemukan kesalahan pembaruan.

5.6.3 Konvergensi deret

Pola akumulator juga digunakan untuk jumlah parsial deret, misalnya

$$S_N = \sum_{n=1}^{N} \frac{1}{n^2} \longrightarrow \frac{\pi^2}{6} \quad (N \to \infty).$$

Alih-alih menetapkan N di awal, kita dapat berhenti ketika suku berikutnya lebih kecil dari toleransi. Program berikut melakukan keduanya: mencetak galat untuk beberapa N, lalu menjumlahkan sampai $1/n^2 < 10^{-6}$.

```c
#include <stdio.h>

int main(void)
{
    const double pi = 3.141592653589793;
    const double reference = pi * pi / 6.0;

    double sum = 0.0;
    int checkpoint = 10;

    printf("%8s %16s %12s\n", "N", "S_N", "error");
    for (int n = 1; n <= 10000; n++)
    {
        double nd = (double)n;
        sum += 1.0 / (nd * nd);
        if (n == checkpoint)
        {
            printf("%8d %16.12f %12.3e\n", n, sum, reference - sum);
            checkpoint *= 10;
        }
    }

    const double tolerance = 1e-6;
    int n = 1;
    double term = 1.0;
    sum = 0.0;
    while (term >= tolerance)
    {
        sum += term;
        n++;
        term = 1.0 / ((double)n * n);
    }
    printf("\nStopped at n = %d, S = %.12f, error = %.3e\n",
           n, sum, reference - sum);
```

```
    return 0;
}
```

```
       N              S_N        error
      10   1.549767731167    9.517e-02
     100   1.634983900185    9.950e-03
    1000   1.643934566682    9.995e-04
   10000   1.644834071848    1.000e-04

Stopped at n = 1001, S = 1.643934566682, error = 9.995e-04
```

Galat berkurang kira-kira sebanding dengan $1/N$: setiap kali N dikalikan sepuluh, galat menjadi sepersepuluhnya. Perhatikan juga bahwa penghentian berbasis ukuran suku berhenti ketika suku sudah di bawah 10^{-6}, tetapi galat total masih sekitar 10^{-3}. Ukuran suku yang kecil *tidak* selalu berarti galat total kecil; interpretasi kriteria penghentian harus disesuaikan dengan teori deret yang bersangkutan.

5.7 Studi kasus: analisis getaran mesin

Kita merancang program yang membaca data percepatan getaran dari akselerometer yang dipasang pada mesin. Sinyal getaran berosilasi sehingga bernilai positif maupun negatif; rata-rata saja tidak cukup untuk menggambarkan besar getaran. Ukuran yang lebih tepat adalah nilai **RMS** (*root mean square*),

$$a_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} a_i^2},$$

yang selalu tidak negatif dan sensitif terhadap amplitudo. Pasangan nilai +10 dan −10 mempunyai rata-rata nol, tetapi RMS-nya $\sqrt{(10^2 + 10^2)/2} = 10$.

5.7.1 Spesifikasi dan algoritma

Program membaca data sampai *sentinel* 9999. Data valid berada pada rentang $-50 \le a \le 50$ m/s^2; data di luar rentang ditolak. Ambang amplitudo adalah $a_{\mathrm{limit}} = 8$ m/s^2, dan suatu data melampaui ambang jika $|a_i| > a_{\mathrm{limit}}$. Keluaran program adalah banyaknya data valid, rata-rata, RMS, minimum, maksimum, serta banyak dan persentase data di atas ambang.

Algoritma ini menggabungkan hampir semua pola pada kuliah ini. Dibutuhkan dua akumulator, *sum* dan *sum_square*; akumulator kedua memungkinkan RMS dihitung tanpa menyimpan seluruh data. Karena data valid pertama tidak selalu merupakan masukan pertama, minimum dan maksimum diinisialisasi pada saat *count* = 0.

```
sum ← 0;  sum_square ← 0;  count ← 0;  count_high ← 0
WHILE true
    INPUT a
    IF a = sentinel
        BREAK
```

```
        END IF
        IF a < −50 OR a > 50
            OUTPUT "tidak valid";   CONTINUE
        END IF
        IF count = 0
            minimum ← a;   maximum ← a
        ELSE
            perbarui minimum dan maximum
        END IF
        sum ← sum + a;   sum_square ← sum_square + a · a;   count ← count + 1
        IF |a| > a_limit
            count_high ← count_high + 1
        END IF
    END WHILE
```

5.7.2 Implementasi

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    const double sentinel = 9999.0;
    const double min_valid = -50.0;
    const double max_valid = 50.0;
    const double limit = 8.0;

    double sum = 0.0, sum_square = 0.0;
    double minimum = 0.0, maximum = 0.0;
    int count = 0, count_high = 0;

    while (1)
    {
        double a;
        printf("Acceleration (9999 to stop): ");
        scanf("%lf", &a);

        if (a == sentinel)
        {
            break;
        }
        if (a < min_valid || a > max_valid)
        {
            printf("  invalid measurement\n");
            continue;
        }

        if (count == 0)
        {
            minimum = a;
```

```
            maximum = a;
        }
        else
        {
            if (a < minimum) minimum = a;
            if (a > maximum) maximum = a;
        }

        sum += a;
        sum_square += a * a;
        count++;

        if (fabs(a) > limit)
        {
            count_high++;
        }
    }

    if (count == 0)
    {
        printf("No valid measurement.\n");
        return 0;
    }

    double mean = sum / count;
    double rms = sqrt(sum_square / count);
    double percentage_high = 100.0 * count_high / count;

    printf("\nValid data  = %d\n", count);
    printf("Mean        = %.3f m/s^2\n", mean);
    printf("RMS         = %.3f m/s^2\n", rms);
    printf("Minimum     = %.3f m/s^2\n", minimum);
    printf("Maximum     = %.3f m/s^2\n", maximum);
    printf("Above limit = %d (%.1f %%)\n", count_high, percentage_high);

    return 0;
}
```

```
Acceleration (9999 to stop): 1.2
Acceleration (9999 to stop): -2.4
Acceleration (9999 to stop): 8.7
Acceleration (9999 to stop): 75
  invalid measurement
Acceleration (9999 to stop): -9.1
Acceleration (9999 to stop): 3.6
Acceleration (9999 to stop): 9999

Valid data  = 5
Mean        = 0.400 m/s^2
RMS         = 5.978 m/s^2
```

```
Minimum     = -9.100 m/s^2
Maximum     = 8.700 m/s^2
Above limit = 2 (40.0 %)
```

5.7.3 Penelusuran

Penelusuran manual untuk data yang sama memperlihatkan perubahan keadaan utama.

Masukan	Valid?	count	sum	sum_square	minimum	maximum	count_high
1.2	ya	1	1.2	1.44	1.2	1.2	0
−2.4	ya	2	−1.2	7.20	−2.4	1.2	0
8.7	ya	3	7.5	82.89	−2.4	8.7	1
75	tidak	3	7.5	82.89	−2.4	8.7	1
−9.1	ya	4	−1.6	165.70	−9.1	8.7	2
3.6	ya	5	2.0	178.66	−9.1	8.7	2
9999	*sentinel*	5	2.0	178.66	−9.1	8.7	2

Dari tabel diperoleh $\bar{a} = 2.0/5 = 0.4$ m/s^2 dan $a_{\text{RMS}} = \sqrt{178.66/5} \approx 5.978$ m/s^2, sesuai dengan keluaran program. Walaupun rata-ratanya hanya 0.4 m/s^2, RMS-nya hampir 6 m/s^2 karena data berosilasi cukup besar di sekitar nol. Rata-rata menggambarkan kecenderungan nilai bertanda, sedangkan RMS menggambarkan amplitudo efektif.

5.8 Kesalahan umum dan efisiensi

- **Inisialisasi tidak sesuai.** Nilai awal harus konsisten dengan makna variabel dan invariannya; `maximum = 0` salah untuk data yang mungkin seluruhnya negatif.
- **Lupa memperbarui keadaan.** Jika variabel kondisi tidak pernah berubah, misalnya `while (i < 10)` tanpa `i++`, pengulangan tidak pernah berhenti.
- **Pembaruan pada posisi yang salah.** Memperbarui variabel terlalu awal dapat menghilangkan nilai lama sebelum digunakan. Penelusuran beberapa iterasi pertama membantu melihat apakah setiap variabel berubah pada saat yang tepat.
- ***Sentinel* ikut dihitung.** *Sentinel* harus diperiksa *sebelum* data diproses agar tidak masuk ke `sum`, `count`, minimum, atau maksimum.
- **Pembagian dengan nol.** Jika pengguna langsung memasukkan *sentinel*, `count` tetap nol. Periksa *count* > 0 sebelum menghitung rata-rata.
- **Kriteria berhenti tidak realistis.** `while (x != target)` tidak aman untuk `double`; gunakan toleransi $|x - \text{target}| < \varepsilon$ dan sediakan batas iterasi maksimum.
- **Ukuran langkah tidak tepat.** Langkah terlalu besar dapat melewati solusi, sedangkan langkah terlalu kecil membutuhkan sangat banyak iterasi.

Dari sisi efisiensi, pola pencacah, akumulator, minimum–maksimum, dan validasi membutuhkan pekerjaan konstan untuk setiap data, sehingga kompleksitas waktunya $O(N)$. Statistik sederhana tidak memerlukan penyimpanan seluruh data; jika data diproses satu per satu tanpa disimpan, kebutuhan memori tambahannya hanya $O(1)$. Pendekatan ini

disebut pemrosesan ***streaming***. Pengulangan bersarang dengan dua batas berukuran N menghasilkan $O(N^2)$. Analisis kompleksitas membantu menilai apakah suatu pola masih masuk akal ketika data membesar, tetapi kebenaran algoritma tetap menjadi syarat pertama.

Sebagai pedoman merancang algoritma iteratif: tentukan variabel keadaan yang benar-benar diperlukan dan tuliskan maknanya; tentukan nilai awal setiap variabel beserta alasannya; tuliskan kondisi penghentian secara eksplisit; telusuri dengan data kecil yang memuat kasus biasa, kasus batas, data negatif, data kosong, dan masukan tidak valid; lalu periksa hasilnya secara matematis dan fisis.

5.9 Cek pemahaman

- Apa yang dimaksud dengan keadaan dalam algoritma iteratif, dan apa tiga komponen utama iterasi?
- Apa fungsi pencacah dan akumulator? Mengapa akumulator penjumlahan dimulai dari nol?
- Mengapa minimum dan maksimum sebaiknya diinisialisasi menggunakan data valid pertama?
- Apa yang dimaksud dengan *sentinel* dan pembacaan awal (*priming read*)? Mengapa *sentinel* tidak boleh ikut diproses?
- Apa perbedaan validasi masukan dan *sentinel*?
- Mengapa pencacah bilangan bulat lebih aman untuk membuat langkah real tetap?
- Apa fungsi toleransi, dan mengapa batas iterasi maksimum tetap diperlukan?
- Mengapa metode Babilonia jauh lebih cepat daripada pencarian dengan langkah tetap?
- Mengapa RMS lebih informatif daripada rata-rata untuk data getaran?
- Apa yang dimaksud dengan pemrosesan *streaming*, dan berapa kebutuhan memori tambahannya?

5.10 Latihan

1. **Arus listrik secara *streaming*.** Sebuah sistem akuisisi membaca arus listrik satu per satu sampai *sentinel* -999; data valid berada pada rentang $-20 \le I \le 20$ A. Buat program yang dalam satu kali penelusuran menghitung banyak data valid, rata-rata, minimum, maksimum, RMS, dan banyaknya data dengan $|I| > 10$ A, tanpa menyimpan seluruh data. Telusuri dengan sekurang-kurangnya enam masukan yang memuat data valid, data tidak valid, dan *sentinel*.
2. **Getaran dengan dua ambang.** Klasifikasikan setiap data percepatan berdasarkan $|a_i|$ menjadi `NORMAL`, `WARNING` ($|a_i| > 5$ m/s^2), atau `DANGER` ($|a_i| > 10$ m/s^2). Dalam satu pengulangan, hitung banyaknya data pada setiap kategori, RMS keseluruhan, dan persentase kategori `DANGER`.

3. **Konvergensi deret.** Ubah program deret pada kuliah ini untuk $S_N = \sum_{n=1}^{N} (-1)^{n+1}/n$, yang konvergen ke $\ln 2$. Buat tabel N, S_N, dan galat absolut untuk $N = 10, 100, 1000, 10000$. Bandingkan laju konvergensinya dengan deret $1/n^2$.
4. **Ukuran langkah.** Cari pendekatan $\sqrt{7}$ dengan metode langkah tetap untuk $\Delta x = 0.1$, 0.01, 0.001, dan 0.0001. Catat hasil, galat absolut, dan banyaknya langkah. Jelaskan hubungan antara ukuran langkah, ketelitian, dan biaya komputasi.
5. **Metode Babilonia.** Buat program metode Babilonia yang membaca $S > 0$ dan toleransi dari pengguna, dengan batas iterasi maksimum. Tampilkan nomor iterasi, x_k, x_{k+1}, dan $|x_{k+1} - x_k|$. Uji untuk $S = 2$, $S = 10^6$, dan $S = 10^{-6}$, lalu bandingkan banyaknya iterasi.
6. **Pemetaan daya dengan batas.** Hitung $P = VI$ untuk $V = 5, 10, \ldots, 30$ V dan $I = 0.5, 1.0, \ldots, 5.0$ A dengan dua pengulangan bersarang (gunakan pencacah bilangan bulat). Cari kombinasi yang menghasilkan daya terbesar tanpa melebihi 100 W. Berapa banyak kombinasi yang diperiksa?
7. **Validasi berlapis.** Sebuah sensor tekanan seharusnya menghasilkan data $80 \le P \le 120$ kPa, dengan *sentinel* -1. Hitung rata-rata, minimum, maksimum, dan banyaknya data yang menyimpang lebih dari 10 kPa dari 100 kPa. Program harus tetap benar jika beberapa masukan pertama tidak valid; jelaskan bagaimana data valid pertama digunakan untuk inisialisasi.
8. **Dua kriteria penghentian.** Untuk iterasi $x_{k+1} = \cos x_k$ dengan $x_0 = 1$, hentikan proses jika $|x_{k+1} - x_k| < \varepsilon$ atau jika iterasi mencapai `max_iteration`. Jalankan dengan $\varepsilon = 10^{-10}$ dan `max_iteration` = 20, lalu = 200. Mana yang berhenti karena toleransi dan mana yang karena batas iterasi?
9. **Integral dengan jumlah Riemann.** Gunakan pola akumulator untuk mendekati $\int_0^1 x^2\,dx$ dengan jumlah Riemann titik tengah: bagi interval menjadi N bagian dengan $\Delta x = 1/N$ dan jumlahkan $f(x_i)\Delta x$ pada titik tengah setiap subinterval. Uji untuk $N = 10, 100, 1000, 10000$ dan bandingkan dengan nilai eksak $1/3$.
10. **Proyek mini: *crest factor*.** Kembangkan studi kasus getaran mesin sehingga juga menghitung *crest factor* $C = \max|a_i| / a_{\mathrm{RMS}}$. Program harus memperbarui `max_abs` selama proses tanpa menyimpan seluruh data. Lakukan penelusuran manual untuk satu himpunan data uji, bandingkan dengan keluaran program, dan jelaskan informasi yang berbeda dari rata-rata, RMS, dan *crest factor*.

5.11 Rangkuman

- Pola algoritma iteratif membantu mengenali struktur penyelesaian masalah yang berulang pada banyak kasus. Setiap iterasi mempunyai keadaan awal, aturan pembaruan, dan kondisi penghentian.
- Pencacah menghitung banyaknya kejadian; akumulator mengumpulkan nilai. Keduanya biasanya dimulai dari nol, dan rata-rata hanya boleh dihitung jika banyaknya data lebih dari nol.
- Minimum dan maksimum diinisialisasi menggunakan data valid pertama. Beberapa pola dapat diperbarui sekaligus dalam satu pengulangan.

- *Sentinel* menandai akhir masukan ketika banyaknya data tidak diketahui dan harus diperiksa sebelum data diproses. Validasi memastikan data memenuhi syarat matematis atau fisis.
- Tabel komputasi dibuat dengan pengulangan; pencacah bilangan bulat menghasilkan langkah real yang konsisten. Pengulangan bersarang memetakan model pada dua parameter.
- Pembaruan iteratif $x_{k+1} = f(x_k)$ dihentikan oleh kriteria berbasis toleransi yang dikombinasikan dengan batas iterasi maksimum. Metode yang baik, seperti metode Babilonia, dapat konvergen jauh lebih cepat daripada pencarian dengan langkah tetap.
- Banyak statistik sederhana dapat dihitung dalam satu kali penelusuran data dengan kompleksitas waktu $O(N)$ dan memori tambahan $O(1)$.
- Kuliah berikutnya membahas larik, fungsi, dan modularisasi program, yang memungkinkan kita menyimpan banyak data secara terstruktur dan memecah program besar menjadi bagian-bagian yang mudah diuji serta digunakan kembali.

KULIAH

6

Larik, Fungsi, dan Modularisasi Program

Variabel, percabangan, pengulangan, dan pola iteratif sudah cukup untuk memproses data satu per satu. Pendekatan tersebut mulai tidak praktis ketika kita perlu menyimpan banyak data atau menggunakan proses yang sama berkali-kali. Jika sebuah eksperimen menghasilkan sepuluh nilai tegangan, membuat variabel `v1`, `v2`, sampai `v10` sulit diperluas dan tidak cocok dengan pengulangan. Jika rumus rata-rata atau energi kinetik ditulis ulang di banyak tempat, program menjadi panjang dan sulit diperiksa.

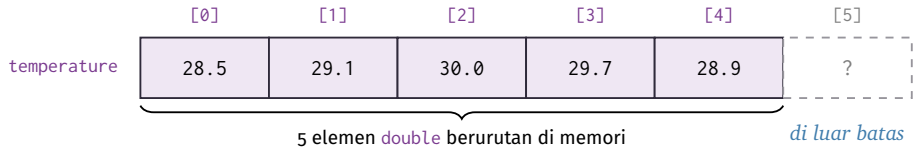
Kuliah ini memperkenalkan dua alat untuk mengatasi persoalan tersebut. **Larik** (*array*) menyimpan sekumpulan data bertipe sama, sedangkan **fungsi** membagi program menjadi bagian-bagian dengan tanggung jawab tertentu. Keduanya membawa kita pada hubungan antara data dan memori, sehingga kita juga memperkenalkan *pointer* secara terbatas, serta *string* sebagai larik karakter.

6.1 Larik satu dimensi

Larik satu dimensi adalah sekumpulan elemen bertipe sama yang disimpan berurutan di memori dan diakses dengan **indeks**. Lima data temperatur $T_0, T_1, \ldots, T_4$ dapat disimpan dalam

```
double temperature[5];
```

Deklarasi ini membuat lima elemen bertipe `double` dengan indeks yang valid `0` sampai `4`. Secara umum, larik berukuran N mempunyai indeks $0 \le i \le N - 1$; C selalu memulai indeks dari nol, sehingga indeks N berada *di luar* larik.



Setiap elemen diperlakukan seperti variabel biasa: `temperature[0] = 28.5;` menyimpan nilai, dan `double first = temperature[0];` membacanya. Perbedaan antara nomor urut manusia (“data kelima”) dan indeks program (`temperature[4]`) harus selalu diperhatikan.

Larik dapat diinisialisasi saat dideklarasikan. Jika seluruh nilai awal dituliskan, ukurannya boleh ditentukan oleh *compiler*, dan banyaknya elemen dapat dihitung dengan `sizeof`:

```
double temperature[] = {28.5, 29.1, 30.0, 29.7, 28.9};
int n = sizeof(temperature) / sizeof(temperature[0]);   /* n = 5 */
```

Jika satu `double` berukuran 8 byte, `sizeof(temperature)` bernilai 40 byte, sehingga $n = 40/8 = 5$. Teknik ini hanya berlaku di tempat larik dideklarasikan; di dalam fungsi yang menerima larik sebagai parameter, teknik ini tidak berlaku (Subbab 6.5).

6.1.1 Penelusuran, kapasitas, dan batas larik

Larik sangat cocok dipadukan dengan `for`. Pola penelusuran seluruh elemen adalah `for (int i = 0; i < n; i++)`; batas `i < n` menunjukkan secara langsung bahwa indeks `n` tidak diakses. Semua pola iteratif dari Kuliah 5 dapat langsung diterapkan pada larik; bedanya, nilai diambil dari `data[i]`, bukan dibaca dari masukan.

```
double sum = 0.0;
double minimum = data[0];
double maximum = data[0];

for (int i = 0; i < n; i++)
{
    sum += data[i];
    if (data[i] < minimum) minimum = data[i];
    if (data[i] > maximum) maximum = data[i];
}
double mean = sum / n;
```

Ukuran larik di C bersifat tetap. Jika banyaknya data baru diketahui saat program berjalan, kita menyediakan **kapasitas** maksimum dan menyimpan **banyaknya data aktual** pada variabel terpisah:

```
#define MAX_DATA 100

double data[MAX_DATA];
int n;                      /* banyaknya data yang benar-benar dipakai */

scanf("%d", &n);
if (n <= 0 || n > MAX_DATA)
{
    printf("Invalid number of data.\n");
    return 1;
}
for (int i = 0; i < n; i++)
{
    scanf("%lf", &data[i]);
```

```
}
```

Direktif `#define MAX_DATA 100` mendefinisikan konstanta yang dapat dipakai sebagai ukuran larik. Pemeriksaan `n > MAX_DATA` sangat penting, karena **C tidak memeriksa batas larik secara otomatis**. Mengakses `data[100]` atau indeks negatif disebut *out-of-bounds access*; akibatnya tidak dapat diramalkan: nilai aneh, program berhenti tiba-tiba, atau—yang paling berbahaya—program tampak berjalan normal padahal telah merusak data lain di memori.

6.1.2 Contoh: kecepatan dari data posisi

Sebuah eksperimen gerak satu dimensi menghasilkan posisi pada selang waktu tetap $\Delta t = 0.5$ s. Kita menyimpan posisi dalam larik karena setiap nilai digunakan kembali untuk menghitung kecepatan rata-rata pada interval ke-i,

$$v_i = \frac{x_{i+1} - x_i}{\Delta t}.$$

```c
#include <stdio.h>

int main(void)
{
    double position[] = {0.0, 1.2, 2.5, 3.9, 5.4, 7.0};   /* m */
    int n = sizeof(position) / sizeof(position[0]);
    const double dt = 0.5;                                  /* s */

    for (int i = 0; i < n - 1; i++)
    {
        double velocity = (position[i + 1] - position[i]) / dt;
        printf("interval %d: v = %.2f m/s\n", i, velocity);
    }

    return 0;
}
```

```
interval 0: v = 2.40 m/s
interval 1: v = 2.60 m/s
interval 2: v = 2.80 m/s
interval 3: v = 3.00 m/s
interval 4: v = 3.20 m/s
```

Dari N titik posisi hanya terdapat $N-1$ interval, sehingga batas `i < n - 1` mempunyai makna fisis sekaligus makna komputasi: indeks `i + 1` tidak boleh melewati `n - 1`. Kecepatan yang terus bertambah menunjukkan bahwa benda dipercepat.

Mengapa menyimpan data dalam larik, padahal pemrosesan *streaming* pada Kuliah 5 hanya memerlukan memori $O(1)$? Karena dengan larik, data yang sama dapat dianalisis berkali-kali: menghitung rata-rata, RMS, perubahan antarsampel, melakukan pencarian, atau mengurutkan data tanpa membaca ulang masukan. Harganya adalah kebutuhan memori yang bertumbuh sebanding dengan banyaknya data, yaitu $O(N)$.

6.2 Larik dua dimensi

Data berbentuk tabel, misalnya tiga sensor yang masing-masing menghasilkan empat pengukuran, dapat dipandang sebagai matriks 3×4 dan disimpan dalam larik dua dimensi `double voltage[3][4];`. Elemen pada baris kedua dan kolom ketiga diakses sebagai `voltage[1][2]`; kedua indeks dimulai dari nol.

	sampel 0	sampel 1	sampel 2	sampel 3
sensor 0	1.0	1.1	1.2	1.3
sensor 1	2.0	2.1	2.2	2.3
sensor 2	3.0	3.1	3.2	3.3

sel berwarna: `voltage[1][2]`

Larik dua dimensi diinisialisasi baris demi baris dan biasanya ditelusuri dengan dua pengulangan bersarang: pengulangan luar memilih baris, pengulangan dalam memilih kolom. Program berikut menghitung rata-rata setiap sensor.

```c
#include <stdio.h>

int main(void)
{
    double voltage[3][4] =
    {
        {1.0, 1.1, 1.2, 1.3},
        {2.0, 2.1, 2.2, 2.3},
        {3.0, 3.1, 3.2, 3.3}
    };

    for (int sensor = 0; sensor < 3; sensor++)
    {
        double sum = 0.0;
        for (int sample = 0; sample < 4; sample++)
        {
            sum += voltage[sensor][sample];
        }
        printf("Sensor %d mean = %.3f V\n", sensor, sum / 4.0);
    }

    return 0;
}
```

```
Sensor 0 mean = 1.150 V
Sensor 1 mean = 2.150 V
Sensor 2 mean = 3.150 V
```

Kedua indeks mempunyai makna fisis: indeks pertama memilih sensor, indeks kedua memilih pengukuran. Jika banyaknya baris dan kolom sama-sama N, penelusuran seluruh elemen membutuhkan sekitar N^2 operasi, yaitu $O(N^2)$.

6.3 Fungsi

Fungsi adalah bagian program yang mempunyai nama dan tugas tertentu. Kita sebenarnya sudah menggunakan fungsi sejak awal: `main`, `printf`, `scanf`, dan `sqrt`. Sekarang kita belajar membuat fungsi sendiri. Bentuk umumnya adalah

```
tipe_kembalian nama_fungsi(daftar_parameter)
{
    pernyataan
    return nilai;
}
```

Misalnya, fungsi berikut menerima massa dan kecepatan, lalu mengembalikan energi kinetik.

```
double kinetic_energy(double mass, double velocity)
{
    return 0.5 * mass * velocity * velocity;
}
```

Tipe `double` di depan nama menyatakan tipe nilai kembalian. `mass` dan `velocity` adalah **parameter**, yaitu variabel yang menerima data ketika fungsi dipanggil. Nilai yang benar-benar dikirim saat pemanggilan, misalnya `2.0` dan `3.0` pada `kinetic_energy(2.0, 3.0)`, disebut **argumen**. Pernyataan `return` mengakhiri fungsi dan menyerahkan hasilnya kepada pemanggil; pemanggilan fungsi dapat digunakan langsung di dalam ekspresi lain.

6.3.1 Prototipe

Fungsi harus dikenal *compiler* sebelum dipanggil. Jika definisinya diletakkan setelah `main`, kita menuliskan **prototipe** di bagian atas program. Prototipe memberi tahu nama fungsi, tipe kembalian, serta tipe parameternya, sehingga *compiler* dapat memeriksa setiap pemanggilan. Dengan susunan ini, `main` dapat diletakkan di awal program sehingga alur besarnya mudah dibaca.

```
#include <stdio.h>

double celsius_to_kelvin(double celsius);     /* prototipe */
void print_separator(void);

int main(void)
{
    double values[] = {-40.0, 0.0, 25.0, 100.0};

    print_separator();
    for (int i = 0; i < 4; i++)
    {
```

```
        printf("%7.2f C = %7.2f K\n", values[i], celsius_to_kelvin(values[i]));
    }
    print_separator();

    return 0;
}

double celsius_to_kelvin(double celsius)
{
    return celsius + 273.15;
}

void print_separator(void)
{
    printf("------------------------\n");
}
```

```
------------------------
 -40.00 C =  233.15 K
   0.00 C =  273.15 K
  25.00 C =  298.15 K
 100.00 C =  373.15 K
------------------------
```

Fungsi `print_separator` bertipe `void`: ia hanya melakukan tindakan dan tidak mengembalikan nilai. Dalam C, gagasan "prosedur" diwujudkan sebagai fungsi `void`.

6.3.2 Pengiriman berdasarkan nilai

Untuk parameter bertipe skalar seperti `int` dan `double`, C menggunakan **pengiriman berdasarkan nilai** (*pass by value*): fungsi menerima *salinan* argumen, bukan variabel asli milik pemanggil.

```
#include <stdio.h>

void add_one(int x)
{
    x = x + 1;
    printf("inside add_one: x = %d\n", x);
}

int main(void)
{
    int value = 5;

    add_one(value);
    printf("in main       : value = %d\n", value);

    return 0;
}
```

```
inside add_one: x = 6
in main       : value = 5
```

Perubahan terhadap `x` hanya memengaruhi salinan lokal. Cara agar fungsi dapat mengubah variabel pemanggil akan dibahas bersama *pointer* pada Subbab 6.6.

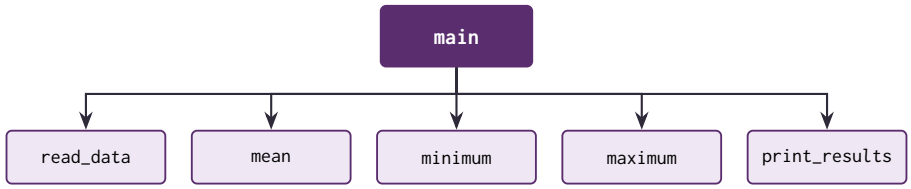
6.3.3 Ruang lingkup variabel

Ruang lingkup (*scope*) menentukan bagian program tempat suatu nama dapat digunakan. Variabel yang dideklarasikan di dalam fungsi, termasuk parameternya, adalah **variabel lokal**: hanya tersedia di dalam fungsi tersebut dan hilang ketika fungsi selesai. Nama yang sama di fungsi lain merujuk ke variabel yang berbeda. Variabel yang dideklarasikan di luar semua fungsi adalah **variabel global** dan dapat dipakai oleh semua fungsi dalam berkas yang sama.

Variabel global memang praktis, tetapi variabel global yang dapat berubah menciptakan ketergantungan tersembunyi: hasil suatu fungsi tidak lagi hanya ditentukan oleh argumennya. Fungsi semacam itu sulit diuji dan digunakan kembali. Prinsip umumnya adalah menggunakan ruang lingkup sekecil mungkin dan mengirim data melalui parameter.

6.4 Modularisasi program

Modularisasi berarti membagi program menjadi bagian-bagian dengan tanggung jawab yang jelas, dan fungsi adalah alat utamanya di C. Program analisis data, misalnya, dapat dipecah menjadi fungsi untuk membaca data, menghitung rata-rata, minimum, dan maksimum, serta menampilkan hasil. Fungsi `main` kemudian hanya berperan sebagai **pengatur alur**.



Program modular lebih mudah dibaca dan diuji. Jika satu bagian bermasalah, pemeriksaan dapat dipusatkan pada fungsi yang bertanggung jawab. Fungsi yang menerima seluruh datanya melalui parameter juga dapat digunakan kembali dalam konteks lain tanpa perubahan.

```c
#include <stdio.h>

double kinetic_energy(double mass, double velocity);
double momentum(double mass, double velocity);

int main(void)
{
    double mass = 2.0;       /* kg  */
    double velocity = 3.0;   /* m/s */
```

```
    printf("Energy   = %.3f J\n", kinetic_energy(mass, velocity));
    printf("Momentum = %.3f kg m/s\n", momentum(mass, velocity));

    return 0;
}

double kinetic_energy(double mass, double velocity)
{
    return 0.5 * mass * velocity * velocity;
}

double momentum(double mass, double velocity)
{
    return mass * velocity;
}
```

```
Energy   = 9.000 J
Momentum = 6.000 kg m/s
```

6.4.1 Merancang fungsi yang baik

Fungsi yang baik mempunyai **satu tanggung jawab utama** dan nama yang menjelaskan tugasnya, misalnya `mean`, `rms`, `kinetic_energy`, `celsius_to_kelvin`, atau `read_measurements`. Data yang dibutuhkan diterima melalui parameter, dan hasilnya dikembalikan melalui `return` (atau melalui parameter *pointer* jika hasilnya lebih dari satu).

Tidak ada aturan bahwa fungsi harus sangat pendek. Fungsi yang mengerjakan banyak tugas yang tidak berkaitan sebaiknya dipecah; sebaliknya, memecah setiap satu atau dua baris menjadi fungsi terpisah juga membuat program sulit diikuti. Tujuan modularisasi bukan memperbanyak fungsi, melainkan membuat setiap bagian program mempunyai peran yang dapat dijelaskan dan diuji.

6.5 Fungsi yang menerima larik

Fungsi menjadi sangat berguna ketika digabungkan dengan larik. Fungsi rata-rata berikut menerima larik `double` dan banyaknya elemen.

```
double mean(const double data[], int n)
{
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        sum += data[i];
    }
    return sum / n;          /* prakondisi: n > 0 */
}
```

Dua hal perlu diperhatikan.

Pertama, ukuran larik harus dikirim secara eksplisit. Ketika larik dikirim ke fungsi, yang sebenarnya dikirim adalah alamat elemen pertamanya. Informasi ukuran total tidak ikut terkirim, sehingga `sizeof` di dalam fungsi tidak memberikan banyaknya elemen:

```c
#include <stdio.h>

void show_size(const double data[])
{
    printf("inside function: sizeof(data) = %zu byte\n", sizeof(data));
}

int main(void)
{
    double values[10] = {0.0};

    printf("in main        : sizeof(values) = %zu byte\n", sizeof(values));
    show_size(values);

    return 0;
}
```

Saat dikompilasi, *compiler* sudah memperingatkan masalah ini:

```
program.c: In function ‘show_size’:
program.c:5:64: warning: ‘sizeof’ on array function parameter ‘data’ will return size of ‘
     ↪ const double *’ [-Wsizeof-array-argument]
    5 |     printf("inside function: sizeof(data) = %zu byte\n", sizeof(data));
      |                                                                ^
program.c:3:29: note: declared here
    3 | void show_size(const double data[])
      |                ~~~~~~~~~~~~~~~^~~~~
```

Keluaran programnya adalah

```
in main        : sizeof(values) = 80 byte
inside function: sizeof(data) = 8 byte
```

Di dalam `main`, `sizeof(values)` adalah $10 \times 8 = 80$ byte, sedangkan di dalam fungsi hasilnya 8 byte, yaitu ukuran sebuah alamat. Dalam C, pemrogram bertanggung jawab mengelola informasi ukuran, berbeda dengan bahasa seperti Python yang menyimpan panjang sebagai bagian dari objek larik.

Kedua, gunakan `const` jika fungsi hanya membaca larik. Karena fungsi bekerja langsung pada alamat data asli, fungsi *dapat* mengubah isi larik pemanggil. Kata `const` menyatakan bahwa fungsi tidak bermaksud mengubahnya, dan *compiler* akan menolak upaya perubahan yang tidak disengaja. Jika fungsi memang harus mengubah larik, misalnya menambahkan *offset* kalibrasi ke setiap elemen, `const` tidak digunakan:

```c
void add_offset(double data[], int n, double offset)
{
    for (int i = 0; i < n; i++)
```

```
    {
        data[i] += offset;          /* mengubah larik asli */
    }
}
```

Untuk larik dua dimensi, fungsi perlu mengetahui ukuran dimensi kedua agar posisi setiap elemen dapat dihitung, misalnya `void print_matrix(double data[][4], int rows)`.

6.5.1 Studi kasus: statistik data getaran

Pada Kuliah 5 kita menganalisis data getaran secara *streaming*. Jika data disimpan dalam larik, setiap besaran dapat dihitung oleh fungsi tersendiri. Selain rata-rata dan RMS, kita menghitung amplitudo maksimum absolut dan *crest factor* $C = \max|a_i| / a_{\text{RMS}}$, yang menunjukkan seberapa "runcing" puncak sinyal dibandingkan amplitudo efektifnya.

```c
#include <stdio.h>
#include <math.h>

double mean(const double data[], int n);
double rms(const double data[], int n);
double max_abs(const double data[], int n);

int main(void)
{
    double acceleration[] = {1.2, -2.4, 8.7, -9.1, 3.6};   /* m/s^2 */
    int n = sizeof(acceleration) / sizeof(acceleration[0]);

    double a_mean = mean(acceleration, n);
    double a_rms = rms(acceleration, n);
    double a_peak = max_abs(acceleration, n);

    printf("Mean         = %.3f m/s^2\n", a_mean);
    printf("RMS          = %.3f m/s^2\n", a_rms);
    printf("Max |a|      = %.3f m/s^2\n", a_peak);
    printf("Crest factor = %.3f\n", a_peak / a_rms);

    return 0;
}

double mean(const double data[], int n)
{
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        sum += data[i];
    }
    return sum / n;
}

double rms(const double data[], int n)
```

```
{
    double sum_square = 0.0;
    for (int i = 0; i < n; i++)
    {
        sum_square += data[i] * data[i];
    }
    return sqrt(sum_square / n);
}

double max_abs(const double data[], int n)
{
    double result = fabs(data[0]);
    for (int i = 1; i < n; i++)
    {
        if (fabs(data[i]) > result)
        {
            result = fabs(data[i]);
        }
    }
    return result;
}
```

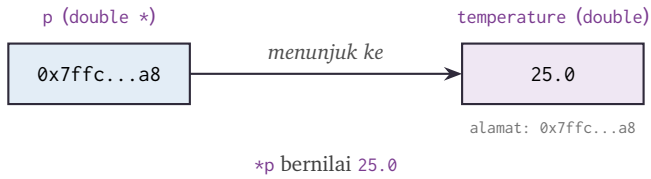
```
Mean         = 0.400 m/s^2
RMS          = 5.978 m/s^2
Max |a|      = 9.100 m/s^2
Crest factor = 1.522
```

Hasil rata-rata dan RMS sama dengan studi kasus Kuliah 5 (tanpa data tidak valid 75). Setiap fungsi mempunyai satu tanggung jawab dan hanya membutuhkan larik serta ukurannya, sehingga dapat diuji secara terpisah dengan data yang hasilnya diketahui—misalnya larik $\{3, -3\}$ yang harus menghasilkan rata-rata 0, RMS 3, dan *crest factor* 1.

6.6 Pengantar *pointer*

Pointer adalah variabel yang menyimpan alamat memori. Konsep ini menjelaskan mengapa `scanf` memerlukan tanda `&`, mengapa larik dapat berubah ketika dikirim ke fungsi, dan bagaimana fungsi dapat menghasilkan lebih dari satu keluaran.

Operator `&` menghasilkan alamat suatu variabel. Alamat tersebut dapat disimpan dalam variabel bertipe *pointer*: `double *p = &temperature;` membuat `p` bertipe `double *`, yaitu "penunjuk ke `double`". Operator `*` di depan *pointer* melakukan **dereferensi**: `*p` adalah nilai yang tersimpan di alamat yang ditunjuk `p`.




```c
#include <stdio.h>

int main(void)
{
    double temperature = 25.0;
    double *p = &temperature;

    printf("temperature = %.2f\n", temperature);
    printf("*p          = %.2f\n", *p);

    *p = 30.0;        /* mengubah temperature melalui alamatnya */

    printf("temperature = %.2f\n", temperature);

    return 0;
}
```

```
temperature = 25.00
*p          = 25.00
temperature = 30.00
```

Program ini memperlihatkan dua cara mengakses data yang sama: melalui nama variabel dan melalui alamat yang disimpan pada *pointer*.

6.6.1 Fungsi yang mengubah variabel pemanggil

Karena parameter skalar dikirim berdasarkan nilai, fungsi tidak dapat mengubah variabel pemanggil melalui salinannya. Jika yang dikirim adalah *alamat*, fungsi dapat mengubah nilai di alamat tersebut. Contoh klasiknya adalah fungsi `swap`.

```c
#include <stdio.h>

void swap(double *a, double *b)
{
    double temp = *a;
    *a = *b;
    *b = temp;
}

int main(void)
{
    double x = 2.0;
    double y = 5.0;
```

```
    printf("Before: x = %.1f, y = %.1f\n", x, y);
    swap(&x, &y);
    printf("After : x = %.1f, y = %.1f\n", x, y);

    return 0;
}
```

```
Before: x = 2.0, y = 5.0
After : x = 5.0, y = 2.0
```

Pemanggilan `swap(&x, &y)` mengirim alamat `x` dan `y`; inilah pola yang sama dengan `scanf("%lf", &x)`.

Pola ini juga memungkinkan fungsi menghasilkan lebih dari satu keluaran. Satu `return` hanya dapat mengembalikan satu nilai, tetapi minimum dan maksimum dapat dihitung sekaligus melalui dua parameter *pointer*:

```
void min_max(const double data[], int n, double *minimum, double *maximum)
{
    *minimum = data[0];
    *maximum = data[0];
    for (int i = 1; i < n; i++)
    {
        if (data[i] < *minimum) *minimum = data[i];
        if (data[i] > *maximum) *maximum = data[i];
    }
}
```

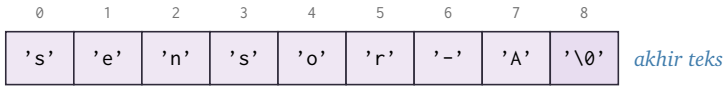
Fungsi ini dipanggil dengan `min_max(data, n, &min_value, &max_value);`. Setelah fungsi selesai, kedua variabel pemanggil telah berisi hasil.

6.6.2 Larik dan *pointer*

Elemen larik tersimpan berurutan di memori, dan dalam kebanyakan ekspresi nama larik berperilaku seperti alamat elemen pertamanya: `data` setara dengan `&data[0]`. Itulah sebabnya parameter `const double data[]` setara dengan `const double *data`, dan itulah sebabnya fungsi seperti `add_offset` dapat mengubah larik asli. Secara konseptual, `data[i]` berarti "nilai pada alamat `data` digeser *i* elemen", yang dapat ditulis `*(data + i)`. Aritmetika *pointer* semacam ini bukan fokus kuliah ini; untuk program tingkat awal, notasi indeks lebih mudah dibaca. Alokasi memori dinamis dan struktur data berbasis *pointer* juga berada di luar cakupan buku ini.

6.7 *String* sebagai larik karakter

C tidak mempunyai tipe *string* bawaan. *String* direpresentasikan sebagai larik `char` yang diakhiri karakter khusus `'\0'`, yaitu ***null terminator***. Deklarasi `char name[] = "sensor-A";` membuat larik sembilan elemen: delapan karakter dan satu *null terminator*.



Fungsi `printf` dengan format `%s` mencetak karakter mulai dari alamat awal sampai menemukan `'\0'`. Jika *null terminator* hilang, pembacaan berlanjut melewati batas larik. Karena itu, kapasitas larik karakter harus selalu menyediakan satu tempat untuk `'\0'`: larik `char name[100]` dapat menampung paling banyak 99 karakter teks.

Untuk membaca satu baris teks, `fgets(name, sizeof(name), stdin)` lebih aman daripada `scanf("%s", name)` karena menerima batas kapasitas dan dapat membaca spasi. Akan tetapi, `fgets` juga menyimpan karakter baris baru `'\n'` jika masih ada ruang, sehingga karakter itu biasanya dihapus dengan `name[strcspn(name, "\n")] = '\0';`.

Pustaka `string.h` menyediakan fungsi pengolah *string*, misalnya `strlen` (panjang teks sebelum `'\0'`), `strcmp` (membandingkan isi dua *string*), `strcpy` (menyalin), dan `strcat` (menggabungkan). Dua *string* **tidak** boleh dibandingkan dengan `==`, karena operator itu membandingkan alamat kedua larik, bukan isinya. `strcmp(a, b)` bernilai nol jika isi keduanya sama. Penyalinan dan penggabungan harus selalu memperhatikan kapasitas larik tujuan.

Program berikut menggabungkan larik karakter dan larik numerik: label eksperimen disimpan sebagai *string*, sedangkan hasil pengukuran sebagai `double`.

```c
#include <stdio.h>
#include <string.h>

double mean(const double data[], int n);

int main(void)
{
    char experiment[100];
    double voltage[4];

    printf("Experiment name: ");
    fgets(experiment, sizeof(experiment), stdin);
    experiment[strcspn(experiment, "\n")] = '\0';

    for (int i = 0; i < 4; i++)
    {
        printf("V%d: ", i);
        scanf("%lf", &voltage[i]);
    }

    printf("\nExperiment : %s (%zu characters)\n",
           experiment, strlen(experiment));
    printf("Mean voltage = %.3f V\n", mean(voltage, 4));

    if (strcmp(experiment, "kalibrasi") == 0)
    {
```

```
        printf("Note: calibration run.\n");
    }

    return 0;
}

double mean(const double data[], int n)
{
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        sum += data[i];
    }
    return sum / n;
}
```

```
Experiment name: Pengukuran
V0: V1: V2: V3:
Experiment : Pengukuran (10 characters)
Mean voltage = 0.000 V
```

Perhatikan bahwa nama eksperimen dapat memuat spasi karena dibaca dengan `fgets`, dan panjangnya tidak menghitung `'\n'` yang telah dihapus.

6.8 Kesalahan umum dan kebiasaan yang baik

- **Indeks keluar batas.** Indeks terakhir larik berukuran N adalah $N - 1$. Gunakan pola `i < n` dan periksa banyaknya data terhadap kapasitas.
- **Ukuran yang salah.** Jika hanya sebagian kapasitas larik yang terisi, telusuri sampai banyaknya data aktual, bukan sampai kapasitas; elemen yang belum diberi nilai berisi nilai sembarang.
- **Mengandalkan `sizeof` pada parameter larik.** Parameter larik hanyalah alamat; kirim ukurannya sebagai parameter tambahan.
- **Lupa bahwa fungsi dapat mengubah larik.** Gunakan `const` untuk parameter larik yang hanya dibaca.
- **Prototipe tidak ditulis atau tipe kembalian tidak sesuai.** Tanpa prototipe, *compiler* tidak dapat memeriksa argumen; fungsi yang tidak menghasilkan nilai sebaiknya bertipe `void`.
- **Terlalu bergantung pada variabel global.** Kirim data melalui parameter agar hubungan masukan dan keluaran fungsi terlihat jelas.
- ***Pointer* yang belum menunjuk objek valid.** Dereferensi *pointer* yang belum diinisialisasi menyebabkan perilaku tak terdefinisi. Pastikan tipenya sesuai (`double *` untuk `double`) dan gunakan *pointer* hanya jika memang diperlukan.
- **Kesalahan pada *string*.** Lupa menyediakan ruang untuk `'\0'`, membandingkan *string* dengan `==`, atau membaca masukan melebihi kapasitas.

Kebiasaan yang dianjurkan: pisahkan kapasitas dari banyaknya data; beri setiap fungsi satu tanggung jawab dan nama yang jelas; tulis prototipe di bagian atas program; gunakan `const` untuk data yang hanya dibaca; nyatakan prakondisi fungsi (misalnya $n > 0$) dalam komentar dan pastikan pemanggil memenuhinya; serta uji setiap fungsi secara terpisah dengan larik kecil yang hasilnya dapat dihitung manual.

6.9 Cek pemahaman

- Mengapa indeks larik berukuran N berjalan dari 0 sampai $N - 1$? Apa akibat *out-of-bounds access*?
- Apa perbedaan kapasitas larik dan banyaknya data aktual?
- Apa keuntungan dan kerugian menyimpan data dalam larik dibandingkan pemrosesan *streaming*?
- Apa perbedaan parameter dan argumen? Apa fungsi prototipe?
- Apa yang dimaksud dengan pengiriman berdasarkan nilai? Mengapa fungsi `add_one` tidak mengubah variabel pemanggil?
- Mengapa variabel global yang dapat berubah sebaiknya dihindari?
- Mengapa fungsi yang menerima larik juga memerlukan parameter ukuran? Apa fungsi `const` pada parameter larik?
- Apa arti `&x`, `double *p`, dan `*p`?
- Bagaimana fungsi dapat menghasilkan lebih dari satu keluaran?
- Mengapa *string* memerlukan *null terminator*, dan mengapa dua *string* tidak boleh dibandingkan dengan `==`?

6.10 Latihan

1. **Statistik tegangan.** Baca paling banyak 50 data tegangan ke dalam larik. Hitung rata-rata, minimum, maksimum, dan RMS menggunakan fungsi terpisah dengan prototipe dan parameter `const`. Uji dengan data yang hasilnya dapat dihitung manual.
2. **Kecepatan dari posisi.** Buat fungsi `void velocities(const double x[], int n, double dt, double v[])` yang mengisi larik kecepatan $v_i = (x_{i+1} - x_i)/\Delta t$. Jelaskan mengapa larik kecepatan berukuran $N-1$, dan uji dengan data gerak lurus beraturan yang hasilnya mudah diperiksa.
3. **Energi kinetik banyak partikel.** Diberikan larik massa dan larik kecepatan dengan panjang sama. Buat fungsi yang mengisi larik energi kinetik $E_{k,i} = \frac{1}{2} m_i v_i^2$ dan fungsi lain yang menghitung energi total. Jangan gunakan variabel global.
4. **Gelombang diskrit.** Isi larik dengan sampel $y_i = 0.03 \sin(2\pi \cdot 2 \cdot t_i)$ untuk $t_i = 0, 0.01, \ldots, 0.99$ s (gunakan pencacah bilangan bulat). Buat fungsi untuk amplitudo maksimum absolut dan RMS, lalu bandingkan RMS dengan $A/\sqrt{2}$.

5. **Matriks sensor.** Simpan data empat sensor pada enam waktu pengukuran dalam larik dua dimensi. Hitung rata-rata setiap sensor dan rata-rata semua sensor pada setiap waktu dengan fungsi yang sesuai. Jelaskan kompleksitas penelusurannya.
6. **Normalisasi.** Buat fungsi `void normalize(double data[], int n)` yang mengubah data sehingga nilai maksimum absolutnya menjadi satu, tanpa pembagian dengan nol jika semua data nol. Tampilkan larik sebelum dan sesudah pemanggilan untuk menunjukkan bahwa larik asli berubah.
7. **Fungsi `min_max`.** Buat program lengkap yang memanggil fungsi `min_max` pada kuliah ini. Jelaskan mengapa dua hasil tidak dapat dikembalikan dengan satu `return` biasa dan mengapa argumen pemanggilannya memerlukan `&`.
8. **Operasi vektor.** Representasikan vektor tiga dimensi dengan larik tiga elemen. Buat fungsi penjumlahan vektor (mengisi larik keluaran), hasil kali titik $\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z$ (mengembalikan `double`), dan besar vektor $|\mathbf{a}|$. Jelaskan mengapa antarmuka ketiga fungsi berbeda.
9. **Identitas eksperimen.** Baca nama eksperimen dan nama operator dengan `fgets`, hapus `'\n'`, lalu tampilkan keduanya beserta panjangnya. Tolak masukan kosong dengan meminta pengguna mengulang. Kemudian baca dua nama sensor dan bandingkan dengan `strcmp`; jelaskan mengapa `==` tidak dapat digunakan.
10. **Proyek mini: gerak satu dimensi.** Simpan data waktu dan posisi dalam dua larik. Buat fungsi untuk menghitung kecepatan setiap interval, rata-rata kecepatan, kecepatan maksimum absolut, serta indeks interval tempat maksimum terjadi. Tampilkan tabel waktu awal, waktu akhir, posisi awal, posisi akhir, dan kecepatan setiap interval.

6.11 Rangkuman

- Larik menyimpan sekumpulan data bertipe sama secara berurutan. Indeks dimulai dari nol, indeks terakhir adalah $N - 1$, dan C tidak memeriksa batas larik secara otomatis.
- Kapasitas larik dipisahkan dari banyaknya data aktual. Menyimpan N data membutuhkan memori $O(N)$, tetapi memungkinkan data dianalisis berkali-kali.
- Larik dua dimensi merepresentasikan data tabel dan ditelusuri dengan pengulangan bersarang.
- Fungsi mempunyai tipe kembalian, nama, parameter, dan tubuh. Prototipe memungkinkan *compiler* memeriksa pemanggilan. Fungsi `void` tidak mengembalikan nilai.
- Parameter skalar dikirim berdasarkan nilai. Variabel lokal hanya berlaku di dalam fungsinya; variabel global yang dapat berubah sebaiknya dihindari.
- Modularisasi membagi program menjadi fungsi dengan tanggung jawab yang jelas; `main` berperan sebagai pengatur alur.
- Fungsi yang menerima larik memerlukan ukuran larik sebagai parameter terpisah. Fungsi dapat mengubah larik asli; `const` menandai larik yang hanya dibaca.

- *Pointer* menyimpan alamat memori. Operator `&` mengambil alamat dan `*` melakukan dereferensi. Parameter *pointer* memungkinkan fungsi mengubah variabel pemanggil dan menghasilkan lebih dari satu keluaran.
- *String* adalah larik `char` yang diakhiri `'\0'`. Gunakan `fgets` untuk membaca baris teks dan `strcmp` untuk membandingkan isi *string*.
- Kuliah berikutnya membahas data terstruktur dengan `struct` dan pengolahan berkas, sehingga data eksperimen dapat dibaca dari dan disimpan ke berkas.

KULIAH

7

Data Terstruktur dan Pengolahan Berkas

Larik, fungsi, *pointer*, dan *string* memungkinkan kita menyimpan banyak nilai bertipe sama dan membagi program menjadi bagian-bagian yang jelas. Namun, data teknik jarang terdiri atas satu besaran saja. Satu titik pengukuran dapat memuat waktu, temperatur, tegangan, nama sensor, dan status yang semuanya membentuk satu kesatuan. Jika setiap besaran disimpan dalam larik terpisah, program masih dapat bekerja, tetapi hubungan bahwa nilai-nilai tersebut berasal dari satu pengukuran menjadi tidak terlihat.

Selain itu, data di memori hilang ketika program selesai. Data eksperimen biasanya berasal dari berkas dan hasil analisis perlu disimpan ke berkas. Kuliah ini membahas dua hal tersebut: **data terstruktur** dengan `struct` dan **pengolahan berkas** teks, khususnya format CSV.

7.1 Data terstruktur dengan `struct`

`struct` mengelompokkan beberapa variabel, yang boleh berbeda tipe, menjadi satu tipe baru. Setiap bagiannya disebut **anggota** (*member*). Tipe untuk satu pengukuran temperatur dapat didefinisikan sebagai

```
struct Measurement
{
    double time;          /* s */
    double temperature;   /* C */
};
```

Definisi ini belum membuat variabel; ia hanya mendefinisikan *bentuk* data. Variabelnya dibuat dengan `struct Measurement m;`, dan anggotanya diakses dengan operator titik: `m.time = 2.5;` serta `m.temperature = 31.2;`.

Penulisan `struct Measurement` dapat disingkat dengan `typedef`, sehingga variabel cukup dideklarasikan sebagai `Measurement m;`. Bentuk `typedef struct` ini akan kita gunakan seterusnya.

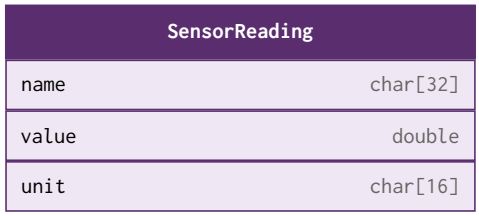
```
typedef struct
```

```
{
    double time;
    double temperature;
} Measurement;

Measurement a = {2.5, 31.2};                          /* urut sesuai anggota */
Measurement b = {.time = 3.0, .temperature = 31.9};   /* designated initializer */
```

Inisialisasi pertama mengikuti urutan anggota, sedangkan bentuk kedua (*designated initializer*) menyebutkan nama anggota secara eksplisit sehingga lebih mudah dibaca dan tidak bergantung pada urutan.

Anggota `struct` tidak harus bertipe sama. Satu rekaman pembacaan sensor dapat memuat bilangan dan teks sekaligus:



satu rekaman:
`"pressure_1"`, `101.3`, `"kPa"`

```
typedef struct
{
    char name[32];
    double value;
    char unit[16];
} SensorReading;

SensorReading reading = {.name = "pressure_1", .value = 101.3, .unit = "kPa"};
```

Karena `name` adalah larik karakter, isinya tidak dapat diganti dengan *assignment* biasa: `reading.name = "sensor_A";` tidak valid. Gunakan `strcpy(reading.name, "sensor_A");` dari `string.h`, setelah memastikan kapasitas tujuan mencukupi.

Data terstruktur membuat hubungan antarbesaran terlihat langsung di dalam program: ketika kita melihat `m.time` dan `m.temperature`, kita tahu keduanya berasal dari pengukuran yang sama. Satu `struct` dapat dipandang sebagai satu **rekaman**, dan—seperti akan kita lihat—satu baris dalam berkas CSV.

7.2 Larik `struct` dan fungsi

Banyak rekaman dengan bentuk yang sama disimpan dalam larik `struct`. Larik mengatur banyak rekaman, sedangkan `struct` mengatur isi setiap rekaman: `data[0].time` adalah waktu rekaman pertama dan `data[0].temperature` temperaturnya.

```
#include <stdio.h>
```

```c
typedef struct
{
    double time;
    double temperature;
} Measurement;

int main(void)
{
    Measurement data[] =
    {
        {0.0, 25.1},
        {1.0, 25.8},
        {2.0, 26.7},
        {3.0, 27.4}
    };
    int n = sizeof(data) / sizeof(data[0]);

    for (int i = 0; i < n; i++)
    {
        printf("t = %.1f s, T = %.2f C\n", data[i].time, data[i].temperature);
    }

    return 0;
}
```

```
t = 0.0 s, T = 25.10 C
t = 1.0 s, T = 25.80 C
t = 2.0 s, T = 26.70 C
t = 3.0 s, T = 27.40 C
```

7.2.1 Mengirim struct ke fungsi

struct dapat dikirim ke fungsi seperti tipe lain. Untuk struct kecil, pengiriman berdasarkan nilai sudah memadai. Untuk struct yang lebih besar, atau jika fungsi perlu mengubahnya, kita mengirim alamatnya dan mengakses anggota dengan operator panah ->; bentuk m->time setara dengan (*m).time. Model fisika dapat dipetakan langsung ke fungsi seperti ini. Untuk data listrik dengan daya sesaat $P_i = V_i I_i$:

```c
typedef struct
{
    double time;      /* s */
    double voltage;   /* V */
    double current;   /* A */
} ElectricalMeasurement;

double power(const ElectricalMeasurement *m)
{
    return m->voltage * m->current;
}
```

Setiap rekaman menyimpan semua data yang dibutuhkan untuk satu evaluasi model.

Fungsi juga dapat *mengembalikan* `struct`. Ini berguna ketika beberapa besaran secara logis merupakan satu hasil, misalnya rentang data. Bandingkan dengan fungsi `min_max` pada Kuliah 6 yang menggunakan dua parameter *pointer*; keduanya sah, tetapi `struct` sering membuat hasil yang saling berkaitan lebih mudah dipahami.

```
typedef struct
{
    double minimum;
    double maximum;
} Range;

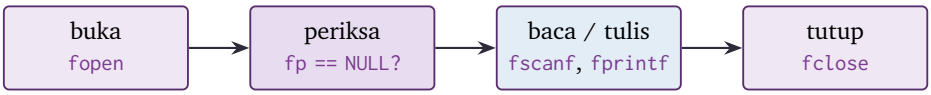
Range find_range(const double data[], int n)
{
    Range result = {.minimum = data[0], .maximum = data[0]};

    for (int i = 1; i < n; i++)
    {
        if (data[i] < result.minimum) result.minimum = data[i];
        if (data[i] > result.maximum) result.maximum = data[i];
    }
    return result;
}
```

7.3 Berkas teks: membuka, menulis, dan menutup

Larik dan `struct` hanya hidup selama program berjalan. Agar data dapat digunakan kembali, dianalisis program lain, atau diperiksa manusia, data perlu disimpan ke **berkas** (*file*). Kita berfokus pada berkas teks karena formatnya transparan, mudah diperiksa, dan cocok untuk data pengukuran.

Pustaka `stdio.h` menyediakan tipe `FILE`; program bekerja dengan *pointer* `FILE *fp` yang menyimpan informasi untuk mengelola aliran data ke atau dari berkas (bukan isi berkas itu sendiri). Setiap operasi berkas mengikuti pola empat langkah yang sama.



Fungsi `fopen(nama, mode)` membuka berkas dan mengembalikan `NULL` jika gagal. Mode dasar yang penting adalah sebagai berikut.

Mode	Kegunaan
`"r"`	membaca berkas yang sudah ada
`"w"`	menulis dari awal; isi lama *dihapus*
`"a"`	menambahkan data pada akhir berkas; isi lama dipertahankan

Mode `"w"` harus digunakan dengan hati-hati karena dapat menghapus data mentah yang berharga.

7.3.1 Memeriksa keberhasilan `fopen`

`fopen` dapat gagal karena nama berkas salah, direktori tidak ada, atau izin akses tidak mencukupi. *Pointer* hasil `fopen` tidak boleh digunakan sebelum diperiksa. Fungsi `perror` menampilkan pesan yang menjelaskan penyebab kegagalan dari sistem operasi, sehingga lebih informatif daripada pesan umum.

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("data_missing.csv", "r");

    if (fp == NULL)
    {
        perror("data_missing.csv");
        return 1;
    }

    /* baca data di sini */

    fclose(fp);
    return 0;
}
```

```
data_missing.csv: No such file or directory
```

Setelah selesai, berkas harus ditutup dengan `fclose(fp)` agar data yang masih tertahan di *buffer* benar-benar ditulis dan sumber daya dilepaskan. Untuk berkas keluaran yang penting, nilai kembalian `fclose` (nol jika berhasil) juga dapat diperiksa.

7.3.2 Menulis dengan `fprintf`

`fprintf` bekerja seperti `printf`, tetapi argumen pertamanya adalah berkas tujuan. Program berikut menulis tabel temperatur yang naik linear, $T(t) = 25.0 + 0.8t$.

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("temperature.txt", "w");
    if (fp == NULL)
    {
        perror("temperature.txt");
        return 1;
    }
```

```c
    for (int i = 0; i <= 5; i++)
    {
        double time = i * 1.0;
        double temperature = 25.0 + 0.8 * time;
        fprintf(fp, "%.2f %.2f\n", time, temperature);
    }

    fclose(fp);
    printf("Data written to temperature.txt\n");

    return 0;
}
```

```
Data written to temperature.txt
```

Isi berkas `temperature.txt` yang dihasilkan:

```
0.00 25.00
1.00 25.80
2.00 26.60
3.00 27.40
4.00 28.20
5.00 29.00
```

Format teks sederhana ini mudah dibaca manusia dan mudah diproses kembali oleh program.

7.4 Membaca berkas teks

`fscanf` bekerja seperti `scanf`, tetapi membaca dari berkas. Nilai kembaliannya adalah banyaknya item yang berhasil dibaca, sehingga pola pembacaan yang benar adalah mengulang *selama pembacaan berhasil*:

```c
#include <stdio.h>

int main(void)
{
    FILE *fp = fopen("temperature.txt", "r");
    if (fp == NULL)
    {
        perror("temperature.txt");
        return 1;
    }

    double time, temperature;
    double sum = 0.0;
    int n = 0;

    while (fscanf(fp, "%lf %lf", &time, &temperature) == 2)
    {
```

```
        sum += temperature;
        n++;
    }
    fclose(fp);

    if (n > 0)
    {
        printf("Records read     = %d\n", n);
        printf("Mean temperature = %.3f C\n", sum / n);
    }

    return 0;
}
```

```
Records read     = 6
Mean temperature = 27.000 C
```

Pengulangan berhenti ketika `fscanf` tidak lagi berhasil membaca dua nilai, baik karena akhir berkas tercapai maupun karena format tidak sesuai.

Pola yang sering ditemui pemula, `while (!feof(fp))`, sebaiknya dihindari. Fungsi `feof` baru bernilai benar *setelah* suatu operasi baca mencoba melewati akhir berkas. Akibatnya, pengulangan berjalan satu kali terlalu banyak: pembacaan terakhir gagal, tetapi data lama di variabel tetap diproses sebagai rekaman tambahan. Selalu periksa hasil operasi baca itu sendiri.

Untuk membaca teks per baris, gunakan `fgets(line, sizeof(line), fp)`, yang mengembalikan `NULL` jika tidak ada lagi baris yang dapat dibaca. Pola `fgets` inilah dasar pembacaan CSV yang andal.

7.5 Format CSV

CSV (*comma-separated values*) menyimpan data tabel dalam berkas teks: setiap baris adalah satu rekaman dan kolomnya dipisahkan koma. Baris pertama biasanya berisi nama kolom (*header*). Format ini dapat dibuka oleh perangkat lunak lembar kerja dan dibaca dengan mudah oleh Python (Kuliah 9). Contoh berkas data listrik `electrical.csv`:

```
time,voltage,current
0.0,5.02,0.21
0.5,5.01,0.22
1.0,4.98,0.24
1.5,4.95,0.25
2.0,4.93,0.26
2.5,4.90,0.27
```

Sebelum menulis atau membaca CSV, tetapkan **skema** berkasnya: urutan kolom, nama kolom, dan satuannya. Nama kolom yang memuat satuan, misalnya `time_s` atau `voltage_V`, mencegah salah tafsir di kemudian hari.

7.5.1 Membaca CSV dengan `fgets` dan `sscanf`

Pendekatan yang andal adalah membaca satu baris penuh dengan `fgets`, lalu menguraikannya (*parsing*) dengan `sscanf`, yang bekerja seperti `scanf` tetapi membaca dari *string*. Dengan cara ini, setiap baris diperiksa terpisah; baris yang rusak dapat ditolak tanpa mengacaukan pembacaan baris berikutnya. *Header* dilewati dengan satu `fgets` di awal. Setiap baris yang valid dipetakan ke satu `struct`, sehingga representasi data di berkas konsisten dengan representasi di memori.

Misalkan sebuah berkas data memuat beberapa baris rusak:

```
time,voltage,current
0.0,5.02,0.21
0.5,5.01
1.0,4.98,0.24
error,sensor,timeout
1.5,4.95,0.25
```

```c
#include <stdio.h>

#define MAX_DATA 1000

typedef struct
{
    double time;
    double voltage;
    double current;
} ElectricalMeasurement;

int main(void)
{
    FILE *fp = fopen("sensor_raw.csv", "r");
    if (fp == NULL)
    {
        perror("sensor_raw.csv");
        return 1;
    }

    ElectricalMeasurement data[MAX_DATA];
    int n = 0;
    int rejected = 0;
    char line[256];

    if (fgets(line, sizeof(line), fp) == NULL)     /* lewati header */
    {
        printf("File is empty.\n");
        fclose(fp);
        return 1;
    }

    while (fgets(line, sizeof(line), fp) != NULL)
```

```
    {
        ElectricalMeasurement m;
        int matched = sscanf(line, "%lf,%lf,%lf", &m.time, &m.voltage, &m.current);

        if (matched != 3)
        {
            printf("Skipping invalid line: %s", line);
            rejected++;
            continue;
        }
        if (n >= MAX_DATA)
        {
            printf("Data capacity exceeded.\n");
            break;
        }
        data[n] = m;
        n++;
    }
    fclose(fp);

    printf("Valid records = %d, rejected = %d\n", n, rejected);
    for (int i = 0; i < n; i++)
    {
        printf("%4.1f s  %5.2f V  %5.2f A\n", data[i].time, data[i].voltage, data[i].
    ↪ current);
    }

    return 0;
}
```

```
Skipping invalid line: 0.5,5.01
Skipping invalid line: error,sensor,timeout
Valid records = 3, rejected = 2
 0.0 s   5.02 V   0.21 A
 1.0 s   4.98 V   0.24 A
 1.5 s   4.95 V   0.25 A
```

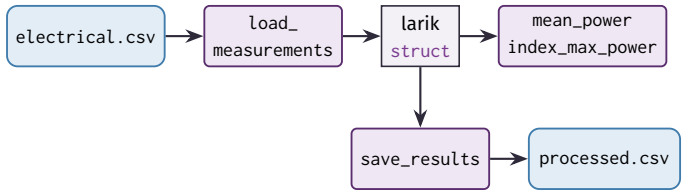
Program ini menerapkan beberapa pemeriksaan sekaligus: hasil `fopen`, keberadaan *header*, banyaknya kolom yang berhasil diurai, dan kapasitas larik.

Perlu dicatat bahwa **data yang berhasil diurai belum tentu valid**. Baris `1.0,-500.0,0.2` lolos dari `sscanf`, tetapi tegangan −500 V mungkin mustahil untuk rangkaian 5 V. Setelah parsing, data perlu divalidasi terhadap rentang fisis, misalnya tegangan dan arus tidak negatif serta waktu yang selalu bertambah.

7.6 Memisahkan masukan-keluaran dan analisis

Kode yang membuka berkas, mengurai baris, dan menghitung hasil di satu tempat sulit dibaca dan diuji. Prinsip modularisasi dari Kuliah 6 diterapkan dengan memisahkan

lapisan masukan-keluaran (I/O) dari **lapisan perhitungan**: fungsi pemuatan mengubah teks menjadi larik `struct`, fungsi analisis bekerja pada larik tanpa mengetahui asal datanya, dan fungsi penyimpanan menulis hasil.



Fungsi pemuatan berikut mengelola seluruh siklus buka–periksa–baca–tutup dan mengembalikan banyaknya rekaman valid, atau −1 jika berkas tidak dapat dibuka. Dengan antarmuka ini, jelas pula siapa yang bertanggung jawab menutup berkas.

```c
int load_measurements(const char filename[], ElectricalMeasurement data[], int max_data)
{
    FILE *fp = fopen(filename, "r");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    char line[256];
    if (fgets(line, sizeof(line), fp) == NULL)     /* header */
    {
        fclose(fp);
        return 0;
    }

    int n = 0;
    while (n < max_data && fgets(line, sizeof(line), fp) != NULL)
    {
        ElectricalMeasurement m;
        if (sscanf(line, "%lf,%lf,%lf", &m.time, &m.voltage, &m.current) != 3)
        {
            printf("Skipping invalid line: %s", line);
            continue;
        }
        data[n] = m;
        n++;
    }

    fclose(fp);
    return n;
}
```

Pemisahan ini juga memudahkan pengujian: fungsi analisis dapat diuji dengan larik yang dibuat langsung di dalam program, tanpa perlu menyiapkan berkas.

7.7 Studi kasus: pengolahan data listrik

Sekarang kita gabungkan `struct`, larik `struct`, pembacaan CSV, fungsi analisis, dan penulisan hasil. Dari berkas `electrical.csv` kita ingin menghitung daya setiap pengukuran, daya rata-rata, dan daya maksimum beserta waktunya, lalu menulis berkas hasil `processed.csv` dengan kolom tambahan `power`. Berkas data mentah tidak ditimpa; hasil pengolahan selalu ditulis ke berkas baru agar proses dapat diulang dan dibandingkan.

```c
#include <stdio.h>

#define MAX_DATA 1000

typedef struct
{
    double time;      /* s */
    double voltage;   /* V */
    double current;   /* A */
} ElectricalMeasurement;

int load_measurements(const char filename[], ElectricalMeasurement data[], int max_data);
double power(const ElectricalMeasurement *m);
double mean_power(const ElectricalMeasurement data[], int n);
int index_max_power(const ElectricalMeasurement data[], int n);
int save_results(const char filename[], const ElectricalMeasurement data[], int n);

int main(void)
{
    ElectricalMeasurement data[MAX_DATA];

    int n = load_measurements("electrical.csv", data, MAX_DATA);
    if (n <= 0)
    {
        printf("No valid data.\n");
        return 1;
    }

    int k = index_max_power(data, n);

    printf("Records       = %d\n", n);
    printf("Mean power    = %.4f W\n", mean_power(data, n));
    printf("Maximum power = %.4f W at t = %.1f s\n", power(&data[k]), data[k].time);

    if (save_results("processed.csv", data, n) != 0)
    {
        return 1;
    }
    printf("Results written to processed.csv\n");
```

```
    return 0;
}

int load_measurements(const char filename[], ElectricalMeasurement data[], int max_data)
{
    FILE *fp = fopen(filename, "r");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    char line[256];
    if (fgets(line, sizeof(line), fp) == NULL)
    {
        fclose(fp);
        return 0;
    }

    int n = 0;
    while (n < max_data && fgets(line, sizeof(line), fp) != NULL)
    {
        ElectricalMeasurement m;
        if (sscanf(line, "%lf,%lf,%lf", &m.time, &m.voltage, &m.current) != 3)
        {
            printf("Skipping invalid line: %s", line);
            continue;
        }
        data[n] = m;
        n++;
    }

    fclose(fp);
    return n;
}

double power(const ElectricalMeasurement *m)
{
    return m->voltage * m->current;
}

double mean_power(const ElectricalMeasurement data[], int n)
{
    double sum = 0.0;
    for (int i = 0; i < n; i++)
    {
        sum += power(&data[i]);
    }
    return sum / n;
```

```
}

int index_max_power(const ElectricalMeasurement data[], int n)
{
    int index = 0;
    for (int i = 1; i < n; i++)
    {
        if (power(&data[i]) > power(&data[index]))
        {
            index = i;
        }
    }
    return index;
}

int save_results(const char filename[], const ElectricalMeasurement data[], int n)
{
    FILE *fp = fopen(filename, "w");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    fprintf(fp, "time,voltage,current,power\n");
    for (int i = 0; i < n; i++)
    {
        fprintf(fp, "%.2f,%.3f,%.3f,%.5f\n",
                data[i].time, data[i].voltage, data[i].current, power(&data[i]));
    }

    if (fclose(fp) != 0)
    {
        perror(filename);
        return -1;
    }
    return 0;
}
```

```
Records       = 6
Mean power    = 1.1990 W
Maximum power = 1.3230 W at t = 2.5 s
Results written to processed.csv
```

Isi berkas hasil:

```
time,voltage,current,power
0.00,5.020,0.210,1.05420
0.50,5.010,0.220,1.10220
1.00,4.980,0.240,1.19520
```

```
1.50,4.950,0.250,1.23750
2.00,4.930,0.260,1.28180
2.50,4.900,0.270,1.32300
```

Fungsi `index_max_power` sengaja mengembalikan *indeks*, bukan nilai daya. Dengan indeks, kita memperoleh waktu, tegangan, arus, dan daya dari rekaman yang sama. Fungsi `power` juga digunakan ulang oleh tiga fungsi lain.

Hasilnya perlu diperiksa kewajarannya. Daya sekitar 1–1.3 W wajar untuk beban kecil pada catu 5 V. Tegangan yang sedikit menurun ketika arus naik konsisten dengan adanya hambatan dalam sumber. Format numerik `%.5f` pada kolom daya dipilih agar presisi hasil sebanding dengan presisi data masukan (tiga angka penting); menulis lebih banyak digit tidak menambah informasi.

7.7.1 Mode tambah untuk catatan pengukuran

Mode `"a"` menambahkan data di akhir berkas tanpa menghapus isi lama, sehingga cocok untuk catatan (*log*) pengukuran yang terus bertambah. Program berikut menambahkan satu baris setiap kali dijalankan.

```c
#include <stdio.h>
#include <string.h>

int main(void)
{
    char experiment[64];
    double value;

    printf("Experiment name: ");
    fgets(experiment, sizeof(experiment), stdin);
    experiment[strcspn(experiment, "\n")] = '\0';
    printf("Measured value: ");
    scanf("%lf", &value);

    FILE *fp = fopen("experiment.log", "a");
    if (fp == NULL)
    {
        perror("experiment.log");
        return 1;
    }
    fprintf(fp, "%s,%.4f\n", experiment, value);
    fclose(fp);

    return 0;
}
```

Program dijalankan dua kali:

```
Experiment name: kalibrasi-1
Measured value: 1.0012
```

```
Experiment name: kalibrasi-2
Measured value: 0.9987
```

Isi `experiment.log` setelah kedua eksekusi menunjukkan bahwa data lama tidak terhapus:

```
kalibrasi-1,1.0012
kalibrasi-2,0.9987
```

7.8 Kesalahan umum dan kebiasaan yang baik

- **Salah anggota atau tidak menginisialisasi anggota.** Gunakan *designated initializer* agar setiap anggota jelas nilainya.
- **Menyalin *string* dengan `=`.** Gunakan `strcpy` dan periksa kapasitas.
- **Tidak memeriksa `fopen`.** Menggunakan `NULL` sebagai `FILE *` menyebabkan program berhenti tiba-tiba.
- **Mode yang salah.** Membuka berkas data mentah dengan `"w"` menghapus isinya.
- **Lupa menutup berkas.** Data dapat tertahan di *buffer* dan tidak tertulis.
- **Mengabaikan hasil pembacaan** atau menggunakan `while (!feof(fp))`. Periksa nilai kembalian `fscanf`, `fgets`, dan `sscanf`.
- **Tidak memeriksa kapasitas larik** saat memuat data dari berkas yang ukurannya tidak diketahui.
- **Menganggap data valid karena berhasil diurai.** Validasi rentang fisis tetap diperlukan.

Kebiasaan yang dianjurkan: rancang skema berkas (kolom, urutan, satuan) sebelum menulis kode; jangan menimpa data mentah; pisahkan fungsi I/O dari fungsi analisis; laporkan baris yang ditolak; gunakan presisi keluaran yang sesuai dengan presisi data; dan periksa isi berkas hasil secara manual setidaknya sekali.

7.9 Cek pemahaman

- Apa keuntungan `struct` dibandingkan beberapa larik terpisah?
- Apa perbedaan operator `.` dan `->`?
- Apa fungsi `typedef` dan *designated initializer*?
- Mengapa anggota *string* tidak dapat diisi dengan `=`?
- Apa empat langkah pola operasi berkas? Apa perbedaan mode `"r"`, `"w"`, dan `"a"`?
- Mengapa hasil `fopen` harus diperiksa, dan apa kegunaan `perror`?
- Mengapa `while (!feof(fp))` bukan pola pembacaan yang baik?
- Mengapa membaca CSV dengan `fgets` lalu `sscanf` lebih andal daripada `fscanf` langsung?
- Mengapa data yang berhasil diurai masih perlu divalidasi?
- Apa keuntungan memisahkan fungsi I/O dari fungsi analisis?

7.10 Latihan

1. **`struct` gerak lurus.** Buat `struct MotionSample` dengan anggota waktu, posisi, dan kecepatan. Buat larik berisi sekurang-kurangnya lima sampel, tampilkan sebagai tabel, dan tambahkan fungsi yang menghitung energi kinetik satu sampel jika massa diberikan sebagai parameter.
2. **Eksperimen kalor.** Buat tipe `ThermalSample` (waktu, temperatur). Baca N data ke larik `struct`, lalu cari temperatur minimum, maksimum, dan waktu terjadinya maksimum dengan fungsi terpisah. Tolak $N \leq 0$ dan N yang melebihi kapasitas.
3. **Data pegas ke berkas.** Baca konstanta k, hitung $F = -kx$ untuk beberapa nilai x, dan tulis hasilnya ke `spring.csv` dengan kolom `x_m,force_N`. Periksa hasil `fopen` dan `fclose`, lalu buka berkas secara manual untuk memeriksa formatnya.
4. **Posisi dari CSV.** Baca `motion.csv` (kolom `time_s,position_m`) ke larik `MotionSample`, hitung kecepatan rata-rata setiap interval, dan tulis `velocity.csv` dengan kolom `time_start,time_end,velocity`. Tolak baris rusak dan selang waktu yang tidak positif.
5. **Energi listrik.** Dari `electrical.csv`, hitung energi $E \approx \sum_i P_i \Delta t_i$ dengan Δt_i diambil dari data, bukan diasumsikan tetap. Periksa terlebih dahulu bahwa waktu bertambah monoton.
6. **Intensitas relatif.** Baca berkas `time,intensity`, cari intensitas maksimum beserta waktunya, lalu tulis berkas baru dengan kolom tambahan `relative_intensity` = $I_i / I_{\max}$. Tangani kasus ketika maksimum tidak positif.
7. **Parser dengan baris rusak.** Siapkan berkas berisi baris valid dan tidak valid. Hitung banyaknya baris valid dan tidak valid secara terpisah, dan hanya simpan baris valid ke larik `struct`. Tambahkan validasi rentang fisis setelah parsing.
8. **Catatan eksperimen.** Kembangkan program mode tambah pada kuliah ini sehingga setiap baris juga memuat nomor urut pengukuran. Jalankan beberapa kali dan verifikasi bahwa data lama tidak terhapus. Apa yang terjadi jika mode diganti menjadi `"w"`?
9. **Metadata dan data.** Tulis `info.txt` (nama eksperimen, operator, satuan) dan `data.csv` (data numerik), lalu baca kembali `info.txt` untuk menampilkan metadata. Diskusikan keuntungan dan kelemahan memisahkan metadata dari data numerik.
10. **Proyek mini: *pipeline* data ayunan.** Rancang format CSV `time_s,angle_deg` untuk data pendulum. Buat program dengan sekurang-kurangnya tiga fungsi selain `main` yang membaca dan memvalidasi data, mencari sudut maksimum absolut, menghitung rata-rata sudut dan banyaknya data dengan $|\theta| > 10°$, lalu menulis ringkasan ke `summary.txt`. Jelaskan alasan pembagian fungsinya.

7.11 Rangkuman

- `struct` mengelompokkan data yang berkaitan menjadi satu rekaman; anggotanya diakses dengan `.` atau, melalui *pointer*, dengan `->`. `typedef` menyederhanakan nama tipe.

- Larik `struct` menyimpan banyak rekaman; fungsi dapat menerima `struct` (sebaiknya melalui *pointer* `const`) maupun mengembalikannya.
- Operasi berkas mengikuti pola buka–periksa–baca/tulis–tutup. Mode `"r"` membaca, `"w"` menulis ulang, dan `"a"` menambahkan.
- Hasil `fopen` harus diperiksa; `perror` menjelaskan penyebab kegagalan. Pembacaan diulang selama fungsi baca berhasil, bukan dengan `while (!feof(fp))`.
- CSV dibaca baris demi baris dengan `fgets`, diurai dengan `sscanf`, dan setiap baris valid dipetakan ke satu `struct`. Data yang berhasil diurai tetap perlu divalidasi.
- Memisahkan fungsi I/O dari fungsi analisis membuat program lebih mudah dibaca, diuji, dan digunakan kembali. Data mentah tidak ditimpa; hasil ditulis ke berkas baru.
- Kuliah berikutnya mengintegrasikan semua konsep C yang telah dipelajari dalam satu program utuh untuk kasus Teknik Fisika, lengkap dengan pengujian dan pemeriksaan hasil.

KULIAH

8

Integrasi Pemrograman C untuk Kasus Teknik Fisika

Kuliah-kuliah sebelumnya memperkenalkan konsep C satu per satu: variabel dan ekspresi, percabangan dan pengulangan, pola iteratif, larik dan fungsi, `struct`, serta pengolahan berkas. Dalam praktik, konsep-konsep tersebut tidak muncul sendiri-sendiri. Program analisis data eksperimen harus membaca berkas, memvalidasi data, menyimpan rekaman, menghitung besaran fisis, mencari kondisi tertentu, menulis hasil, *dan* meyakinkan kita bahwa hasilnya benar.

Kuliah ini membangun *satu* program utuh untuk satu persoalan, dari perumusan masalah sampai pengujian. Tujuannya bukan memperkenalkan sintaks baru, melainkan melatih cara menyusun program yang dapat dipercaya. Alurnya mengikuti kerangka yang sudah kita kenal sejak Kuliah 1, kini dalam bentuk yang lebih lengkap:

$$\text{masalah} \rightarrow \text{model} \rightarrow \text{rancangan} \rightarrow \\ \text{implementasi bertahap} \rightarrow \text{pengujian} \rightarrow \text{pemeriksaan fisis}$$

8.1 Persoalan: eksperimen gerak satu dimensi

Sebuah sensor posisi merekam gerak benda di sepanjang lintasan lurus. Hasilnya adalah berkas `Data/motion.csv` dengan dua kolom, waktu (s) dan posisi (m):

```
time_s,position_m
0.00,0.400000
0.50,1.087500
1.00,1.950000
1.50,2.987500
2.00,4.200000
2.50,5.587500
3.00,7.150000
3.50,8.887500
4.00,10.800000
4.50,12.887500
5.00,15.150000
```

Dari data tersebut kita ingin mengetahui bagaimana kecepatan benda berubah, berapa kecepatan rata-ratanya, kapan kecepatan terbesar terjadi, dan kapan kecepatan pertama kali melampaui suatu ambang, misalnya 3.0 m/s.

8.1.1 Model dan spesifikasi

Data terdiri atas pasangan (t_i, x_i) dengan $i = 0, 1, \ldots, N-1$. Kecepatan rata-rata pada interval antara dua sampel berurutan adalah

$$v_i = \frac{x_{i+1} - x_i}{t_{i+1} - t_i}, \qquad i = 0, 1, \ldots, N-2.$$

Dari N sampel diperoleh $N-1$ interval; hubungan ini langsung menentukan ukuran larik kecepatan dan batas pengulangan. Rumus tersebut hanya bermakna jika $t_{i+1} > t_i$, sehingga **waktu harus naik secara ketat**; dua waktu yang sama akan menghasilkan pembagian dengan nol, dan waktu yang mundur tidak bermakna fisis.

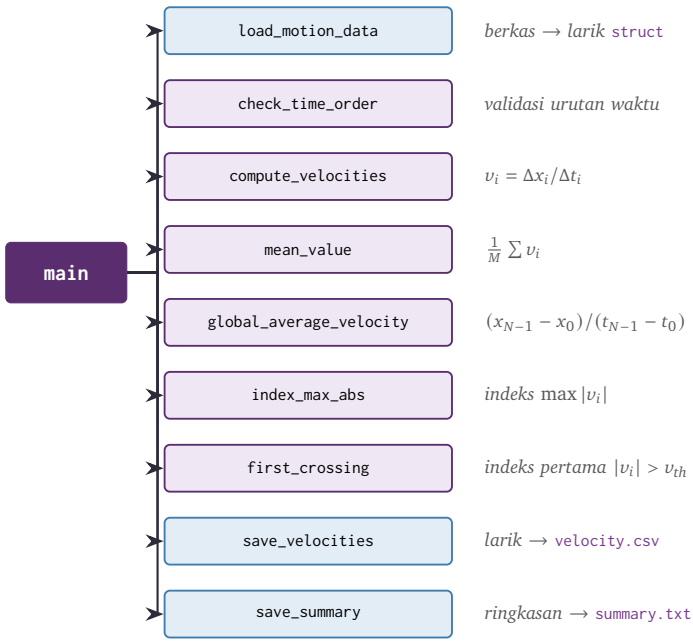
Spesifikasi program adalah sebagai berikut.

Masukan	berkas CSV `time_s,position_m`; ambang kecepatan v_{th}
Prakondisi	minimal dua sampel valid; waktu naik secara ketat
Keluaran	banyaknya sampel; kecepatan setiap interval; rata-rata kecepatan interval; kecepatan rata-rata global; kecepatan maksimum absolut beserta intervalnya; interval pertama dengan $\lvert v_i \rvert > v_{\text{th}}$
Berkas hasil	`velocity.csv` (kecepatan per interval) dan `summary.txt` (ringkasan)

8.2 Rancangan program

8.2.1 Dekomposisi dan tanggung jawab fungsi

Persoalan dipecah menjadi fungsi-fungsi dengan satu tanggung jawab masing-masing. Fungsi `main` hanya mengatur alur: memanggil fungsi, memeriksa hasilnya, dan menghentikan program jika ada kesalahan.



Fungsi berwarna biru muda bekerja dengan berkas (lapisan I/O); fungsi berwarna ungu muda hanya bekerja pada data di memori (lapisan analisis). Dengan pemisahan ini, jika kecepatan yang dihitung salah, kita cukup memeriksa `compute_velocities` tanpa menelusuri kode pembacaan berkas.

8.2.2 Struktur data, kapasitas, dan prakondisi

Satu baris CSV dipetakan ke satu rekaman:

```c
#include <stdio.h>
#include <math.h>

#define MAX_SAMPLES 1000

typedef struct
{
    double time;       /* s */
    double position;   /* m */
} MotionSample;
```

Kapasitas `MAX_SAMPLES` ditetapkan sebagai konstanta dan diperiksa setiap kali data ditambahkan, sehingga berkas yang lebih panjang dari dugaan tidak menyebabkan akses di luar batas larik.

Setiap fungsi mempunyai **prakondisi** (apa yang harus benar sebelum dipanggil) dan **pascakondisi** (apa yang dijamin setelahnya), seperti dibahas pada Kuliah 2. Misalnya, `compute_velocities` mensyaratkan $n \geq 2$ dan waktu yang naik secara ketat, dan menjamin larik kecepatan berisi tepat $n-1$ nilai. Prakondisi dituliskan sebagai komentar di atas fungsi,

dan `main` bertanggung jawab memastikan prakondisi terpenuhi sebelum memanggilnya. Parameter larik yang hanya dibaca diberi `const`, sehingga aliran data terlihat dari prototipe: data masuk melalui parameter `const`, hasil keluar melalui nilai kembalian atau larik tanpa `const`.

8.3 Implementasi bertahap: membaca dan memvalidasi data

Program sebesar ini sebaiknya tidak ditulis sekaligus lalu dikompilasi di akhir. Cara yang jauh lebih aman adalah membangunnya **bertahap**: tulis satu fungsi, kompilasi, uji dengan data kecil, baru lanjut ke fungsi berikutnya. Jika muncul kesalahan, penyebabnya hampir pasti berada pada bagian yang baru ditambahkan.

Tahap pertama adalah membaca data. Fungsi berikut mengikuti pola Kuliah 7, dengan satu tambahan penting: **pesan kesalahan yang informatif**. Pesan “Invalid line” saja menyulitkan pengguna mencari kesalahan dalam berkas berisi ribuan baris; menyebutkan nama berkas dan nomor baris membuat data bermasalah dapat dilacak.

```c
/* Membaca CSV time_s,position_m ke larik data.
   Mengembalikan banyaknya sampel valid, atau -1 jika berkas gagal dibuka. */
int load_motion_data(const char filename[], MotionSample data[], int max_samples)
{
    FILE *fp = fopen(filename, "r");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    char line[256];
    int line_number = 1;
    int n = 0;

    if (fgets(line, sizeof(line), fp) == NULL)          /* header */
    {
        fclose(fp);
        return 0;
    }

    while (fgets(line, sizeof(line), fp) != NULL)
    {
        line_number++;
        MotionSample s;

        if (sscanf(line, "%lf,%lf", &s.time, &s.position) != 2)
        {
            printf("%s:%d: invalid line ignored\n", filename, line_number);
            continue;
        }
        if (n >= max_samples)
        {
```

```
            printf("%s:%d: capacity of %d samples exceeded\n",
                   filename, line_number, max_samples);
            break;
        }
        data[n] = s;
        n++;
    }

    fclose(fp);
    return n;
}
```

Tahap kedua memeriksa prakondisi fisis: waktu harus naik secara ketat. Fungsi ini sengaja *hanya memeriksa*, tidak mengubah data. Memisahkan validasi dari manipulasi data membuat program lebih mudah dipahami; jika validasi sekaligus "memperbaiki" data secara diam-diam, kita kehilangan jejak tentang apa yang sebenarnya diukur.

```
/* Mengembalikan -1 jika waktu naik secara ketat,
   atau indeks i pertama dengan t[i+1] <= t[i]. */
int check_time_order(const MotionSample data[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        if (data[i + 1].time <= data[i].time)
        {
            return i;
        }
    }
    return -1;
}
```

Fungsi mengembalikan *indeks* masalah pertama, bukan sekadar "benar/salah", sehingga pesan kesalahan dapat menunjukkan lokasinya. Pada tahap ini kita sudah dapat menulis `main` sementara yang hanya memuat data, menampilkan banyaknya sampel, dan memeriksa urutan waktu—lalu mengujinya sebelum melanjutkan.

8.4 Implementasi bertahap: analisis

8.4.1 Kecepatan setiap interval

```
/* Prakondisi: n >= 2 dan waktu naik secara ketat.
   Pascakondisi: velocity[0..n-2] berisi kecepatan setiap interval. */
void compute_velocities(const MotionSample data[], int n, double velocity[])
{
    for (int i = 0; i < n - 1; i++)
    {
        double dx = data[i + 1].position - data[i].position;
        double dt = data[i + 1].time - data[i].time;
        velocity[i] = dx / dt;
    }
```

```
}
```

8.4.2 Dua jenis kecepatan rata-rata

Ada dua besaran yang sama-sama dapat disebut “kecepatan rata-rata”, dan keduanya tidak selalu sama. **Rata-rata aritmetika** kecepatan interval memberi bobot yang sama pada setiap interval,

$$\bar{v} = \frac{1}{M} \sum_{i=0}^{M-1} v_i, \qquad M = N - 1,$$

sedangkan **kecepatan rata-rata global** menghubungkan perpindahan total dengan waktu total,

$$v_{\text{global}} = \frac{x_{N-1} - x_0}{t_{N-1} - t_0}.$$

Kecepatan rata-rata global adalah besaran yang dimaksud dalam fisika: ia setara dengan rata-rata v_i yang dibobot oleh lamanya interval, $\sum v_i \Delta t_i / \sum \Delta t_i$. Jika semua Δt_i sama, keduanya identik; jika selang waktu tidak seragam, keduanya dapat berbeda. Kita menghitung keduanya karena perbandingannya juga berguna sebagai pemeriksaan konsistensi.

```c
/* Prakondisi: m > 0. */
double mean_value(const double values[], int m)
{
    double sum = 0.0;
    for (int i = 0; i < m; i++)
    {
        sum += values[i];
    }
    return sum / m;
}

/* Prakondisi: n >= 2 dan waktu naik secara ketat. */
double global_average_velocity(const MotionSample data[], int n)
{
    double dx = data[n - 1].position - data[0].position;
    double dt = data[n - 1].time - data[0].time;
    return dx / dt;
}
```

8.4.3 Pencarian nilai ekstrem dan kondisi

Fungsi pencarian mengembalikan *indeks*, bukan nilai. Dengan indeks, kita dapat memperoleh waktu awal, waktu akhir, posisi, dan kecepatan dari interval yang sama. Nilai -1 digunakan sebagai penanda “tidak ditemukan”, karena tidak ada indeks larik yang bernilai negatif.

```c
/* Prakondisi: m > 0. Indeks elemen dengan nilai absolut terbesar. */
int index_max_abs(const double values[], int m)
{
    int index = 0;
```

```
    for (int i = 1; i < m; i++)
    {
        if (fabs(values[i]) > fabs(values[index]))
        {
            index = i;
        }
    }
    return index;
}

/* Indeks pertama dengan |values[i]| > threshold, atau -1 jika tidak ada. */
int first_crossing(const double values[], int m, double threshold)
{
    for (int i = 0; i < m; i++)
    {
        if (fabs(values[i]) > threshold)
        {
            return i;
        }
    }
    return -1;
}
```

Fungsi `first_crossing` berhenti segera setelah kondisi ditemukan; tidak perlu memeriksa sisa data.

8.5 Program utuh

8.5.1 Menulis hasil

Hasil ditulis ke dua berkas baru. Data mentah tidak pernah ditimpa, dan berkas hasil diberi *header* yang menyertakan satuan.

```
int save_velocities(const char filename[], const MotionSample data[],
                    const double velocity[], int n)
{
    FILE *fp = fopen(filename, "w");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    fprintf(fp, "time_start_s,time_end_s,velocity_m_per_s\n");
    for (int i = 0; i < n - 1; i++)
    {
        fprintf(fp, "%.2f,%.2f,%.4f\n", data[i].time, data[i + 1].time, velocity[i]);
    }

    return (fclose(fp) == 0) ? 0 : -1;
```

```
}

int save_summary(const char filename[], const char source[], int n,
                 double v_mean, double v_global, double v_max, double t_max)
{
    FILE *fp = fopen(filename, "w");
    if (fp == NULL)
    {
        perror(filename);
        return -1;
    }

    fprintf(fp, "source file             : %s\n", source);
    fprintf(fp, "samples                 : %d\n", n);
    fprintf(fp, "mean interval velocity  : %.4f m/s\n", v_mean);
    fprintf(fp, "global average velocity : %.4f m/s\n", v_global);
    fprintf(fp, "maximum |v|             : %.4f m/s (interval starting at %.2f s)\n",
            v_max, t_max);

    return (fclose(fp) == 0) ? 0 : -1;
}
```

Ringkasan menyertakan nama berkas sumber. Kebiasaan kecil ini menjaga **keterlacakan** (*traceability*): setiap hasil dapat ditelusuri kembali ke data asalnya.

8.5.2 Fungsi `main` sebagai pengatur alur

Fungsi `main` menyusun semua fungsi di atas. Perhatikan bahwa setiap tahap diperiksa hasilnya, dan program berhenti dengan pesan yang jelas segera setelah prakondisi tahap berikutnya tidak terpenuhi—menghitung kecepatan dari data dengan waktu yang mundur hanya akan menghasilkan angka yang menyesatkan.

```
int main(void)
{
    const char *input_file = "Data/motion.csv";
    const double threshold = 3.0;              /* m/s */

    MotionSample data[MAX_SAMPLES];
    double velocity[MAX_SAMPLES];

    int n = load_motion_data(input_file, data, MAX_SAMPLES);
    if (n < 2)
    {
        printf("Need at least two valid samples (found %d).\n", n);
        return 1;
    }

    int bad = check_time_order(data, n);
    if (bad >= 0)
    {
```

```
        printf("Time does not increase between samples %d and %d.\n", bad, bad + 1);
        return 1;
    }

    compute_velocities(data, n, velocity);
    int m = n - 1;

    double v_mean = mean_value(velocity, m);
    double v_global = global_average_velocity(data, n);
    int k_max = index_max_abs(velocity, m);
    int k_cross = first_crossing(velocity, m, threshold);

    printf("Samples                : %d (%d intervals)\n", n, m);
    printf("Mean interval velocity  : %.4f m/s\n", v_mean);
    printf("Global average velocity : %.4f m/s\n", v_global);
    printf("Maximum |v|             : %.4f m/s in [%.2f, %.2f] s\n",
           fabs(velocity[k_max]), data[k_max].time, data[k_max + 1].time);
    if (k_cross >= 0)
    {
        printf("|v| > %.1f m/s first in  : [%.2f, %.2f] s\n",
               threshold, data[k_cross].time, data[k_cross + 1].time);
    }
    else
    {
        printf("|v| never exceeds %.1f m/s\n", threshold);
    }

    if (save_velocities("velocity.csv", data, velocity, n) != 0 ||
        save_summary("summary.txt", input_file, n, v_mean, v_global,
                     fabs(velocity[k_max]), data[k_max].time) != 0)
    {
        return 1;
    }

    return 0;
}
```

Program lengkap terdiri atas definisi tipe dan semua fungsi pada subbab-subbab sebelumnya, diikuti `main` ini. Setelah dikompilasi dan dijalankan:

```
Samples                : 11 (10 intervals)
Mean interval velocity  : 2.9500 m/s
Global average velocity : 2.9500 m/s
Maximum |v|             : 4.5250 m/s in [4.50, 5.00] s
|v| > 3.0 m/s first in  : [2.50, 3.00] s
```

Isi `velocity.csv`:

```
time_start_s,time_end_s,velocity_m_per_s
0.00,0.50,1.3750
```

```
0.50,1.00,1.7250
1.00,1.50,2.0750
1.50,2.00,2.4250
2.00,2.50,2.7750
2.50,3.00,3.1250
3.00,3.50,3.4750
3.50,4.00,3.8250
4.00,4.50,4.1750
4.50,5.00,4.5250
```

Isi `summary.txt`:

```
source file             : Data/motion.csv
samples                 : 11
mean interval velocity  : 2.9500 m/s
global average velocity : 2.9500 m/s
maximum |v|             : 4.5250 m/s (interval starting at 4.50 s)
```

8.6 Pengujian program

Keluaran di atas tampak masuk akal, tetapi bagaimana kita *tahu* bahwa program benar? Mencoba satu berkas data nyata tidak cukup, karena kita tidak mengetahui jawaban yang tepat untuk data tersebut. Strategi yang andal adalah menguji setiap fungsi dengan **data sintetis** yang jawabannya dapat diturunkan secara analitik, termasuk **kasus batas**.

Kasus uji	Hasil yang diharapkan
posisi konstan, $x_i = 2$	semua $v_i = 0$
kecepatan konstan, $x_i = 1 + 3t_i$	semua $v_i = 3$; $\bar{v} = v_{\text{global}} = 3$
percepatan konstan, $x = \frac{1}{2}at^2$	$v_i = a\,\bar{t}_i$ dengan $\bar{t}_i$ titik tengah interval
gerak mundur, $x_i = -2t_i$	$v_i = -2$; maksimum absolut 2, bukan -2
waktu tidak naik	`check_time_order` menunjuk interval yang salah
Δt tidak seragam	$\bar{v} \neq v_{\text{global}}$ secara umum
ambang tidak pernah terlampaui	`first_crossing` mengembalikan -1

Kasus percepatan konstan sangat informatif. Untuk $x(t) = \frac{1}{2}at^2$,

$$v_i = \frac{\frac{1}{2}a(t_{i+1}^2 - t_i^2)}{t_{i+1} - t_i} = \frac{1}{2}a(t_{i+1} + t_i) = a\,\bar{t}_i,$$

sehingga kecepatan interval *tepat* sama dengan kecepatan sesaat di titik tengah interval. Ini memberikan jawaban eksak yang dapat dibandingkan dengan hasil program.

Pengujian dapat diotomatiskan dengan program uji kecil. Fungsi `check` berikut mencetak `PASS` atau `FAIL`; hasil *floating-point* dibandingkan dengan toleransi, bukan kesamaan eksak.

```c
int failures = 0;

void check(const char *name, int condition)
```

```
{
    printf("[%s] %s\n", condition ? "PASS" : "FAIL", name);
    if (!condition)
    {
        failures++;
    }
}

int close_to(double a, double b)
{
    return fabs(a - b) < 1e-9;
}

int main(void)
{
    double v[10];

    MotionSample still[] = {{0.0, 2.0}, {1.0, 2.0}, {2.0, 2.0}};
    compute_velocities(still, 3, v);
    check("constant position gives v = 0", close_to(v[0], 0.0) && close_to(v[1], 0.0));

    MotionSample uniform[] = {{0.0, 1.0}, {0.5, 2.5}, {1.5, 5.5}, {2.0, 7.0}};
    compute_velocities(uniform, 4, v);
    check("constant velocity gives v = 3",
          close_to(v[0], 3.0) && close_to(v[1], 3.0) && close_to(v[2], 3.0));
    check("global velocity = 3", close_to(global_average_velocity(uniform, 4), 3.0));

    MotionSample accel[] = {{0.0, 0.0}, {1.0, 1.0}, {2.0, 4.0}, {3.0, 9.0}};  /* a = 2 */
    compute_velocities(accel, 4, v);
    check("constant acceleration: v = a * t_mid",
          close_to(v[0], 1.0) && close_to(v[1], 3.0) && close_to(v[2], 5.0));

    MotionSample backward[] = {{0.0, 0.0}, {1.0, -2.0}, {2.0, -4.0}};
    compute_velocities(backward, 3, v);
    check("maximum |v| keeps sign information", close_to(fabs(v[index_max_abs(v, 2)]),
    ↪ 2.0));

    MotionSample bad_time[] = {{0.0, 0.0}, {1.0, 1.0}, {1.0, 2.0}, {2.0, 3.0}};
    check("repeated time is detected at interval 1", check_time_order(bad_time, 4) == 1);

    MotionSample uneven[] = {{0.0, 0.0}, {1.0, 1.0}, {4.0, 10.0}};
    compute_velocities(uneven, 3, v);
    check("uneven dt: mean != global",
          !close_to(mean_value(v, 2), global_average_velocity(uneven, 3)));

    check("threshold never crossed returns -1", first_crossing(v, 2, 100.0) == -1);

    printf("%d test(s) failed\n", failures);
    return failures == 0 ? 0 : 1;
}
```

```
[PASS] constant position gives v = 0
[PASS] constant velocity gives v = 3
[PASS] global velocity = 3
[PASS] constant acceleration: v = a * t_mid
[PASS] maximum |v| keeps sign information
[PASS] repeated time is detected at interval 1
[PASS] uneven dt: mean != global
[PASS] threshold never crossed returns -1
0 test(s) failed
```

Semua uji lolos. Kasus Δt tidak seragam memperlihatkan perbedaan dua jenis kecepatan rata-rata: interval pertama ($v_0 = 1$ m/s, $\Delta t = 1$ s) dan kedua ($v_1 = 3$ m/s, $\Delta t = 3$ s) memberi $\bar{v} = 2$ m/s, sedangkan $v_{\text{global}} = 10/4 = 2.5$ m/s.

Selain fungsi analisis, fungsi I/O juga perlu diuji dengan berkas kecil buatan sendiri: berkas yang tidak ada, berkas kosong, berkas hanya berisi *header*, berkas dengan baris rusak, dan berkas dengan satu sampel saja. Untuk setiap kasus, tentukan dahulu perilaku yang diharapkan, lalu periksa apakah program berperilaku demikian. Misalnya, berkas dengan satu sampel harus menghasilkan pesan yang jelas, bukan pembagian dengan nol.

8.7 Memeriksa hasil secara fisis

Lolos uji belum menjamin hasil bermakna untuk data nyata. Pemeriksaan fisis merupakan lapisan pertahanan terakhir.

- **Dimensi.** $[v] = [\Delta x]/[\Delta t] = \text{m/s}$. Jika kolom waktu ternyata dalam milidetik, semua kecepatan menjadi seribu kali terlalu kecil.
- **Orde besaran.** Kecepatan beberapa meter per sekon wajar untuk troli di lintasan laboratorium; nilai ribuan meter per sekon pasti menandakan kesalahan.
- **Tanda.** Kecepatan negatif berarti benda bergerak ke arah x negatif. Itulah sebabnya pencarian maksimum menggunakan nilai absolut.
- **Konsistensi internal.** Jumlah perpindahan setiap interval harus sama dengan perpindahan total, $\sum v_i \Delta t_i = x_{N-1} - x_0$. Untuk data kita, $15.15 - 0.40 = 14.75$ m.
- **Kesesuaian dengan model.** Kecepatan pada `velocity.csv` naik sebesar 0.35 m/s setiap 0.5 s, sehingga percepatannya konstan, $a = 0.35/0.5 = 0.7$ m/s^2. Dengan hasil pengujian di atas, $v_i = v_0 + a\bar{t}_i$; interval pertama ($\bar{t} = 0.25$ s, $v = 1.375$ m/s) memberikan $v_0 = 1.375 - 0.7(0.25) = 1.2$ m/s. Data sesuai dengan model $x(t) = 0.4 + 1.2t + 0.35t^2$.

Data eksperimen nyata selalu mengandung **derau** (*noise*). Karena kecepatan dihitung dari *selisih* posisi, derau posisi diperbesar: galat posisi δx menghasilkan galat kecepatan sekitar $\delta x/\Delta t$. Dengan $\delta x = 1$ mm dan $\Delta t = 0.5$ s, galat kecepatan hanya 2 mm/s; tetapi dengan $\Delta t = 1$ ms galatnya menjadi 1 m/s. Data `motion.csv` di atas dibangkitkan dari model tanpa derau, sehingga pola kecepatannya sangat teratur; pada data nyata, kecepatan akan berfluktuasi di sekitar garis lurus.

Ketika hasil tampak janggal, pisahkan dua kemungkinan penyebab: **kesalahan data** (sensor bermasalah, satuan salah, baris rusak) atau **kesalahan program**. Program yang telah lolos uji dengan data sintetis memperkecil kemungkinan kedua, sehingga perhatian dapat dipusatkan pada data. Untuk itu, hasil juga perlu **dapat direproduksi**: catat berkas masukan, parameter (seperti ambang kecepatan), dan versi program, sehingga orang lain—atau kita sendiri beberapa bulan kemudian—memperoleh hasil yang sama dari data yang sama.

8.8 Pola umum dan kesalahan umum

8.8.1 Pola yang dapat digunakan kembali

Struktur program studi kasus ini berlaku untuk banyak persoalan Teknik Fisika lain; yang berubah hanyalah `struct`, model, dan fungsi analisisnya.

Kasus	Rekaman	Analisis khas
gerak lurus	(t, x)	$v_i = \Delta x/\Delta t$, maksimum, ambang
rangkaian listrik	(t, V, I)	$P = VI$, energi $\sum P\Delta t$
pendinginan benda	(t, T)	laju $\Delta T/\Delta t$, waktu mencapai ambang
intensitas cahaya	(x, I)	puncak, lebar pada setengah maksimum
osilasi	(t, y)	amplitudo, RMS, perpotongan nol

Dari sisi efisiensi, setiap fungsi analisis menelusuri data satu kali, sehingga keseluruhan program berkompleksitas waktu $O(N)$ dengan memori $O(N)$ untuk larik. Beberapa pengulangan sebenarnya dapat digabung menjadi satu, atau data dapat diproses secara *streaming* seperti Kuliah 5 dengan memori $O(1)$. Namun, untuk data berukuran ribuan sampel, perbedaan kinerjanya tidak berarti, sedangkan fungsi-fungsi kecil yang terpisah jauh lebih mudah dipahami dan diuji. Penggabungan baru layak dipertimbangkan jika data sangat besar dan pengukuran menunjukkan bahwa program memang terlalu lambat.

8.8.2 Kesalahan umum pada program terintegrasi

- **Menulis seluruh program sekaligus**, lalu menghadapi puluhan pesan galat tanpa tahu dari mana harus mulai.
- **Mencampur I/O dan perhitungan di semua tempat**, sehingga fungsi analisis tidak dapat diuji tanpa berkas.
- **Mengabaikan prakondisi**, misalnya memanggil `compute_velocities` sebelum memeriksa urutan waktu.
- **Menganggap data valid karena berhasil dibaca.** Parsing yang berhasil hanya menjamin format, bukan makna fisis.
- **Menggunakan indeks tanpa memeriksa batas**, misalnya memakai hasil `first_crossing` tanpa memeriksa nilai -1.
- **Menimpa data mentah** dengan hasil pengolahan.
- **Tidak memeriksa hasil secara fisis**, sehingga kesalahan satuan atau tanda lolos ke laporan.

8.9 Cek pemahaman

- Mengapa N sampel posisi hanya menghasilkan $N - 1$ kecepatan interval?
- Mengapa waktu harus naik secara ketat sebelum kecepatan dihitung?
- Apa keuntungan memisahkan lapisan I/O dari lapisan analisis?
- Apa peran prakondisi dan pascakondisi dalam merancang fungsi?
- Mengapa fungsi pencarian lebih baik mengembalikan indeks daripada nilai?
- Kapan rata-rata aritmetika kecepatan interval berbeda dari kecepatan rata-rata global? Mana yang bermakna fisis?
- Mengapa program sebaiknya dibangun dan diuji secara bertahap?
- Mengapa data sintetis berguna untuk pengujian? Berikan tiga contoh kasus uji beserta hasil yang diharapkan.
- Bagaimana derau posisi memengaruhi kecepatan yang dihitung dari selisih?
- Apa yang dimaksud dengan keterlacakan dan reprodusibilitas hasil?

8.10 Latihan

1. **Program gerak lengkap.** Ketik ulang dan kompilasi program studi kasus kuliah ini. Tambahkan kolom percepatan antarinterval $a_i = (v_{i+1} - v_i)/(\bar{t}_{i+1} - \bar{t}_i)$ ke berkas hasil dan periksa bahwa hasilnya mendekati 0.7 m/s^2 untuk `Data/motion.csv`.
2. **Energi rangkaian listrik.** Baca `electrical.csv` (`time_s,voltage_V,current_A`), hitung $P_i = V_i I_i$, lalu perkirakan energi dengan aturan trapesium $E \approx \sum_i \frac{1}{2}(P_i + P_{i+1})\Delta t_i$. Periksa urutan waktu, cari daya maksimum, dan bandingkan orde energi dengan perkalian daya rata-rata dan lama eksperimen.
3. **Pendinginan benda.** Baca `Data/cooling.csv` (`time_min,temperature_C`). Hitung laju perubahan temperatur setiap interval dan cari saat pertama temperatur turun di bawah ambang yang dimasukkan pengguna. Periksa bahwa tanda laju sesuai dengan proses pendinginan.
4. **Intensitas cahaya.** Baca data posisi dan intensitas, cari intensitas maksimum dan posisinya, lalu cari titik pertama di kanan puncak tempat intensitas turun di bawah 50% maksimum. Gunakan fungsi pencarian yang mengembalikan indeks dan tulis berkas hasil dengan kolom `relative_intensity`.
5. **Osilasi pegas.** Untuk data `time_s,displacement_m`, hitung simpangan maksimum absolut A, RMS simpangan, banyaknya sampel dengan $|x| > 0.8A$, dan banyaknya perubahan tanda berurutan sebagai indikasi penyeberangan titik setimbang.
6. **Kualitas data sensor.** Baca berkas `time_s,value` dan tandai baris yang waktunya tidak naik, nilainya di luar rentang yang ditentukan pengguna, atau lonjakannya terhadap sampel sebelumnya melebihi ambang. Tulis data valid ke `clean.csv` dan catatan masalah (dengan nomor baris) ke `rejected.txt`. Jelaskan pemisahan antara validasi format, urutan waktu, dan validasi fisis.

7. **Jatuh bebas.** Dari data waktu dan posisi vertikal, hitung kecepatan interval dan percepatan antarinterval. Bandingkan orde percepatannya dengan $g \approx 9.8\ \mathrm{m/s^2}$ sebagai pemeriksaan kewajaran, tanpa melakukan *fitting* kurva.
8. **Gelombang sintetis.** Tulis program yang membangkitkan $y(t) = A\sin(2\pi f t)$ ke `wave.csv`, membacanya kembali, dan menghitung minimum, maksimum, rata-rata, serta RMS. Bandingkan RMS dengan $A/\sqrt{2}$. Gunakan fungsi terpisah untuk pembangkitan, pembacaan, dan analisis.
9. **Program uji.** Tambahkan sekurang-kurangnya tiga kasus uji baru ke program pengujian kuliah ini, termasuk satu kasus yang menguji fungsi `mean_value` dengan larik satu elemen dan satu kasus gerak dengan kecepatan negatif yang berubah tanda. Pastikan setiap uji gagal jika fungsi yang diuji sengaja dirusak.
10. **Proyek mini integrasi C.** Pilih satu kasus dari mekanika, listrik, termal, optika, atau gelombang. Rancang program dengan alur masalah → model → struktur data → pembacaan CSV → validasi → analisis → pencarian kondisi → penulisan hasil → pemeriksaan fisis. Gunakan sekurang-kurangnya satu `struct`, lima fungsi selain `main`, dan satu program uji dengan data sintetis.

8.11 Rangkuman

- Program yang dapat dipercaya dibangun mengikuti alur masalah, model, rancangan, implementasi bertahap, pengujian, dan pemeriksaan fisis.
- Spesifikasi yang jelas—masukan, prakondisi, keluaran, dan berkas hasil—ditulis sebelum kode.
- Dekomposisi menghasilkan fungsi dengan satu tanggung jawab. Lapisan I/O dipisahkan dari lapisan analisis, dan `main` hanya mengatur alur serta memeriksa hasil setiap tahap.
- Validasi dilakukan sebelum perhitungan dan dipisahkan dari manipulasi data. Pesan kesalahan menyebutkan lokasi masalah.
- Fungsi pencarian mengembalikan indeks, dengan -1 sebagai penanda “tidak ditemukan”.
- Rata-rata aritmetika kecepatan interval dan kecepatan rata-rata global hanya sama jika selang waktunya seragam; definisi statistik harus sesuai dengan makna fisis.
- Pengujian dengan data sintetis dan kasus batas memberikan jawaban eksak untuk dibandingkan. Pemeriksaan dimensi, orde besaran, tanda, dan konsistensi internal menjadi lapisan pertahanan terakhir.
- Hasil yang baik dapat dilacak ke data asalnya dan dapat direproduksi.
- Kuliah berikutnya memperkenalkan Python, yang memungkinkan analisis dan visualisasi data yang sama dengan kode yang jauh lebih ringkas.

KULIAH

9

Python untuk Komputasi Ilmiah

9.1 Mengapa Python?

Delapan kuliah sebelumnya menggunakan C. Melalui C kita belajar bagaimana algoritma diterjemahkan menjadi instruksi yang dekat dengan mesin: tipe data harus dideklarasikan, ukuran larik harus dikelola sendiri, berkas dibuka dan ditutup secara eksplisit, dan setiap pengulangan ditulis lengkap. Pemahaman tersebut tetap berharga. Akan tetapi, untuk analisis data eksperimen dan visualisasi, menulis semuanya dalam C memakan waktu yang tidak sedikit.

Python adalah bahasa tingkat tinggi yang banyak digunakan dalam sains dan rekayasa. Program Python tidak perlu dikompilasi terlebih dahulu; programnya dijalankan oleh *interpreter*. Kekuatan utamanya untuk komputasi ilmiah terletak pada ekosistem pustakanya:

- **NumPy** untuk larik numerik dan operasi vektor yang cepat;
- **Matplotlib** untuk grafik ilmiah;
- SciPy, pandas, dan banyak pustaka lain untuk metode numerik, statistik, dan pengolahan data tabel.

Pergantian bahasa tidak berarti pergantian cara berpikir. Masalah, model, algoritma, validasi, dan pemeriksaan fisis tetap sama; yang berubah hanyalah cara mengekspresikannya. Kuliah ini memperkenalkan Python melalui perbandingan langsung dengan C, lalu menggunakannya untuk menganalisis dan memvisualisasikan data eksperimen.

9.1.1 Menyiapkan lingkungan

Pada Linux atau WSL, Python biasanya sudah tersedia. Pustaka NumPy dan Matplotlib sebaiknya dipasang dalam **lingkungan virtual** (*virtual environment*) agar tidak bercampur dengan pustaka sistem:

```
python3 -m venv ~/venv-ilmiah
source ~/venv-ilmiah/bin/activate
```

```
pip install numpy matplotlib
```

Pengguna Windows dapat memasang Python dari `https://www.python.org` lalu menjalankan perintah `pip` yang sama; langkah lengkapnya ada di Lampiran A. Program Python disimpan sebagai berkas `.py` dan dijalankan dengan `python3 nama_berkas.py`. Contoh dalam kuliah ini diuji dengan Python 3.11, NumPy 2.4, dan Matplotlib 3.10.

Program Python pertama hanya satu baris:

```
print("Hello, Teknik Fisika!")
```

```
Hello, Teknik Fisika!
```

Tidak ada `#include`, fungsi `main`, titik koma, maupun langkah kompilasi.

9.2 Sintaks dasar: perbandingan dengan C

Konsep	C	Python
deklarasi	`double m = 2.0;`	`m = 2.0`
blok	kurung kurawal `{ }`	indentasi dan titik dua `:`
akhir pernyataan	titik koma `;`	akhir baris
pangkat	`pow(x, y)`	`x ** y`
pembagian	`7 / 2` bernilai `3`	`7 / 2` bernilai `3.5`; `7 // 2` bernilai `3`
logika	`&&`, `||`, `!`	`and`, `or`, `not`
keluaran	`printf("%.2f\n", x);`	`print(f"{x:.2f}")`
masukan	`scanf("%lf", &x);`	`x = float(input())`

9.2.1 Variabel dan tipe dinamis

Variabel Python tidak dideklarasikan; tipenya ditentukan oleh nilai yang disimpan pada saat itu (**tipe dinamis**). Fungsi `type` menunjukkan tipe suatu nilai.

```
mass = 2.0          # float (setara double di C)
count = 5           # int
name = "sensor-A"   # str
active = True       # bool

print(type(mass), type(count), type(name), type(active))
print(7 / 2, 7 // 2, 7 % 2, 2 ** 10)
print(2 ** 100)
print(0.1 + 0.2)
```

```
<class 'float'> <class 'int'> <class 'str'> <class 'bool'>
3.5 3 1 1024
1267650600228229401496703205376
0.30000000000000004
```

Tiga hal patut dicatat. Pertama, operator `/` selalu menghasilkan bilangan real, sehingga jebakan pembagian bilangan bulat dari Kuliah 3 tidak muncul; pembagian bilangan bulat

ditulis eksplisit dengan `//`. Kedua, bilangan bulat Python tidak mempunyai batas ukuran tetap, sehingga 2^{100} dihitung secara eksak tanpa *overflow*. Ketiga, bilangan real Python adalah *floating-point* IEEE 754 presisi ganda—persis `double` di C—sehingga $0.1 + 0.2$ tetap tidak tepat sama dengan 0.3. Konsep representasi dari Kuliah 1 tetap berlaku.

9.2.2 Masukan, keluaran, dan percabangan

Fungsi `input` membaca satu baris teks dari pengguna; teks tersebut diubah menjadi bilangan dengan `float` atau `int`. Keluaran diformat dengan *f-string*: ekspresi di dalam `{}` diganti nilainya, dan `:.3f` berarti tiga digit di belakang titik desimal, sama seperti `%.3f` pada `printf`. Percabangan menggunakan `if`, `elif`, dan `else`; blok ditentukan oleh **indentasi**, bukan kurung kurawal. Python juga mengizinkan perbandingan berantai seperti `20 <= T <= 30`, yang di C harus dipecah dengan `&&`.

```
temperature = float(input("Temperature (C): "))

if temperature < 20:
    status = "DINGIN"
elif temperature <= 30:
    status = "NORMAL"
else:
    status = "PANAS"

print(f"T = {temperature:.1f} C -> {status}")
```

```
Temperature (C): 27.4
T = 27.4 C -> NORMAL
```

Bandingkan dengan program C yang sama pada Kuliah 4: algoritmanya identik, tetapi kodenya jauh lebih ringkas.

9.2.3 Pengulangan

Pengulangan `for` Python menelusuri anggota suatu barisan. Fungsi `range(a, b, langkah)` menghasilkan bilangan bulat dari `a` sampai *sebelum* `b`, sehingga `range(0, n)` setara dengan `for (int i = 0; i < n; i++)`. Batas atas yang tidak termasuk ini konsisten dengan indeks larik yang dimulai dari nol. Pengulangan `while` bekerja seperti di C.

```
x0, v0, a = 2.0, 5.0, 3.0      # m, m/s, m/s^2

print(" t(s)    x(m)")
for t in range(0, 6):
    x = x0 + v0 * t + 0.5 * a * t**2
    print(f"{t:4d} {x:8.2f}")

# metode Babilonia dengan while
s, x, tol = 2.0, 1.0, 1e-12
iterations = 0
while True:
    x_new = 0.5 * (x + s / x)
```

```
    iterations += 1
    if abs(x_new - x) < tol or iterations >= 100:
        break
    x = x_new
print(f"sqrt(2) ~= {x_new:.15f} after {iterations} iterations")
```

```
 t(s)    x(m)
   0     2.00
   1     8.50
   2    18.00
   3    30.50
   4    46.00
   5    64.50
sqrt(2) ~= 1.414213562373095 after 6 iterations
```

Hasilnya sama dengan program C pada Kuliah 4 dan 5. Python tidak mempunyai `do-while`; pola "jalankan minimal sekali" ditulis dengan `while True` dan `break`.

9.3 Fungsi, *list*, dan berkas

Fungsi didefinisikan dengan `def`. Tipe parameter dan nilai kembalian tidak perlu dideklarasikan, dan tidak diperlukan prototipe. Teks di antara tanda kutip tiga di awal fungsi adalah *docstring* yang menjelaskan kegunaan fungsi. Seperti di C, variabel di dalam fungsi bersifat lokal.

```
def kinetic_energy(mass, velocity):
    """Energi kinetik (J) dari massa (kg) dan kecepatan (m/s)."""
    return 0.5 * mass * velocity**2


def momentum(mass, velocity):
    return mass * velocity


print(f"E = {kinetic_energy(2.0, 3.0):.3f} J")
print(f"p = {momentum(2.0, 3.0):.3f} kg m/s")
```

```
E = 9.000 J
p = 6.000 kg m/s
```

9.3.1 *List*

List adalah wadah serbaguna Python. Berbeda dengan larik C, *list* mengetahui panjangnya sendiri (`len`), dapat bertambah dengan `append`, dan dapat berisi data berbagai tipe. Indeks dimulai dari nol, dan indeks negatif menghitung dari belakang: `data[-1]` adalah elemen terakhir. Fungsi bawaan `sum`, `min`, dan `max` langsung menghasilkan statistik dasar.

```
voltage = [2.4, 3.1, 2.8, 4.0]
voltage.append(3.5)
```

```
print("n     =", len(voltage))
print("first =", voltage[0], " last =", voltage[-1])
print(f"mean  = {sum(voltage) / len(voltage):.3f} V")
print("min   =", min(voltage), " max =", max(voltage))

for i, v in enumerate(voltage):
    print(f"V[{i}] = {v:.1f}")
```

```
n     = 5
first = 2.4  last = 3.5
mean  = 3.160 V
min   = 2.4  max = 4.0
V[0] = 2.4
V[1] = 3.1
V[2] = 2.8
V[3] = 4.0
V[4] = 3.5
```

Fungsi `enumerate` memberikan indeks dan nilai sekaligus. Bandingkan dengan program statistik pada Kuliah 6 yang memerlukan tiga fungsi dan parameter ukuran larik.

9.3.2 Membaca berkas teks

Pola buka–baca–tutup dari Kuliah 7 tetap berlaku. Konstruksi `with` menutup berkas secara otomatis ketika bloknya selesai, bahkan jika terjadi galat. Berkas `Data/voltage.dat` memuat dua kolom yang dipisahkan spasi, dengan baris komentar yang diawali `#`.

```
time, voltage = [], []

with open("Data/voltage.dat", "r", encoding="utf-8") as f:
    for line in f:
        if line.startswith("#"):
            continue
        parts = line.split()
        time.append(float(parts[0]))
        voltage.append(float(parts[1]))

print("samples:", len(time))
print(f"V(0) = {voltage[0]:.3f} V, V(end) = {voltage[-1]:.3f} V")
print(f"drop = {100 * (1 - voltage[-1] / voltage[0]):.1f} %")
```

```
samples: 11
V(0) = 5.000 V, V(end) = 2.483 V
drop = 50.3 %
```

Metode `split` memecah baris berdasarkan spasi, dan `float` mengubah teks menjadi bilangan. Pendekatan manual ini cukup untuk data kecil; untuk data numerik, NumPy menyediakan cara yang jauh lebih praktis.

9.4 NumPy: larik numerik

NumPy menyediakan `ndarray`, larik numerik yang mirip larik C: semua elemennya bertipe sama (biasanya `float64`, yaitu `double`) dan tersimpan berurutan di memori. Keunggulannya adalah **operasi elemen demi elemen**: ekspresi matematika diterapkan ke seluruh larik sekaligus tanpa pengulangan eksplisit. NumPy diimpor dengan nama singkat `np`.

```
import numpy as np

mass = np.array([1.0, 2.0, 3.0, 4.0])        # kg
velocity = np.array([2.0, 3.0, 1.5, 0.5])    # m/s

energy = 0.5 * mass * velocity**2             # tanpa loop
print("E       =", energy)
print("E_total =", energy.sum(), "J")

# perbandingan: list biasa memerlukan loop
energy_list = [0.5 * m * v**2 for m, v in zip([1.0, 2.0, 3.0, 4.0], [2.0, 3.0, 1.5, 0.5])
    ↪ ]
print("list    =", energy_list)
```

```
E       = [2.    9.    3.375 0.5  ]
E_total = 14.875 J
list    = [2.0, 9.0, 3.375, 0.5]
```

Ekspresi `0.5 * mass * velocity**2` terlihat sama seperti rumus fisikanya, tetapi dihitung untuk keempat partikel. Di balik layar, NumPy menjalankan pengulangan dalam kode C yang telah dikompilasi, sehingga jauh lebih cepat daripada pengulangan Python. Perhatikan perbedaan dengan *list*: `[1, 2] * 2` menghasilkan `[1, 2, 1, 2]`, bukan `[2, 4]`. *List* adalah wadah umum; larik NumPy adalah objek matematis.

9.4.1 Membuat larik dan mengevaluasi model

Fungsi `np.linspace(a, b, n)` membuat n titik berjarak sama dari a sampai b (termasuk b), sedangkan `np.arange(a, b, langkah)` bekerja seperti `range` untuk bilangan real. Untuk langkah real, `linspace` lebih aman karena banyaknya titik ditentukan secara eksplisit—masalah akumulasi pembulatan pada Kuliah 5 tidak muncul. Model fisika dapat langsung dievaluasi pada seluruh titik:

```
import numpy as np

t = np.linspace(0.0, 2.0, 21)            # 21 titik, langkah 0.1 s
x = 2.0 + 5.0 * t + 0.5 * 3.0 * t**2     # x(t) = x0 + v0 t + a t^2 / 2

print("number of points:", t.size)
print("t[:4] =", t[:4])
print("x[-1] =", x[-1], "m")
print(f"mean = {x.mean():.3f}, std = {x.std():.3f}, max = {x.max():.3f}")
```

```
number of points: 21
```

```
t[:4] = [0.  0.1 0.2 0.3]
x[-1] = 18.0 m
mean = 9.050, std = 4.869, max = 18.000
```

Notasi `t[:4]` (*slicing*) mengambil empat elemen pertama. Metode `mean`, `std`, `min`, `max`, dan `sum` menghasilkan statistik dasar.

9.4.2 Penyaringan dengan kondisi Boolean

Perbandingan terhadap larik menghasilkan larik Boolean, yang dapat digunakan sebagai ***mask*** untuk memilih elemen. Pola pencacah dan penyaringan dari Kuliah 5 menjadi satu baris.

```python
import numpy as np

T = np.array([29.5, 30.2, 28.9, 31.1, 30.0, 19.4, 33.0])

high = T > 30.0
print("mask       :", high)
print("above 30 C :", T[high])
print("count      :", np.count_nonzero(high))
print(f"percentage : {100 * np.mean(high):.1f} %")

outside = (T < 20.0) | (T > 30.0)
print("outside 20-30 C:", T[outside])
```

```
mask       : [False  True False  True False False  True]
above 30 C : [30.2 31.1 33. ]
count      : 3
percentage : 42.9 %
outside 20-30 C: [30.2 31.1 19.4 33. ]
```

Untuk larik, operator logika ditulis `&` (AND), `|` (OR), dan `~` (NOT), dan setiap kondisi harus diberi tanda kurung. `np.mean` dari larik Boolean adalah fraksi elemen yang bernilai benar.

9.5 Membaca data dan menghitung selisih

Fungsi `np.loadtxt` membaca berkas numerik sederhana dan otomatis melewati baris komentar `#`. Untuk CSV dengan *header*, `np.genfromtxt` dengan `names=True` memungkinkan kolom diakses dengan namanya, sehingga makna setiap kolom tetap terjaga. Atribut `shape` dan `size` perlu diperiksa untuk memastikan data terbaca sesuai harapan.

```python
import numpy as np

volt = np.loadtxt("Data/voltage.dat")
print("voltage.dat shape:", volt.shape)

data = np.genfromtxt("Data/motion.csv", delimiter=",", names=True)
print("columns:", data.dtype.names)
t = data["time_s"]
```

```
x = data["position_m"]
print("samples:", t.size)
```

```
voltage.dat shape: (11, 2)
columns: ('time_s', 'position_m')
samples: 11
```

Larik `volt` berbentuk 11×2: kolom `volt[:, 0]` berisi waktu dan `volt[:, 1]` berisi tegangan.

Fungsi `np.diff` menghitung selisih elemen berurutan, $\Delta x_i = x_{i+1} - x_i$, dan menghasilkan larik dengan satu elemen lebih sedikit—persis $N - 1$ interval seperti pada Kuliah 8. Dengan `np.diff`, validasi urutan waktu dan perhitungan kecepatan interval masing-masing hanya membutuhkan satu baris:

```
import numpy as np

data = np.genfromtxt("Data/motion.csv", delimiter=",", names=True)
t, x = data["time_s"], data["position_m"]

dt = np.diff(t)
if not np.all(dt > 0):
    raise ValueError("time must increase strictly")

v = np.diff(x) / dt
k = np.argmax(np.abs(v))
first = np.argmax(np.abs(v) > 3.0)

print("v =", np.round(v, 4))
print(f"mean interval velocity  = {v.mean():.4f} m/s")
print(f"global average velocity = {(x[-1] - x[0]) / (t[-1] - t[0]):.4f} m/s")
print(f"max |v| = {abs(v[k]):.4f} m/s in [{t[k]:.2f}, {t[k+1]:.2f}] s")
print(f"|v| > 3 m/s first in [{t[first]:.2f}, {t[first+1]:.2f}] s")
```

```
v = [1.375 1.725 2.075 2.425 2.775 3.125 3.475 3.825 4.175 4.525]
mean interval velocity  = 2.9500 m/s
global average velocity = 2.9500 m/s
max |v| = 4.5250 m/s in [4.50, 5.00] s
|v| > 3 m/s first in [2.50, 3.00] s
```

Hasilnya identik dengan program C pada Kuliah 8, yang membutuhkan sekitar 150 baris. Fungsi `np.argmax` mengembalikan indeks elemen terbesar; pada larik Boolean, ia mengembalikan indeks `True` pertama. (Jika tidak ada elemen yang memenuhi, `argmax` mengembalikan 0, sehingga dalam program yang lebih hati-hati kita perlu memeriksa `np.any(...)` terlebih dahulu.) Pernyataan `raise ValueError(...)` menghentikan program dengan pesan kesalahan, setara dengan mencetak pesan lalu `return 1` di C.

9.6 Matplotlib: grafik ilmiah

Grafik adalah alat analisis, bukan sekadar hiasan laporan. Tren, anomali, dan kesesuaian dengan model sering langsung terlihat dari grafik, bahkan sebelum statistik dihitung. Matplotlib menggunakan dua objek utama: `figure` (seluruh gambar) dan `axes` (satu sistem sumbu tempat data digambar). Keduanya dibuat dengan `plt.subplots`.

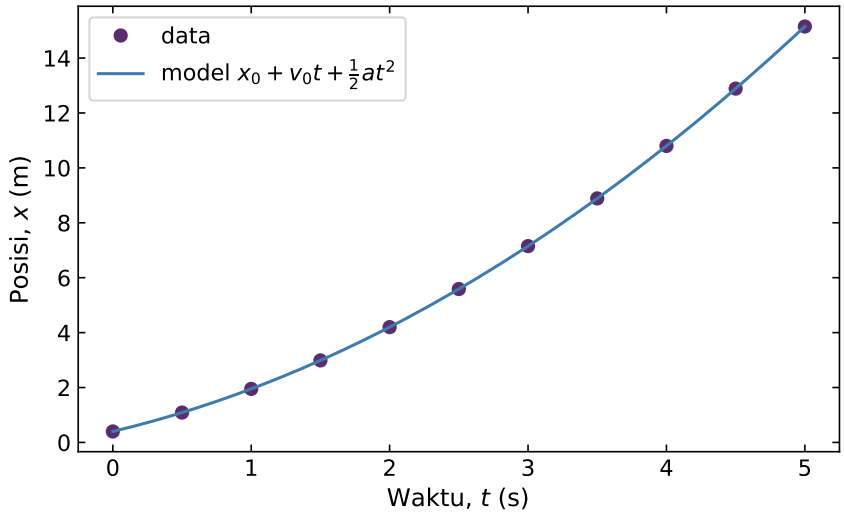
```
import numpy as np
import matplotlib.pyplot as plt

data = np.genfromtxt("Data/motion.csv", delimiter=",", names=True)
t, x = data["time_s"], data["position_m"]

t_model = np.linspace(0.0, 5.0, 200)
x_model = 0.4 + 1.2 * t_model + 0.5 * 0.7 * t_model**2

fig, ax = plt.subplots(figsize=(6.0, 3.8))
ax.plot(t, x, "o", color="#5A2D6E", label="data")
ax.plot(t_model, x_model, "-", color="#3F7CAC",
        label=r"model $x_0 + v_0 t + \frac{1}{2} a t^2$")
ax.set_xlabel("Waktu, $t$ (s)", fontsize=12)
ax.set_ylabel("Posisi, $x$ (m)", fontsize=12)
ax.tick_params(direction="in", top=True, right=True, labelsize=11)
ax.legend(fontsize=11)
fig.tight_layout()
fig.savefig("gerak-posisi.pdf")
print("saved gerak-posisi.pdf")
```

```
saved gerak-posisi.pdf
```



Beberapa kebiasaan untuk grafik ilmiah yang layak:

- **Label sumbu selalu memuat besaran dan satuannya**, misalnya “Waktu, t (s)”. Matplotlib menerima notasi matematika di antara tanda `$`.
- **Data diskret digambar sebagai titik**, sedangkan model kontinu sebagai garis. Dengan begitu pembaca dapat membedakan apa yang diukur dan apa yang dihitung.
- **Legenda** diperlukan jika ada lebih dari satu kurva.
- **Ukuran huruf** harus tetap terbaca setelah gambar diperkecil di dalam laporan.
- **Tanda skala** (*tick*) yang menghadap ke dalam pada keempat sisi adalah gaya yang lazim dalam publikasi fisika.

Fungsi `savefig` menyimpan grafik. Format PDF atau SVG menghasilkan gambar vektor yang tetap tajam pada perbesaran apa pun dan cocok untuk dokumen LaTeX; format PNG dengan `dpi=300` cocok untuk dokumen pengolah kata dan presentasi. Saat bekerja interaktif, `plt.show()` menampilkan grafik di layar.

Grafik di atas menggabungkan data dan model. Model $x(t) = 0.4 + 1.2t + 0.35t^2$ menggunakan parameter yang diperoleh dari analisis kecepatan pada Kuliah 8 ($x_0 = 0.4$ m, $v_0 = 1.2$ m/s, $a = 0.7$ m/s^2), dan kurvanya melewati semua titik data. Kesesuaian visual seperti ini adalah pemeriksaan fisis yang sangat kuat.

9.7 Studi kasus: data pendinginan dan gelombang

9.7.1 Pendinginan benda

Berkas `Data/cooling.csv` memuat temperatur sebuah benda (°C) terhadap waktu (menit). Menurut hukum pendinginan Newton,

$$T(t) = T_{\text{ruang}} + \left(T_0 - T_{\text{ruang}}\right) e^{-kt},$$

sehingga temperatur seharusnya turun makin lambat menuju temperatur ruang. Laju perubahan temperatur setiap interval dihitung dengan `np.diff`; nilainya negatif karena temperatur menurun, sehingga interval dengan pendinginan tercepat dicari dengan `np.argmin`.

```
import numpy as np
import matplotlib.pyplot as plt

data = np.genfromtxt("Data/cooling.csv", delimiter=",", names=True)
t, T = data["time_min"], data["temperature_C"]

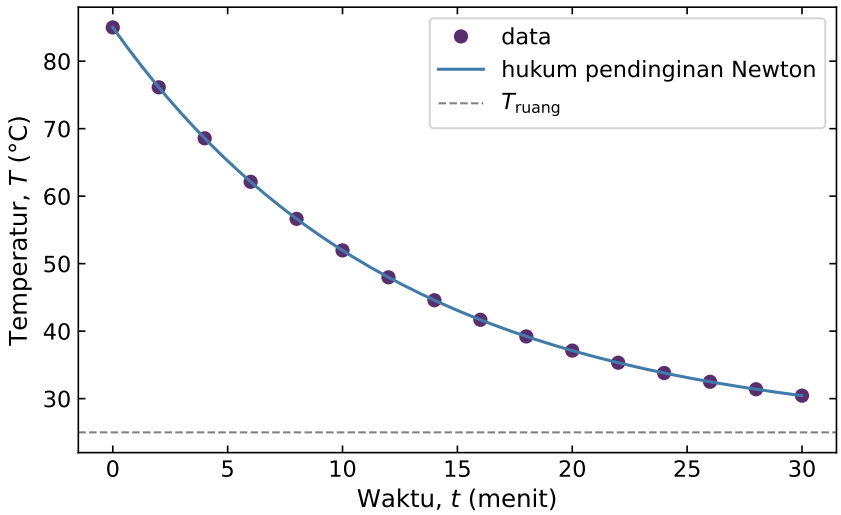
rate = np.diff(T) / np.diff(t)          # C/min
i = np.argmin(rate)

print(f"samples          : {T.size}")
print(f"initial / final  : {T[0]:.2f} C / {T[-1]:.2f} C")
print(f"all rates < 0    : {np.all(rate < 0)}")
print(f"fastest cooling  : {rate[i]:.3f} C/min in [{t[i]:.0f}, {t[i+1]:.0f}] min")
print(f"slowest cooling  : {rate[-1]:.3f} C/min in [{t[-2]:.0f}, {t[-1]:.0f}] min")
```

```
T_room, k = 25.0, 0.08                  # parameter model
t_model = np.linspace(0, 30, 200)
T_model = T_room + (T[0] - T_room) * np.exp(-k * t_model)

fig, ax = plt.subplots(figsize=(6.0, 3.8))
ax.plot(t, T, "o", color="#5A2D6E", label="data")
ax.plot(t_model, T_model, "-", color="#3F7CAC", label="hukum pendinginan Newton")
ax.axhline(T_room, color="gray", linestyle="--", linewidth=1, label=r"$T_{\mathrm{ruang}}
    ↪ $")
ax.set_xlabel("Waktu, $t$ (menit)", fontsize=12)
ax.set_ylabel("Temperatur, $T$ (°C)", fontsize=12)
ax.tick_params(direction="in", top=True, right=True, labelsize=11)
ax.legend(fontsize=11)
fig.tight_layout()
fig.savefig("pendinginan.pdf")
```

```
samples          : 16
initial / final  : 85.00 C / 30.44 C
all rates < 0    : True
fastest cooling  : -4.436 C/min in [0, 2] min
slowest cooling  : -0.472 C/min in [28, 30] min
```



Semua laju bernilai negatif dan besarnya makin kecil seiring waktu, sesuai dengan perilaku eksponensial. Kurva model dengan $T_{\mathrm{ruang}} = 25\,^{\circ}\mathrm{C}$ dan $k = 0.08\ \mathrm{min}^{-1}$ melewati semua titik data. (Data ini memang dibangkitkan dari model tersebut; menentukan parameter T_{ruang} dan k secara otomatis dari data nyata disebut *fitting* dan dibahas dalam mata kuliah metode numerik.)

9.7.2 Gelombang sinus

Berkas `Data/wave.csv` merepresentasikan simpangan gelombang $y(t) = A\sin(2\pi f t)$ dengan amplitudo $A = 0.03$ m dan frekuensi $f = 2$ Hz. Untuk sinus ideal, nilai RMS-nya adalah $A/\sqrt{2}$. Banyaknya perubahan tanda berurutan menunjukkan berapa kali gelombang melewati titik setimbang.

```
import numpy as np
import matplotlib.pyplot as plt

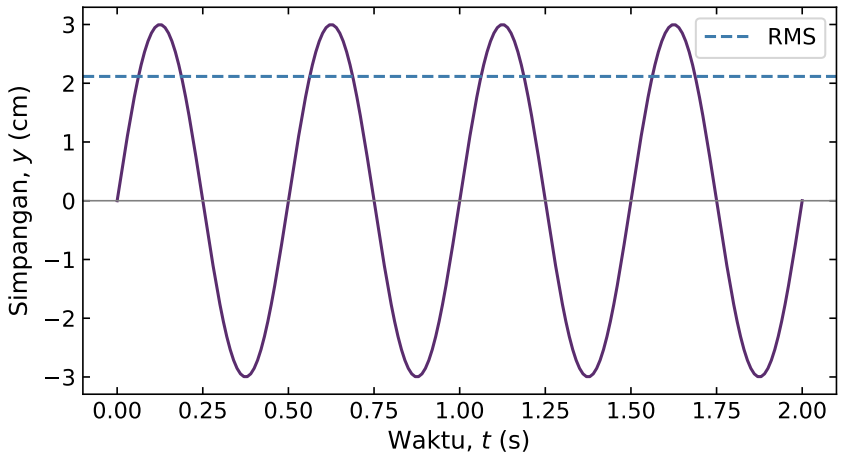
data = np.genfromtxt("Data/wave.csv", delimiter=",", names=True)
t, y = data["time_s"], data["displacement_m"]

amplitude = np.max(np.abs(y))
rms = np.sqrt(np.mean(y**2))
signs = np.sign(y)
signs = signs[signs != 0]               # abaikan sampel yang tepat nol
crossings = np.count_nonzero(signs[:-1] * signs[1:] < 0)

print(f"samples   = {y.size}, duration = {t[-1] - t[0]:.2f} s")
print(f"amplitude = {amplitude:.6f} m")
print(f"RMS       = {rms:.6f} m")
print(f"A/sqrt(2) = {amplitude / np.sqrt(2):.6f} m")
print(f"sign changes = {crossings}")

fig, ax = plt.subplots(figsize=(6.0, 3.4))
ax.plot(t, 100 * y, "-", color="#5A2D6E", linewidth=1.5)
ax.axhline(0, color="gray", linewidth=0.8)
ax.axhline(100 * rms, color="#3F7CAC", linestyle="--", label="RMS")
ax.set_xlabel("Waktu, $t$ (s)", fontsize=12)
ax.set_ylabel("Simpangan, $y$ (cm)", fontsize=12)
ax.tick_params(direction="in", top=True, right=True, labelsize=11)
ax.legend(fontsize=11, loc="upper right")
fig.tight_layout()
fig.savefig("gelombang.pdf")
```

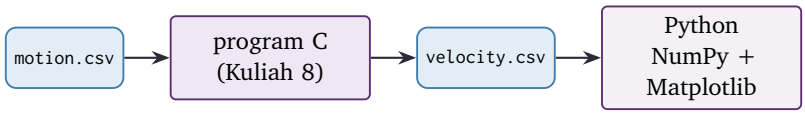
```
samples   = 201, duration = 2.00 s
amplitude = 0.029941 m
RMS       = 0.021160 m
A/sqrt(2) = 0.021171 m
sign changes = 7
```



RMS numerik berbeda dari $A/\sqrt{2}$ hanya sekitar 0.05%. Amplitudo terukur 0.029941 m sedikit lebih kecil dari 0.03 m karena tidak ada titik sampel yang jatuh tepat di puncak gelombang. Selama 2 s, gelombang 2 Hz menempuh empat periode dan melewati titik setimbang dua kali setiap periode. Perpotongan di dalam selang terbuka $(0, 2)$ s berjumlah $2 \times 4 - 1 = 7$, karena perpotongan pada $t = 0$ dan $t = 2$ s tepat berada di ujung data. Perhatikan pula baris `signs[signs != 0]`: dengan langkah 0.01 s, setiap perpotongan nol jatuh *tepat* pada titik sampel bernilai nol, sehingga tanpa baris tersebut tidak ada satu pun perubahan tanda yang terdeteksi. Detail semacam ini mengingatkan bahwa definisi besaran diskret harus selalu diperiksa terhadap teorinya. Simpangan digambar dalam sentimeter agar angka pada sumbu mudah dibaca—satuan pada label sumbu harus mengikuti perubahan ini.

9.8 C dan Python dalam satu alur kerja

C dan Python tidak harus dipilih salah satu. Dalam praktik riset dan industri, keduanya sering dipadukan. Program C yang cepat dan dekat dengan perangkat keras—misalnya pada sistem akuisisi data atau simulasi berat—menulis hasilnya ke berkas CSV, lalu Python membaca berkas tersebut untuk analisis lanjutan dan visualisasi. Berkas teks dengan skema yang jelas menjadi "antarmuka" di antara keduanya.



Program Python berikut membaca `velocity.csv` yang dihasilkan program C pada Kuliah 8 dan menggambarkan kecepatan setiap interval pada titik tengahnya. Dari kemiringan garisnya, percepatan diperkirakan dengan `np.polyfit`, yang mencocokkan polinomial derajat satu terhadap data.

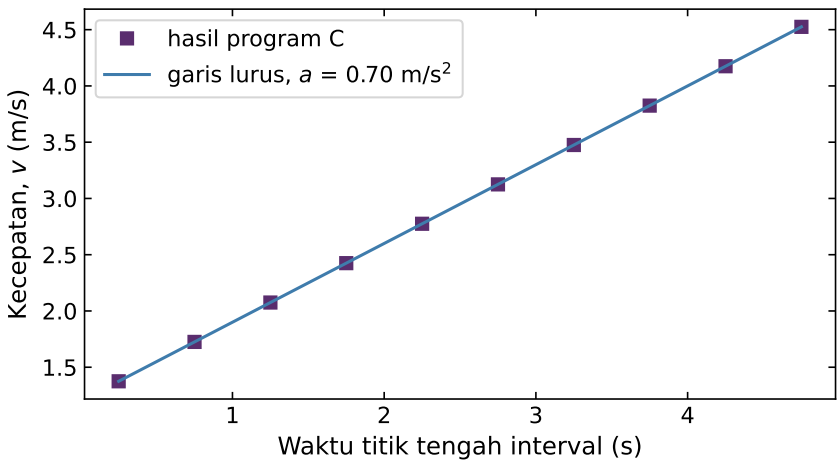
```python
import numpy as np
import matplotlib.pyplot as plt

data = np.genfromtxt("velocity.csv", delimiter=",", names=True)
t_mid = 0.5 * (data["time_start_s"] + data["time_end_s"])
v = data["velocity_m_per_s"]

slope, intercept = np.polyfit(t_mid, v, 1)
print(f"a  = {slope:.4f} m/s^2")
print(f"v0 = {intercept:.4f} m/s")

fig, ax = plt.subplots(figsize=(6.0, 3.4))
ax.plot(t_mid, v, "s", color="#5A2D6E", label="hasil program C")
ax.plot(t_mid, slope * t_mid + intercept, "-", color="#3F7CAC",
        label=f"garis lurus, $a$ = {slope:.2f} m/s$^2$")
ax.set_xlabel("Waktu titik tengah interval (s)", fontsize=12)
ax.set_ylabel("Kecepatan, $v$ (m/s)", fontsize=12)
ax.tick_params(direction="in", top=True, right=True, labelsize=11)
ax.legend(fontsize=11)
fig.tight_layout()
fig.savefig("kecepatan.pdf")
```

```
a  = 0.7000 m/s^2
v0 = 1.2000 m/s
```



Percepatan 0.7 m/s^2 dan kecepatan awal 1.2 m/s sesuai dengan analisis manual pada Kuliah 8. Kedua bahasa bekerja sama: C menangani pembacaan, validasi, dan perhitungan inti dengan kendali penuh; Python menangani eksplorasi, statistik lanjutan, dan grafik dengan kode yang ringkas.

9.8.1 Kesalahan umum saat beralih ke Python

- **Menulis titik koma dan kurung kurawal.** Blok Python ditentukan oleh indentasi; indentasi yang tidak konsisten menyebabkan `IndentationError`.
- **Lupa titik dua** di akhir baris `if`, `for`, `while`, dan `def`.
- **Salah memahami `range`.** `range(1, 10)` berhenti di 9, bukan 10.
- **Menganggap *list* sama dengan larik NumPy.** Operasi aritmetika pada *list* tidak bekerja elemen demi elemen.
- **Mengabaikan bentuk larik.** Periksa `shape` dan `size` setelah membaca data; kolom yang tertukar atau baris yang hilang tidak selalu menimbulkan galat.
- **Mengabaikan validasi data.** Kemudahan Python tidak menghapus kebutuhan memeriksa urutan waktu, rentang fisis, satuan, dan orde besaran.

9.9 Cek pemahaman

- Apa perbedaan utama cara menjalankan program C dan program Python?
- Apa yang dimaksud dengan tipe dinamis? Apakah 0.1 + 0.2 tepat sama dengan 0.3 di Python? Mengapa?
- Apa perbedaan `/` dan `//` di Python?
- Bagaimana blok kode ditentukan dalam Python?
- Mengapa `range(0, n)` setara dengan `for (int i = 0; i < n; i++)`?
- Apa perbedaan *list* dan larik NumPy? Apa hasil `[1, 2] * 2` dan `np.array([1, 2]) * 2`?
- Apa keuntungan konstruksi `with` saat membuka berkas?
- Kapan `np.loadtxt` dan `np.genfromtxt` digunakan? Mengapa ukuran hasil `np.diff` satu elemen lebih kecil?
- Bagaimana *Boolean mask* digunakan untuk menyaring data?
- Apa saja unsur grafik ilmiah yang layak? Kapan format vektor lebih baik daripada PNG?

9.10 Latihan

1. **Transisi C ke Python.** Tulis ulang program energi kinetik atau klasifikasi temperatur dari Kuliah 3 atau 4 dalam Python, lengkap dengan masukan pengguna dan satu pemeriksaan masukan. Jelaskan bagian yang berubah secara sintaks dan bagian yang tetap sama secara algoritmik.
2. **Gerak dengan NumPy.** Baca `Data/motion.csv`, hitung kecepatan interval dengan `np.diff`, lalu hitung minimum, maksimum, rata-rata aritmetika, dan kecepatan rata-rata global. Hapus satu baris data di tengah berkas (sehingga Δt tidak seragam) dan jelaskan mengapa kedua jenis rata-rata kini berbeda.
3. **Analisis pendinginan.** Dari `Data/cooling.csv`, hitung temperatur awal, akhir, minimum, maksimum, dan rata-rata, serta interval dengan penurunan temperatur

terbesar. Gambarkan $\ln(T - T_{\text{ruang}})$ terhadap t dengan $T_{\text{ruang}} = 25\,^{\circ}\text{C}$. Mengapa grafiknya berupa garis lurus, dan berapa kemiringannya?

4. **Gelombang dan RMS.** Dari `Data/wave.csv`, hitung amplitudo dan RMS, bandingkan dengan $A/\sqrt{2}$, dan hitung frekuensi gelombang dari banyaknya perpotongan nol. Simpan grafiknya dalam format PNG 300 dpi.
5. **Peluruhan tegangan.** Baca `Data/voltage.dat` dengan `np.loadtxt`. Hitung persentase penurunan tegangan, gambarkan $V(t)$ pada skala biasa dan skala logaritmik (`ax.set_yscale("log")`), lalu periksa apakah trennya konsisten dengan peluruhan eksponensial $V = V_0 e^{-t/\tau}$. Perkirakan τ.
6. ***Mask* data sensor.** Buat larik 50 temperatur sintetis dengan generator acak NumPy, yaitu `rng = np.random.default_rng(1)` lalu `T = rng.normal(25, 4, 50)`. Gunakan *Boolean mask* untuk mengambil data di luar rentang aman 20–30 °C, hitung banyak dan persentasenya, lalu gambarkan data terhadap indeks sampel dengan titik di luar rentang diberi warna berbeda.
7. **Model dan data berderau.** Bangkitkan data gerak $x(t) = x_0 + v_0 t + \frac{1}{2}at^2$ dengan parameter pilihan Anda, lalu tambahkan derau acak kecil. Gambarkan data dan model dalam satu grafik dengan legenda, hitung galat RMS antara data dan model, dan jelaskan maknanya.
8. **Daya listrik.** Baca `electrical.csv` dari Kuliah 7 dengan NumPy, hitung $P = VI$, daya maksimum, daya rata-rata, dan energi dengan aturan trapesium (`np.trapezoid`). Gambarkan daya terhadap waktu dan simpan hasilnya ke CSV baru dengan `np.savetxt`.
9. **Optika sederhana.** Buat data intensitas sintetis $I(x) = I_0 \exp\!\left[-(x - x_0)^2/(2\sigma^2)\right]$. Cari posisi puncak, intensitas relatif, serta lebar penuh pada setengah maksimum (FWHM) secara numerik, lalu bandingkan dengan nilai teoritis $2\sqrt{2\ln 2}\,\sigma$.
10. **Proyek mini Python ilmiah.** Pilih satu persoalan dari mekanika, listrik, termal, optika, atau gelombang. Bangun alur kerja data → NumPy → analisis → grafik → berkas hasil dengan sekurang-kurangnya tiga operasi analisis, satu grafik ilmiah yang layak, dan pemeriksaan kewajaran berdasarkan satuan, tanda, rentang, atau nilai referensi.

9.11 Rangkuman

- Python adalah bahasa tingkat tinggi yang dijalankan oleh *interpreter*. Ekosistem NumPy dan Matplotlib menjadikannya alat utama untuk analisis dan visualisasi data ilmiah.
- Konsep algoritmik tetap sama dengan C; yang berubah adalah sintaks: tipe dinamis, blok dengan indentasi, `/` yang selalu menghasilkan bilangan real, `**` untuk pangkat, dan *f-string* untuk keluaran.
- Bilangan real Python adalah `double` IEEE 754, sehingga sifat *floating-point* dari Kuliah 1 tetap berlaku.
- Fungsi didefinisikan dengan `def`; *list* adalah wadah serbaguna yang mengetahui panjangnya sendiri; `with open(...)` menutup berkas secara otomatis.

- Larik NumPy mendukung operasi elemen demi elemen, statistik bawaan, dan *Boolean mask*, sehingga banyak pola iteratif dari Kuliah 5 cukup ditulis dalam satu baris.
- `np.loadtxt` dan `np.genfromtxt` membaca data; `np.diff` menghitung selisih berurutan dengan $N - 1$ elemen.
- Grafik ilmiah yang layak mempunyai label sumbu dengan satuan, membedakan data dan model, memiliki legenda, dan disimpan dalam format yang sesuai.
- C dan Python saling melengkapi; berkas CSV dengan skema yang jelas menjadi penghubung keduanya. Validasi data dan pemeriksaan fisis tetap menjadi tanggung jawab pemrogram, apa pun bahasanya.

LAMPIRAN

A

Menyiapkan Lingkungan Pemrograman

Lampiran ini menjelaskan cara memasang *compiler* C (GCC), editor Visual Studio Code (VS Code), dan Python beserta pustaka ilmiahnya. Terdapat dua jalur untuk pengguna Windows: **MSYS2** (GCC berjalan langsung di Windows) dan **WSL** (*Windows Subsystem for Linux*, GCC berjalan di dalam Linux). Penulis menyarankan WSL atau Linux karena lingkungannya sama dengan yang digunakan pada sebagian besar contoh buku ini. Pengguna Linux cukup mengikuti Subbab A.2.2.

Tampilan menu dan nama tombol dapat sedikit berbeda antarversi perangkat lunak; yang penting adalah urutan langkahnya. Setiap tahap diakhiri dengan langkah **verifikasi**. Jangan melanjutkan ke tahap berikutnya sebelum verifikasi berhasil.

A.1 GCC di Windows melalui MSYS2

A.1.1 Memasang MSYS2

1. Kunjungi situs resmi `https://www.msys2.org` dan unduh berkas penginstal (`msys2-x86_64-...exe`).
2. Jalankan penginstal dan gunakan direktori instalasi bawaan, `C:\msys64`.
3. Selesaikan proses instalasi.

Verifikasi: folder `C:\msys64` beserta subfoldernya telah terbentuk.

A.1.2 Memasang GCC

1. Buka terminal **MSYS2 UCRT64** dari menu Start.
2. Pasang *compiler* GCC, G++, *debugger* GDB, dan pustaka pendukung:

   ```
   pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
   ```

3. Saat muncul pertanyaan `Enter a selection (default=all)`, tekan **Enter** untuk memilih semua paket.

4. Saat diminta konfirmasi `Proceed with installation? [Y/n]`, ketik `Y` lalu tekan **Enter**, dan tunggu sampai selesai.

Verifikasi: pada terminal MSYS2 UCRT64, perintah `gcc -version` menampilkan versi GCC.

A.1.3 Menambahkan GCC ke variabel `Path`

Agar GCC dapat dipanggil dari terminal mana pun, termasuk terminal VS Code, lokasinya perlu didaftarkan ke variabel sistem `Path`.

1. Buka menu Start, ketik **Environment Variables**, lalu pilih **Edit the system environment variables**.
2. Klik tombol **Environment Variables...**.
3. Pada **System variables** (atau **User variables** jika tidak memiliki hak administrator), pilih variabel `Path`, lalu klik **Edit...**.
4. Klik **New** dan masukkan `C:\msys64\ucrt64\bin`.
5. Klik **OK** pada semua jendela.

Verifikasi: buka jendela *baru* Command Prompt atau PowerShell (jendela lama tidak memuat variabel baru), lalu jalankan

```
gcc --version
gdb --version
```

Jika versi ditampilkan tanpa pesan "perintah tidak dikenal", konfigurasi berhasil.

A.2 GCC melalui WSL

A.2.1 Memasang WSL

Buka PowerShell sebagai administrator dan jalankan

```
wsl --install
```

Nyalakan ulang komputer jika diminta, lalu tentukan nama pengguna (*username*) dan kata sandi (*password*) untuk distribusi Linux (biasanya Ubuntu). Saat mengetik kata sandi di Linux, tidak ada karakter yang tampil di layar; setiap ketikan tetap terbaca, jadi ketiklah dengan hati-hati.

A.2.2 Memasang GCC di Linux atau WSL

Buka terminal Linux (atau terminal WSL), perbarui daftar paket, dan pasang paket kompilasi:

```
sudo apt update
sudo apt install -y build-essential gdb
```

Verifikasi: `gcc -version` dan `gdb -version` menampilkan versi masing-masing.

A.3 Visual Studio Code

A.3.1 Ekstensi yang diperlukan

1. Unduh dan pasang VS Code dari `https://code.visualstudio.com/download`.
2. Buka menu Extensions dengan `Ctrl+Shift+X`.
3. Pasang ekstensi **C/C++** dari Microsoft.
4. Pengguna WSL juga memasang ekstensi **WSL** dari Microsoft.
5. Untuk Kuliah 9, pasang ekstensi **Python** dari Microsoft.

Verifikasi: pada daftar ekstensi terpasang, tombol ekstensi berubah menjadi **Disable** atau **Uninstall**.

A.3.2 Bekerja di dalam WSL

Buat direktori kerja di sistem berkas Linux, lalu buka VS Code dari sana:

```
mkdir -p ~/proyek-c && cd ~/proyek-c
code .
```

Pada eksekusi pertama, komponen VS Code Server dipasang otomatis di dalam Linux. Setelah itu, pojok kiri bawah jendela VS Code menampilkan nama distribusi WSL yang aktif. Cara lain adalah menekan `Ctrl+Shift+P` dan memilih **WSL: Connect to WSL**. Ekstensi C/C++ harus dipasang juga di sisi WSL dengan tombol **Install in WSL** agar fitur analisis kode dapat membaca berkas *header* Linux.

A.3.3 Menguji kompilasi

1. Buka folder proyek melalui **File** > **Open Folder…**.
2. Buat berkas `hello.c` berisi program pada Kuliah 1, lalu simpan dengan `Ctrl+S`.
3. Buka terminal terintegrasi melalui **Terminal** > **New Terminal**.
4. Kompilasi dan jalankan. Di Linux/WSL:

   ```
   gcc -Wall -Wextra hello.c -o hello
   ./hello
   ```

   Di Windows dengan MSYS2:

   ```
   gcc -Wall -Wextra hello.c -o hello.exe
   .\hello.exe
   ```

Verifikasi: terminal menampilkan `Hello, Teknik Fisika!` tanpa pesan galat.

A.4 Python dan pustaka ilmiah

Di Linux atau WSL, Python 3 biasanya sudah terpasang. Pasang modul lingkungan virtual, buat satu lingkungan untuk mata kuliah ini, lalu pasang NumPy dan Matplotlib:

```
sudo apt install -y python3-venv
python3 -m venv ~/venv-ilmiah
source ~/venv-ilmiah/bin/activate
pip install numpy matplotlib
```

Perintah `source ...` harus dijalankan setiap kali membuka terminal baru sebelum bekerja dengan Python. Di VS Code, pilih *interpreter* lingkungan tersebut melalui `Ctrl+Shift+P` > **Python: Select Interpreter**.

Pengguna Windows tanpa WSL dapat mengunduh penginstal Python dari `https://www.python.org/downloads`. Centang pilihan **Add python.exe to PATH** saat instalasi, lalu jalankan di PowerShell:

```
python -m venv venv-ilmiah
venv-ilmiah\Scripts\activate
pip install numpy matplotlib
```

Verifikasi: perintah berikut menampilkan versi NumPy dan Matplotlib tanpa galat.

```
python3 -c "import numpy, matplotlib; print(numpy.__version__, matplotlib.__version__)"
```